

ĐẠI HỌC QUỐC GIA TP.HCM
TRƯỜNG ĐẠI HỌC BÁCH KHOA TP.HCM
KHOA KHOA HỌC & KỸ THUẬT MÁY TÍNH



LUẬN VĂN TỐT NGHIỆP

**PHÁT TRIỂN NỀN TẢNG NHÚNG ĐA CHỨC NĂNG DỰA
TRÊN YOCTO PROJECT CHO CÁC ỨNG DỤNG TỰ
ĐỘNG HÓA TRONG CÔNG NGHIỆP**

HK252

Giảng viên hướng dẫn: TS.Lê Trọng Nhân

STT	Họ & Tên	MSSV	Lớp	Đánh giá
1	Đỗ Tuấn Anh	2210050	MT22KT01	100%
2	Dương Minh Hiếu	2210978	MT22KT01	100%
3	Nguyễn Hạo Duy	2210512	MT22KT01	100%

TP.Hồ Chí Minh, tháng 5 năm 2025

Lời cam đoan

Nhóm tác giả xin cam đoan rằng nhóm đã hoàn toàn thực hiện đồ án chuyên ngành này, dưới sự hướng dẫn của TS. Lê Trọng Nhân, Khoa Khoa Học và Kỹ Thuật Máy Tính, Trường Đại học Bách Khoa - Đại học Quốc gia. Kết quả nghiên cứu của nhóm chưa được công bố dưới bất kỳ hình thức nào trước đó.

Tất cả các tài liệu được sử dụng trong đồ án này được thu thập từ nhiều hệ thống và nguồn tham khảo khác nhau. Các tài liệu này đều được trích dẫn đầy đủ trong phần tài liệu tham khảo.

Nếu có bất kỳ trường hợp đạo văn nào, nhóm xin chịu mọi trách nhiệm. Đại học Quốc gia Thành phố Hồ Chí Minh sẽ không chịu trách nhiệm về bất kỳ vi phạm bản quyền nào xảy ra trong quá trình nghiên cứu của nhóm.

TP. Hồ Chí Minh, tháng 5 năm 2026

Sinh viên thực hiện

Dương Minh Hiếu

Đỗ Tuấn Anh

Nguyễn Hạo Duy

Lời cảm ơn

Để hoàn thành khóa luận tốt nghiệp này, em đã nhận được rất nhiều sự hỗ trợ, động viên và giúp đỡ quý báu từ quý Thầy Cô, gia đình, bạn bè và những người đã đồng hành cùng em trong suốt quá trình học tập và nghiên cứu. Em xin được gửi lời cảm ơn chân thành đến tất cả mọi người.

Trước hết, em xin trân trọng cảm ơn Ban Giám hiệu cùng quý Thầy Cô Trường Đại học Bách khoa – Đại học Quốc gia Thành phố Hồ Chí Minh nói chung và quý Thầy Cô thuộc Khoa Khoa học và Kỹ thuật Máy tính nói riêng. Những kiến thức chuyên môn, phương pháp tư duy và tinh thần học tập nghiêm túc mà quý Thầy Cô truyền đạt trong suốt thời gian qua là nền tảng quan trọng giúp em có thể thực hiện và hoàn thành khóa luận này.

Em xin bày tỏ lòng biết ơn sâu sắc và chân thành nhất đến Thầy Lê Trọng Nhân, người đã trực tiếp hướng dẫn và đồng hành cùng em trong quá trình thực hiện đề tài “Phát triển nền tảng nhúng đa chức năng dựa trên Yocto Project cho các ứng dụng tự động hóa trong công nghiệp”. Thầy không chỉ tận tình định hướng nội dung nghiên cứu mà còn luôn hỗ trợ, góp ý và chia sẻ nhiều kinh nghiệm quý báu trong quá trình triển khai hệ thống Linux Embedded, Yocto Project và phát triển Kernel Driver. Những kiến thức và kinh nghiệm mà Thầy truyền đạt là hành trang quan trọng giúp em hoàn thiện đề tài cũng như định hướng cho công việc và quá trình nghiên cứu sau này.

Em cũng xin gửi lời cảm ơn đến các anh chị, bạn bè đã luôn hỗ trợ, trao đổi kiến thức và chia sẻ kinh nghiệm trong suốt quá trình thực hiện khóa luận. Những sự động viên và giúp đỡ đó đã giúp em có thêm động lực để vượt qua những khó khăn trong quá trình nghiên cứu và phát triển hệ thống.

Cuối cùng, em xin gửi lời tri ân sâu sắc đến gia đình và người thân, những người đã luôn bên cạnh động viên, ủng hộ và tạo điều kiện tốt nhất để em có thể yên tâm học tập và hoàn thành khóa luận tốt nghiệp này.

Mặc dù đã cố gắng hoàn thiện khóa luận một cách tốt nhất, tuy nhiên do hạn chế về thời gian, kiến thức và kinh nghiệm thực tế, khóa luận khó tránh khỏi những thiếu sót nhất định. Em rất mong nhận được những ý kiến đóng góp quý báu từ quý Thầy Cô để đề tài được hoàn thiện hơn.

TP. Hồ Chí Minh, tháng 5 năm 2026

Sinh viên thực hiện

Dương Minh Hiếu

Đỗ Tuấn Anh

Nguyễn Hạo Duy

Mục Lục

1	Phần mở đầu	13
1.1	Đặt vấn đề	14
1.2	Mục tiêu nghiên cứu	15
1.3	Phạm vi và giới hạn nghiên cứu	16
1.4	Bố cục luận văn	16
2	Cơ sở lý thuyết	18
2.1	Kiến trúc Build System trong Yocto Project	19
2.1.1	Các thành phần chính trong Layer của Yocto Project	19
2.1.1.1	Recipe (.bb)	19
2.1.1.2	Configuration	19
2.1.1.3	Patch	20
2.1.1.4	Source Code	20
2.1.2	Recipe trong Yocto	20
2.1.3	BitBake Build Engine	21
2.1.4	Cross-compilation trong Yocto	23
2.2	Linux Device Driver	23
2.2.1	Tổng quan Linux Kernel	23
2.2.2	Linux Device Driver	24
2.2.3	Device Tree	25
2.2.4	Kernel Module	26
2.2.5	Cơ chế tự động nạp module	28
2.2.6	Xây dựng và tích hợp Driver trong Yocto	29
2.3	Edge AI trong hệ thống nhúng	30
2.3.1	Khái niệm Edge AI và lợi ích so với Cloud AI	30
2.3.2	Tổng quan mô hình TCN (Temporal Convolutional Network) và lý do lựa chọn TCN	31

2.3.2.1	Tổng quan về kiến trúc TCN	31
2.3.2.2	Lý do lựa chọn TCN thay vì LSTM/RNN	32
2.3.3	Định dạng mô hình trung gian và tối ưu hóa trên thiết bị Edge	32
2.3.3.1	Định dạng mô hình ONNX (Open Neural Network Exchange)	32
2.3.3.2	Định dạng TensorFlow Lite (TFLite) và tối ưu hóa Float16	33
2.4	Giao tiếp phần cứng công nghiệp	34
2.4.1	Chuẩn vật lý RS485	34
2.4.2	Giao thức Modbus RTU và cấu trúc khung tin	34
2.4.2.1	Giao thức Modbus RTU	34
2.4.2.2	Cấu trúc khung tin	35
2.5	Kết nối Cloud và IoT	36
2.5.1	Tổng quan về CoreIOT	36
2.5.2	Giao thức MQTT trong hệ thống nhúng	37
3	Thiết kế hệ thống	38
3.1	Phân tích yêu cầu	39
3.1.1	Mô tả bài toán	39
3.1.2	Yêu cầu hệ thống	40
3.1.2.1	Yêu cầu chức năng	40
3.1.2.2	Yêu cầu phi chức năng	40
3.2	Kiến trúc tổng quan hệ thống	41
3.3	Thiết kế phần cứng hệ thống	41
3.3.1	Raspberry Pi4	41
3.3.2	Chuẩn giao tiếp vật lý RS485	42
3.4	Thiết kế nền tảng nhúng	43
3.4.1	Thiết kế hệ thống bảo mật cho Linux nhúng	43
3.4.1.1	Xây dựng cơ chế kiểm soát truy cập và phân quyền	43
3.4.1.2	Tăng cường bảo mật dịch vụ SSH	44
3.4.1.3	Thiết kế hệ thống quản lý package an toàn	45
3.4.1.4	Triển khai cơ chế bảo mật SELinux	48
3.4.1.5	Bảo vệ tính toàn vẹn hệ thống với dm-verity	49
3.4.2	Phát triển Driver cho Linux nhúng	51
3.4.2.1	Kiến trúc tổng quan Driver trong hệ thống Yocto Linux	51
3.4.2.2	Driver RS485	53
3.4.2.3	Driver I2C	56
3.4.3	Thiết kế khối xử lý AI	60
3.4.3.1	Thiết kế dữ liệu đầu vào và đầu ra	61
3.4.3.2	Quy trình tiền xử lý và chuẩn hóa dữ liệu	62

3.4.3.3	Kiến trúc mạng nơ-ron TCN (Temporal Convolutional Network)	62
3.4.3.3.1	a. Tích chập nhân quả (Causal Convolution) . . .	63
3.4.3.3.2	b. Tích chập giãn cách (Dilated Convolution) . .	63
3.4.3.3.3	c. Kết nối tắt (Residual Connections)	63
3.4.3.4	Thiết kế chiến lược huấn luyện	63
3.4.3.5	Thiết kế quy trình tối ưu hóa Edge AI (TFLite Quantization)	64
3.4.3.6	Mô hình FSM khối AI	64
3.4.4	Thiết kế khối xử lý OTA	66
3.4.4.1	Mô tả chi tiết các thành phần hệ thống	66
3.4.4.2	Luồng dữ liệu và logic xử lý (Data Flow Description) . . .	67
3.4.4.3	Mô hình FSM khối OTA	68
3.4.5	Thiết kế khối xử lý Tool	70
3.4.5.1	Lớp Giao diện Người dùng (Client User Interface)	71
3.4.5.2	Lớp Cửa ngõ API (API Gateway Layer)	71
3.4.5.3	Lớp Hệ thống Đích và Dịch vụ Ngoại vi (Target System & External Services)	72
3.4.5.4	Mô hình FSM khối Tool	72
3.5	Thiết kế dịch vụ đám mây và IoT	75
3.5.1	Thiết kế cơ chế truyền dữ liệu giữa Gateway và CoreIoT	75
3.5.2	Thiết kế Rule Chain xử lý dữ liệu IoT	77
4	Hiện thực nền tảng nhúng	79
4.1	Hiện thực bảo mật hệ thống	80
4.1.1	Hiện thực giảm thiểu bề mặt tấn công	80
4.1.2	Hiện thực Selinux	81
4.1.3	Hiện thực dm-verity	83
4.1.3.1	Giai đoạn tiền xử lý và tách phân vùng dữ liệu	83
4.1.3.2	Khởi tạo cấu trúc cây băm (Hash Tree) và xác thực metadata	83
4.1.3.3	Triển khai và cấu hình giai đoạn khởi động sớm	84
4.1.3.4	Cơ chế kiểm chứng thực thi tại thiết bị đích	85
4.2	Hiện thực Device Driver	87
4.2.1	Tổng quan kiến trúc Driver	87
4.2.2	Hiện thực driver I2C	87
4.2.2.1	Kiến trúc driver	87
4.2.2.2	Nguyên lý giao thức I2C	88
4.2.2.3	Các chức năng chính của driver	88
4.2.2.4	Thiết kế cơ chế IOCTL	89

4.2.2.5	Giao tiếp giữa user space và kernel space	89
4.2.2.6	Hiện thực giao thức I2C bằng GPIO Bit-banging	89
4.2.2.7	Tối ưu timing cho giao tiếp I2C	90
4.2.2.8	Tích hợp driver vào hệ thống Yocto	90
4.2.3	Hiện thực Driver RS485 Modbus RTU	92
4.2.3.1	Kiến trúc Driver	92
4.2.3.2	Nguyên lý giao thức	93
4.2.3.3	Các chức năng chính của Driver	93
4.2.3.4	Thiết kế cơ chế hoạt động	94
4.2.3.5	Tích hợp Driver vào hệ thống Yocto	95
4.3	Hiện thực Khối xử lý AI	97
4.3.1	Thiết lập môi trường và Công cụ hỗ trợ	97
4.3.2	Hiện thực Tiền xử lý và Cấu trúc hóa Dữ liệu	97
4.3.2.1	Làm mượt và Chuẩn hóa dữ liệu	97
4.3.2.2	Cấu trúc cửa sổ trượt (Sliding Window)	98
4.3.3	Hiện thực Kiến trúc Mạng TCN	98
4.3.3.1	Phân tích chi tiết từ hiện thực	99
4.3.4	Hiện thực Quy trình Huấn luyện và Giám sát	99
4.3.4.1	Hiện thực cấu hình tham số huấn luyện	99
4.3.4.2	Hiện thực cơ chế Giám sát và Dừng sớm (Early Stopping)	100
4.3.4.3	Nhật ký thực thi quá trình huấn luyện	100
4.4	Hiện thực hệ thống cập nhật từ xa OTA	101
4.4.1	Tích hợp các Layer và Cấu hình môi trường Yocto	101
4.4.2	Hiện thực và Tùy chỉnh Recipe SWUpdate	102
4.4.3	Xây dựng Kịch bản Mô tả Hệ thống	103
4.4.4	Hiện thực Kịch bản Hậu cập nhật và Giao tiếp Bootloader	104
4.4.5	Quy trình Đóng gói và Triển khai Vận hành Thực tế	105
4.5	Hiện thực Tool Build Yocto	105
4.5.1	Kiến trúc phân lớp và Luồng dữ liệu	105
4.5.2	Hiện thực Lớp Backend (Flask API)	106
4.5.2.1	Kỹ thuật điều khiển cấu hình bằng Biểu thức chính quy (Regex)	106
4.5.2.2	Quản lý quy trình thực thi BitBake	107
4.5.3	Hiện thực Giao diện người dùng (Frontend TUI)	107
4.5.3.1	Quản lý trạng thái (State Management)	107
4.5.3.2	Xử lý sự kiện (Event Handling)	108
4.5.4	Tích hợp các module phân tích thông minh	108
4.5.4.1	Module AI Debugger (Gemini Analytics)	108

4.5.4.2	Module Quét lỗ hổng bảo mật (CVE Scan Integration) . . .	109
5	Kết quả và đánh giá	110
5.1	Tổng quan kết quả hiện thực	111
5.2	Bảo mật hệ thống	113
5.2.1	Selinux	113
5.2.2	dm-verity	114
5.2.3	SSH Hardening	114
5.3	Device Driver	117
5.3.1	Driver I2C	117
5.3.2	Driver RS485 Modbus RTU	118
5.4	Khối xử lý AI	120
5.4.1	Các tệp tin sản phẩm sau hiện thực (Artifacts)	120
5.4.2	Đặc tính kỹ thuật và Tài nguyên hệ thống	120
5.4.3	Phân tích quá trình huấn luyện và hội tụ	121
5.4.4	Đánh giá sơ bộ các chỉ số sai số (Metrics)	123
5.4.5	Kiểm định khả năng Inference trên phần cứng Raspberry Pi 4	124
5.4.5.1	Đánh giá độ nhạy dự đoán qua thực nghiệm tác động môi trường	124
5.4.5.2	Đánh giá hiệu năng thời gian thực (Inference Time)	125
5.4.5.3	Tổng kết khả năng thực thi	126
5.5	Công cụ Secure-Yocto Builder	126
5.5.1	Kết quả hiện thực công cụ Secure-Yocto Builder	126
5.5.1.1	Giao diện cấu hình thiết bị mục tiêu (Target Device)	127
5.5.1.2	Quản lý lớp (Layer Setup) và Recipe bổ sung	127
5.5.1.3	Cấu hình bảo mật hệ thống (Security Hardening)	127
5.5.1.4	Quản lý Driver ngoại vi (Peripheral Drivers)	128
5.5.1.5	Hệ thống Build và Giám sát tiến độ	128
5.5.1.6	Tính năng thông minh: AI Debug và CVE Scan	128
5.5.2	Đánh giá và nhận xét	129
5.5.2.1	Về tính năng và vận hành	129
5.5.2.2	Về trải nghiệm người dùng (UX)	129
5.5.2.3	Hạn chế còn tồn tại	130
5.6	Hệ thống OTA	130
5.6.1	Giao diện quản trị Web (Web UI)	130
5.6.2	Quá trình thực thi và phân tích Log hệ thống	131
5.6.3	Đánh giá hiệu năng và độ tin cậy	132
5.7	Dashboard giám sát	132



5.7.1	Thiết kế giao diện Dashboard CoreIot	132
5.7.2	Thiết kế giao diện HDMI Dashboard	134
6	Kết luận và hướng phát triển	137
6.1	Kết luận	138
6.2	Hướng phát triển	138

Danh sách bảng

2.1	Các biến cơ bản trong Recipe	21
2.2	Các task chính của BitBake	22
2.3	Mô hình Cross-compilation	23
2.4	Các loại Linux Device Driver	25
2.5	Các thông tin trong Device Tree	26
2.6	Các lệnh kiểm tra Kernel Module	29
3.1	Yêu cầu chức năng của hệ thống	40
3.2	Yêu cầu phi chức năng của hệ thống	40
3.3	So sánh các nền tảng phần cứng hỗ trợ Yocto Project	42
3.4	So sánh các package manager trên Linux	46
3.5	So sánh các policy trong SELinux	49
3.6	Cơ chế xác định trạng thái hệ thống	76
3.7	Cấu trúc payload dữ liệu gửi lên CoreIoT	77
4.1	So sánh các policy phổ biến của SELinux	82
5.1	Kết quả kiểm thử xác thực SSH	115
5.2	Kết quả kiểm thử SSH với mật khẩu sai	116
5.3	Kết quả kiểm thử SSH với tài khoản không hợp lệ	116
5.4	Kết quả kiểm thử SSH với tài khoản root	117
5.5	Thông số kỹ thuật của mô hình TCN sau huấn luyện	121

Danh sách hình

1.1	An toàn lao động trong kho xưởng	15
2.1	Yocto Project	19
2.2	Cloud AI và Edge AI	30
2.3	Kiến trúc mô hình TCN	31
2.4	Định dạng ONNX	32
2.5	Định dạng TensorFlow Lite	33
2.6	RS485 TTL	34
2.7	Tổng quan về CoreIOT	36
2.8	Giao thức MQTT trong hệ thống nhúng	37
3.1	Mô tả bài toán	39
3.2	Các chuẩn giao tiếp vật lý công nghiệp	43
3.3	Cơ chế SSH	44
3.4	Thiết kế cơ chế tăng cường bảo mật SSH	44
3.5	Nginx repository server cho quản lý package an toàn	46
3.6	Kiến trúc quản lý package an toàn	47
3.7	Bảo mật quyền truy cập hệ thống với SELinux trong Yocto	48
3.8	Cấu trúc Merkle Hash Tree trong cơ chế dm-verity	50
3.9	Activity diagram thiết kế dm-verity	51
3.10	Kiến trúc tổng quan driver trong hệ thống Yocto Linux	52
3.11	Máy trạng thái cho driver RS458	53
3.12	Activity Diagram của driver RS485	55
3.13	Máy trạng thái cho driver I2C	57
3.14	Activity Diagram của driver I2C	59
3.15	Sơ đồ khối Module AI	61
3.16	mô hình TCN	62
3.17	Mô hình FSM khối AI	65

3.18	Sơ đồ khối OTA	66
3.19	Mô hình FSM khối OTA	68
3.20	Sơ đồ khối Tool	71
3.21	Mô hình FSM khối Tool	73
4.1	Giảm thiểu bề mặt tấn công trong Yocto Project	80
4.2	Cấu trúc thư mục chính của layer meta-selinux	81
4.3	Selinux	83
4.4	Cấu trúc cây băm của rootfs	84
4.5	Giá trị băm gốc (root hash)	84
4.6	Kết quả xác thực metadata của dm-verity	84
4.7	Mô hình TCN	99
4.8	Nhật ký quá trình huấn luyện mô hình TCN trên môi trường máy trạm	100
5.1	Thông tin phiên bản hệ điều hành và nhân Linux	111
5.2	Thông tin bộ xử lý và các lỗ hổng bảo mật	112
5.3	Dung lượng bộ nhớ và thông tin board Raspberry Pi 4	112
5.4	SELinux hoạt động ở chế độ enforcing với policy targeted	113
5.5	Hệ thống khởi động thành công với root filesystem được bảo vệ bởi dm-verity	114
5.6	Đăng nhập SSH với mật khẩu	115
5.7	Giới hạn nhập sai mật khẩu trong SSH	115
5.8	Đăng nhập SSH bằng tài khoản không được phép	116
5.9	Đăng nhập SSH bằng tài khoản root	116
5.10	Tính khả dụng của Driver	117
5.11	Kết quả hiển thị dữ liệu cảm biến trên màn hình OLED SSD1306	118
5.12	Tính khả dụng của Driver	118
5.13	Thu thập dữ liệu từ Driver	119
5.14	Lưu trữ dữ liệu từ Driver	119
5.15	cấu trúc file sau khi train mô hình	120
5.16	Kết quả train mô hình	122
5.17	Đồ thị train mô hình	122
5.18	Kết quả test	125
5.19	Kết quả Inference	126
5.20	Giao diện lựa chọn Target Device và phản hồi cập nhật local.conf.	127
5.21	Kết quả kiểm tra và thêm gói phần mềm thủ công.	127
5.22	Bảng điều khiển các tính năng bảo mật nâng cao.	128
5.23	Giao diện bật/tắt Driver RS-485 và I ² C Soft Driver.	128
5.24	Tổng kết cấu hình và thanh tiến độ hoàn thành bản build.	128

5.25	Kết quả phân tích lỗi của AI và bảng danh sách lỗ hổng CVE.	129
5.26	Giao diện UI người dùng	130
5.27	Kết quả OTA	131
5.28	Trang chủ Dashboard CoreIot	132
5.29	Trang thiết bị Dashboard CoreIot	133
5.30	Giao diện HDMI Dashboard tổng thể	135
5.31	Màn hình hiển thị dữ liệu các cảm biến	135
5.32	Giao diện tab Notifications	136

Chương 1

Phần mở đầu

- 1.1** Đặt vấn đề
- 1.2** Mục tiêu nghiên cứu
- 1.3** Phạm vi và giới hạn nghiên cứu
- 1.4** Bố cục luận văn

1.1 Đặt vấn đề

Trong bối cảnh Cách mạng công nghiệp 4.0, các hệ thống tự động hóa đang có xu hướng dịch chuyển từ mô hình xử lý tập trung sang mô hình xử lý phân tán tại biên, hay còn gọi là Edge Computing. Trong mô hình này, thiết bị Edge Gateway đóng vai trò trung gian giữa tầng cảm biến, thiết bị hiện trường và hệ thống giám sát trung tâm. Thiết bị không chỉ thu thập và truyền dữ liệu, mà còn có thể thực hiện các tác vụ như tiền xử lý tín hiệu, suy diễn mô hình trí tuệ nhân tạo, mã hóa dữ liệu, quản lý kết nối mạng và cập nhật phần mềm từ xa.

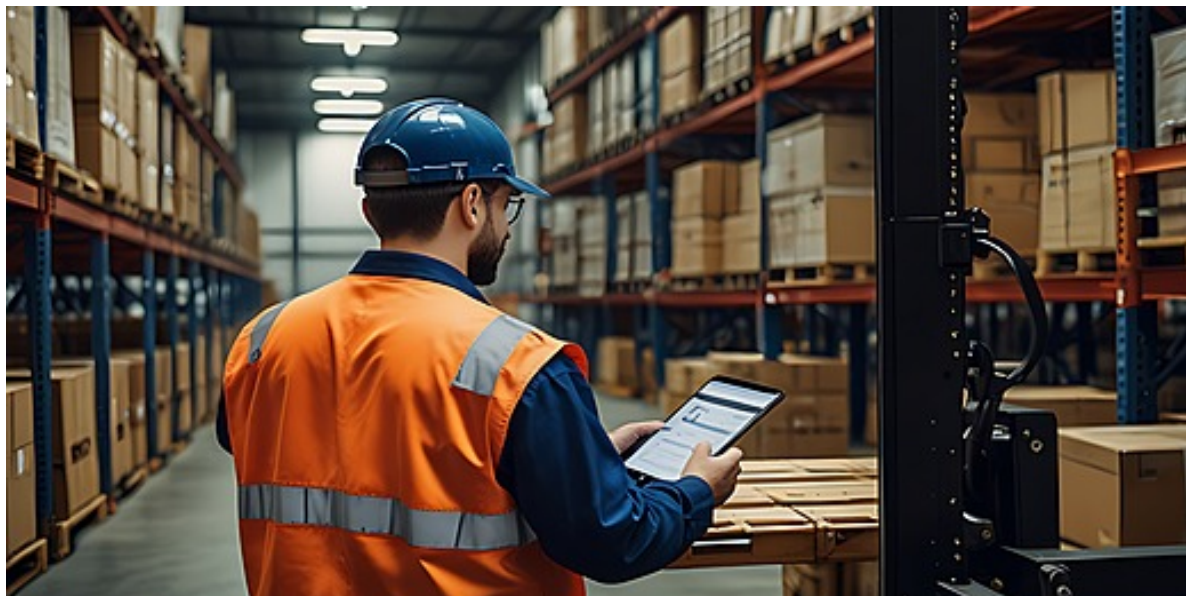
Trong nhiều hệ thống thử nghiệm hoặc triển khai quy mô nhỏ, Edge Gateway thường được xây dựng dựa trên các bản phân phối Linux tổng quát như Raspberry Pi OS hoặc Ubuntu Server. Cách tiếp cận này có ưu điểm là dễ cài đặt và thuận tiện trong giai đoạn phát triển ban đầu, nhưng tồn tại nhiều hạn chế khi áp dụng cho môi trường nhúng công nghiệp. Thứ nhất, hệ điều hành tổng quát thường chứa nhiều gói phần mềm không cần thiết, làm tăng kích thước image, tiêu tốn tài nguyên lưu trữ và mở rộng diện tích tản công bảo mật. Thứ hai, việc kiểm soát sâu các thành phần của hệ thống như kernel, dịch vụ khởi động, thư viện, cơ chế phân quyền và chính sách bảo mật còn hạn chế. Thứ ba, việc tích hợp đồng bộ các cơ chế bảo vệ hệ thống, cập nhật phần mềm an toàn và tối ưu hóa cho từng phần cứng cụ thể thường gặp nhiều khó khăn.

Yocto Project cung cấp một hệ sinh thái cho phép xây dựng bản phân phối Linux nhúng tùy biến, trong đó kỹ sư có thể kiểm soát toàn bộ quá trình tạo hệ điều hành, từ cấu hình kernel, lựa chọn gói phần mềm, xây dựng root filesystem, tạo image khởi động cho đến tích hợp các cơ chế bảo mật. Nhờ khả năng tùy biến theo lớp thông qua layer, recipe và machine configuration, Yocto phù hợp để phát triển các nền tảng Edge Gateway yêu cầu tính tối giản, ổn định, bảo mật và có khả năng mở rộng.

Xuất phát từ yêu cầu đó, đề tài tập trung nghiên cứu và xây dựng một nền tảng nhúng đa chức năng dựa trên Yocto Project phục vụ các ứng dụng tự động hóa công nghiệp. Nền tảng hướng đến việc tạo ra một hệ điều hành Linux nhúng tùy biến cho thiết bị Edge Gateway, có khả năng thu thập dữ liệu cảm biến, xử lý tại biên, quản lý dịch vụ hệ thống và tích hợp các cơ chế bảo mật như phân quyền người dùng, giảm thiểu thành phần không cần thiết và bảo vệ hệ thống tập tin.

Để kiểm chứng tính đúng đắn và đánh giá hiệu quả thực tế của nền tảng, đề tài lựa chọn bài toán giám sát an toàn lao động và dự báo nguy cơ cháy nổ trong nhà kho làm ứng dụng thực chứng. Đây là bài toán có tính đại diện cao vì yêu cầu thiết bị phải hoạt động ổn định trong thời gian dài, thu thập dữ liệu từ nhiều nguồn cảm biến, xử lý cảnh báo tại biên và đảm bảo an toàn dữ liệu trong quá trình vận hành. Thông qua ứng dụng này, đề tài đánh giá khả năng đáp ứng của nền tảng Yocto-based Edge Gateway đối với các yêu cầu cốt lõi của hệ thống nhúng công nghiệp, bao gồm tính tối giản, khả năng tùy biến, hiệu năng xử lý, mức độ bảo mật và

khả năng triển khai thực tế.



Hình 1.1: An toàn lao động trong kho xưởng

1.2 Mục tiêu nghiên cứu

Đề tài hướng đến việc nghiên cứu, xây dựng và đánh giá một nền tảng Edge Gateway nhúng dựa trên Yocto Project cho Raspberry Pi 4, phục vụ các ứng dụng tự động hóa công nghiệp theo kiến trúc Device–Edge–Cloud. Nội dung nghiên cứu được triển khai theo sáu mục tiêu cụ thể sau:

- Mục tiêu 1 - Nền tảng hệ điều hành tối giản: Xây dựng Embedded Linux Image tùy biến bằng Yocto Project và chứng minh hiệu quả tối giản hóa thông qua so sánh định lượng với Raspberry Pi OS về kích thước image, dung lượng root filesystem, số lượng package, số tiến trình hoạt động, số dịch vụ khởi động và mức sử dụng RAM/CPU trong điều kiện vận hành thực tế.
- Mục tiêu 2 - Phát triển driver và giao tiếp công nghiệp: Hiện thực Linux Device Driver cho giao tiếp RS485/Modbus RTU dưới dạng Kernel Module tích hợp trong môi trường Yocto, và kiểm chứng khả năng thu thập dữ liệu ổn định từ các thiết bị cảm biến công nghiệp trong điều kiện vận hành liên tục.
- Mục tiêu 3 - Bảo mật hệ thống đa lớp: Tích hợp và đánh giá hiệu quả của hệ thống bảo mật theo chiều sâu (defense in depth) gồm sáu lớp: dm-verity bảo vệ tính toàn vẹn filesystem, quản lý phân quyền sudo/user/group, cứng hóa SSH, cấu hình kernel parameters qua sysctl, giám sát truy cập bằng auditd và phòng chống brute-force bằng fail2ban. Mỗi lớp được đánh giá cả về hiệu quả bảo vệ lẫn chi phí tài nguyên phát sinh.

- Mục tiêu 4 - Cập nhật hệ thống an toàn: Thiết kế và kiểm chứng cơ chế OTA theo mô hình A/B Slot, đảm bảo khả năng rollback tự động về slot ổn định khi gặp sự cố, kể cả trong tình huống mất điện giữa quá trình cập nhật.
- Mục tiêu 5 - Tích hợp Cloud IoT: Xây dựng luồng dữ liệu từ thiết bị hiện trường đến nền tảng CoreIoT Cloud qua MQTT, bao gồm tiền xử lý tại biên, trực quan hóa trạng thái hệ thống trên dashboard, cấu hình cảnh báo và điều khiển thiết bị từ xa qua RPC.
- Mục tiêu 6 - Ứng dụng thực chứng: Sử dụng bài toán giám sát an toàn lao động tại nhà kho — tích hợp mô hình Temporal Convolutional Network dự báo nguy cơ cháy nổ — làm case study để đánh giá đồng thời toàn bộ nền tảng trên năm tiêu chí: tính tối giản, hiệu năng xử lý tại biên, bảo mật đa lớp, độ tin cậy cập nhật và khả năng kết nối cloud.

1.3 Phạm vi và giới hạn nghiên cứu

Đề tài tập trung xây dựng và đánh giá nền tảng phần mềm cho thiết bị Edge Gateway công nghiệp dựa trên Yocto Project, triển khai trên Raspberry Pi 4 làm bo mạch mục tiêu. Hệ thống được thiết kế theo kiến trúc Device–Edge–Cloud, trong đó Edge Gateway đảm nhiệm thu thập dữ liệu, xử lý tại biên, bảo mật và cập nhật hệ thống, còn CoreIoT Cloud phục vụ giám sát, trực quan hóa dữ liệu, cảnh báo và điều khiển từ xa.

Phạm vi nghiên cứu bao gồm: xây dựng Embedded Linux Image tùy biến bằng Yocto Project; phát triển Linux Device Driver hỗ trợ giao tiếp RS485 và tích hợp phần mềm Modbus RTU; triển khai các cơ chế bảo mật cho hệ thống; xây dựng cơ chế OTA an toàn theo mô hình A/B Slot; tích hợp mô hình AI suy diễn tại biên; kết nối dữ liệu lên CoreIoT Cloud; và triển khai bài toán giám sát an toàn nhà kho làm ứng dụng thực chứng.

Trên cơ sở nền tảng được xây dựng, đề tài triển khai mô hình quản lý kho thông minh nhằm kiểm chứng khả năng ứng dụng thực tế của hệ thống trong việc giám sát môi trường kho, thu thập dữ liệu cảm biến, cảnh báo trạng thái bất thường và quản lý thiết bị từ xa.

Tuy nhiên, do giới hạn về thời gian, tài nguyên và phạm vi nghiên cứu, đề tài mới chỉ dừng lại ở mô hình thử nghiệm, chưa được đánh giá trên quy mô công nghiệp lớn. Các bài toán chuyên sâu như tối ưu logistics, quản lý chuỗi cung ứng hay tích hợp hệ thống ERP/WMS hoàn chỉnh chưa được đề cập. Ngoài ra, đề tài chưa nghiên cứu các cơ chế bảo mật phần cứng nâng cao (TPM, HSM) hoặc giải pháp chống tấn công vật lý.

1.4 Bố cục luận văn

Luận văn được tổ chức thành 6 chương với nội dung chính như sau:

- **Chương 1: Phần mở đầu**

Trình bày bối cảnh nghiên cứu, lý do chọn đề tài, mục tiêu nghiên cứu, phạm vi triển

khai và bố cục tổng thể của luận văn.

- **Chương 2: Tổng quan dự án**

Trình bày tổng quan về Linux Embedded, Yocto Project, Industrial IoT và các công nghệ liên quan đến hệ thống nhúng và tự động hóa công nghiệp.

- **Chương 3: Cơ sở lý thuyết**

Trình bày các cơ sở lý thuyết phục vụ cho việc xây dựng hệ thống, bao gồm Yocto Project, BitBake, Linux Kernel, Linux Device Driver, Kernel Module và các cơ chế tích hợp driver trong Linux Embedded.

- **Chương 4: Thiết kế hệ thống**

Phân tích yêu cầu hệ thống, xây dựng kiến trúc tổng thể và thiết kế các thành phần kỹ thuật như Linux Embedded Platform, Driver phần cứng, bảo mật hệ thống và cơ chế giao tiếp IoT.

- **Chương 5: Triển khai và đánh giá**

Trình bày quá trình triển khai hệ điều hành bằng Yocto Project, xây dựng Linux Device Driver, tích hợp MQTT/HTTP, triển khai hệ thống giám sát và đánh giá kết quả hoạt động của hệ thống.

- **Chương 6: Kết luận và hướng phát triển**

Tổng kết các kết quả đạt được, đánh giá ưu điểm và hạn chế của hệ thống, đồng thời đề xuất các hướng phát triển trong tương lai.

Chương 2

Cơ sở lý thuyết

- 2.1** Kiến trúc Build System trong Yocto Project
- 2.2** Linux Device Driver
- 2.3** Bảo mật hệ điều hành Linux nhúng
- 2.4** Edge AI trong hệ thống nhúng
- 2.5** Giao diện người dùng trên thiết bị nhúng
- 2.6** Giao tiếp phần cứng công nghiệp
- 2.7** Kết nối Cloud và IoT

2.1 Kiến trúc Build System trong Yocto Project



Hình 2.1: Yocto Project

2.1.1 Các thành phần chính trong Layer của Yocto Project

Trong Yocto Project, kiến trúc layer được xây dựng dựa trên cơ chế metadata nhằm tổ chức hệ thống build theo hướng module hóa và có khả năng mở rộng cao. Mỗi layer đóng vai trò như một đơn vị chức năng độc lập, có thể chứa đầy đủ các thành phần cần thiết để xây dựng phần mềm hoặc hệ thống Linux nhúng. Việc phân chia các thành phần trong layer giúp quá trình phát triển, bảo trì và tái sử dụng mã nguồn trở nên thuận tiện hơn. Các thành phần chính thường xuất hiện trong một layer bao gồm Recipe, Configuration, Patch và Source Code.

2.1.1.1 Recipe (.bb)

Recipe là thành phần cốt lõi trong Yocto Project, được sử dụng để mô tả toàn bộ quy trình xây dựng một package hoặc một thành phần phần mềm trong hệ thống nhúng. Các recipe thường có phần mở rộng `.bb` và được xử lý bởi BitBake Build Engine. Có thể xem recipe như một tập hợp metadata dùng để hướng dẫn hệ thống build thực hiện các công việc như tải mã nguồn, giải nén, biên dịch, cài đặt và đóng gói phần mềm.

Trong mỗi recipe, nhiều biến hệ thống được sử dụng để mô tả thông tin liên quan đến package. Các biến này bao gồm tên package, phiên bản, giấy phép sử dụng, dependency, vị trí source code và các task cần thực hiện trong quá trình build. Thông qua recipe, BitBake có thể tự động phân tích dependency giữa các package và xây dựng quy trình biên dịch phù hợp.

2.1.1.2 Configuration

Configuration là thành phần chịu trách nhiệm quản lý và thiết lập môi trường build trong Yocto Project. Các file cấu hình thường được lưu trong thư mục `conf/` và chứa các biến hệ thống phục vụ quá trình xây dựng Linux Embedded. Đây là thành phần quan trọng giúp xác định cách thức hoạt động của toàn bộ hệ thống build. Thông qua configuration, người phát triển có thể thiết lập kiến trúc phần cứng mục tiêu, compiler, toolchain, package manager, loại

filesystem và nhiều tham số hệ thống khác. Ngoài ra, configuration còn cho phép quản lý danh sách layer được sử dụng trong môi trường build cũng như thứ tự ưu tiên giữa các layer.

2.1.1.3 Patch

Patch là thành phần được sử dụng để sửa đổi hoặc cập nhật mã nguồn trong quá trình build mà không cần thay đổi trực tiếp source code gốc. Trong Yocto Project, patch đóng vai trò quan trọng trong việc tùy biến phần mềm, sửa lỗi hệ thống và bổ sung chức năng mới cho các package. Patch thường được lưu dưới dạng file `.patch` hoặc `.diff`. Các file này chứa thông tin thay đổi giữa phiên bản mã nguồn gốc và phiên bản đã được chỉnh sửa. Trong quá trình build, BitBake sẽ tự động áp dụng patch trước khi thực hiện biên dịch mã nguồn.

2.1.1.4 Source Code

Source Code là thành phần chứa mã nguồn của chương trình hoặc hệ thống phần mềm được sử dụng trong quá trình build. Đây là thành phần cốt lõi quyết định chức năng và hành vi của package trong hệ thống Linux Embedded. Trong Yocto Project, source code có thể được lấy từ nhiều nguồn khác nhau như local file, Git repository, tarball package hoặc remote server. Hệ thống BitBake sẽ tự động tải và quản lý source code dựa trên thông tin được khai báo trong recipe. Source code trong hệ thống nhúng thường bao gồm nhiều loại thành phần như application, library, kernel module, bootloader hoặc device driver. Tùy thuộc vào chức năng của package, source code có thể được viết bằng các ngôn ngữ như C, C++, Python hoặc Shell Script.

2.1.2 Recipe trong Yocto

Trong Yocto Project, Recipe là thành phần trung tâm của hệ thống build và đóng vai trò mô tả toàn bộ quy trình xây dựng một package phần mềm. Recipe thường có phần mở rộng `.bb` (BitBake Recipe) và được BitBake sử dụng để thực hiện các tác vụ như tải mã nguồn, cấu hình môi trường build, biên dịch, cài đặt và đóng gói package. [1]

Có thể xem recipe như một tập hợp metadata dùng để hướng dẫn BitBake cách xây dựng một thành phần phần mềm cụ thể trong hệ thống Linux Embedded. Thông qua recipe, Yocto Project có khả năng tự động hóa toàn bộ quy trình build và quản lý dependency giữa các package trong hệ thống. Một recipe thường chứa nhiều biến và task nhằm mô tả thông tin liên quan đến package. Các biến này được sử dụng để khai báo metadata như tên package, phiên bản, license, vị trí source code và phương thức build. Trong khi đó, các task chịu trách nhiệm thực thi từng giai đoạn trong pipeline build của BitBake.

Một số biến quan trọng trong recipe được trình bày trong Bảng 2.1.

Biến	Ý nghĩa
SUMMARY	Mô tả ngắn gọn chức năng hoặc mục đích của package.
DESCRIPTION	Mô tả chi tiết hơn về package và chức năng của phần mềm.
LICENSE	Khai báo license của package nhằm phục vụ quản lý bản quyền phần mềm.
SRC_URI	Khai báo đường dẫn source code hoặc patch được sử dụng trong quá trình build.
S	Xác định thư mục chứa source code sau khi giải nén.
DEPENDS	Khai báo các package phụ thuộc cần thiết trước khi build package hiện tại.
RDEPENDS	Khai báo dependency cần thiết khi package chạy trên hệ thống đích.
inherit	Kế thừa các class của Yocto nhằm bổ sung chức năng build cho recipe.
do_compile	Task dùng để biên dịch source code thành binary executable hoặc kernel module.
do_install	Task dùng để cài đặt các file output vào root filesystem tạm thời.

Bảng 2.1: Các biến cơ bản trong Recipe

Trong quá trình build, BitBake sẽ phân tích dependency giữa các recipe để xác định thứ tự thực thi phù hợp. Nhờ đó, hệ thống có thể tự động build toàn bộ Linux Embedded Image với số lượng package lớn mà vẫn đảm bảo tính nhất quán.

2.1.3 BitBake Build Engine

BitBake là build engine trung tâm của Yocto Project, chịu trách nhiệm phân tích metadata và thực thi các task được định nghĩa trong recipe nhằm xây dựng package hoặc toàn bộ hệ thống Linux Embedded. Có thể xem BitBake như một hệ thống tự động hóa build hoạt động dựa trên metadata, cho phép quản lý dependency, thực thi pipeline build và tạo ra image Linux hoàn chỉnh cho nền tảng nhúng.

Trong Yocto Project, mỗi recipe sẽ được BitBake phân tích để xác định các task cần thực hiện cũng như mối quan hệ phụ thuộc giữa các package. Dựa trên dependency graph, BitBake sẽ tự động xác định thứ tự thực thi phù hợp nhằm đảm bảo quá trình build diễn ra chính xác và nhất quán.

Một trong những ưu điểm quan trọng của BitBake là khả năng tự động hóa toàn bộ quá trình build. Thay vì thực hiện thủ công từng bước như tải source code, cấu hình compiler hoặc cài đặt package, BitBake cho phép toàn bộ pipeline build được mô tả thông qua metadata trong recipe. Điều này giúp giảm đáng kể độ phức tạp trong quá trình phát triển hệ thống Linux Embedded.

Các task chính của BitBake được trình bày trong Bảng 2.2.

Task	Chức năng
do_fetch	Tải source code
do_unpack	Giải nén source
do_patch	Áp dụng patch
do_compile	Biên dịch mã nguồn
do_install	Cài đặt package
do_package	Đóng gói package

Bảng 2.2: Các task chính của BitBake

Task do_fetch

Task do_fetch là giai đoạn đầu tiên trong pipeline build của BitBake, chịu trách nhiệm tải source code và các tài nguyên cần thiết cho quá trình build. Dữ liệu có thể được lấy từ Git repository, HTTP server, FTP server, local file hoặc tarball package thông qua biến SRC_URI.

Task do_unpack

Sau khi tải source code, BitBake thực hiện task do_unpack để giải nén và chuẩn bị mã nguồn cho các bước xử lý tiếp theo. Các file như .tar.gz, .zip hoặc source package sẽ được giải nén vào thư mục làm việc của BitBake, đồng thời source code từ Git repository sẽ được checkout vào workspace tương ứng.

Task do_patch

Task do_patch được sử dụng để áp dụng patch lên source code sau khi giải nén hoàn tất. Cơ chế này cho phép sửa lỗi phần mềm, tùy biến kernel hoặc driver, bổ sung tính năng mới và tối ưu hệ thống cho phần cứng mục tiêu mà không cần thay đổi trực tiếp source code gốc.

Task do_compile

Task do_compile chịu trách nhiệm biên dịch source code thành binary executable cho hệ thống đích. Trong Linux Embedded, quá trình này sử dụng cross-compiler để tạo binary phù hợp với các kiến trúc như ARM, x86 hoặc RISC-V. BitBake sẽ tự động cấu hình compiler, sysroot và môi trường build dựa trên metadata của hệ thống. Task này hỗ trợ build application, library, kernel module và Linux Kernel.

Task do_install

Sau khi biên dịch hoàn tất, BitBake thực hiện task do_install để cài đặt các file output vào thư mục staging hoặc root filesystem tạm thời. Trong giai đoạn này, các binary, library và configuration file sẽ được sắp xếp theo đúng cấu trúc filesystem của Linux nhằm phục vụ quá trình tạo image hệ thống.

Task do_package

Task do_package là giai đoạn cuối trong pipeline build của BitBake, chịu trách nhiệm đóng gói các file đã cài đặt thành package triển khai trên hệ thống đích. BitBake hỗ trợ nhiều định dạng package như RPM, DEB và IPK.

2.1.4 Cross-compilation trong Yocto

Trong các hệ thống Linux Embedded, kiến trúc phần cứng của thiết bị đích thường khác với kiến trúc của máy tính dùng để phát triển phần mềm. Do đó, quá trình biên dịch mã nguồn không thể thực hiện trực tiếp bằng compiler thông thường trên hệ thống host. Để giải quyết vấn đề này, Yocto Project sử dụng cơ chế Cross-compilation nhằm hỗ trợ biên dịch phần mềm cho nhiều nền tảng phần cứng khác nhau.

Cross-compilation là quá trình biên dịch mã nguồn trên một hệ thống máy tính nhưng chương trình được tạo ra sẽ chạy trên một hệ thống khác có kiến trúc phần cứng khác biệt. Trong Linux Embedded, quá trình này thường diễn ra khi source code được build trên máy tính x86 nhưng binary output được thiết kế để chạy trên thiết bị ARM, RISC-V hoặc các nền tảng nhúng chuyên dụng khác. Cơ chế này đóng vai trò rất quan trọng trong phát triển hệ thống nhúng vì phần lớn thiết bị embedded có tài nguyên hạn chế, không đủ khả năng thực hiện quá trình biên dịch trực tiếp trên thiết bị. Việc build trên máy host giúp tận dụng hiệu năng xử lý cao của máy tính phát triển, từ đó giảm đáng kể thời gian build và tối ưu quy trình phát triển phần mềm.

Trong mô hình cross-compilation, hệ thống thường bao gồm ba thành phần chính: Build Machine, Host Machine và Target Machine. Mỗi thành phần đảm nhiệm một vai trò khác nhau trong quá trình xây dựng phần mềm. Các thành phần trong mô hình cross-compilation được trình bày trong Bảng 2.3.

Thành phần	Vai trò
Build Machine	Hệ thống thực hiện quá trình build package hoặc Embedded Linux Image bằng BitBake và Yocto Project.
Host Machine	Môi trường chứa toolchain và compiler phục vụ quá trình cross-compilation cho hệ thống đích.
Target Machine	Thiết bị nhúng đích chạy binary sau khi biên dịch như board ARM, gateway IoT hoặc hệ thống Embedded Linux.

Bảng 2.3: Mô hình Cross-compilation

2.2 Linux Device Driver

2.2.1 Tổng quan Linux Kernel

Linux Kernel là thành phần lõi của hệ điều hành Linux, đóng vai trò trung gian giữa phần cứng và các ứng dụng người dùng. Kernel chịu trách nhiệm quản lý tài nguyên hệ thống, điều phối hoạt động của phần cứng và cung cấp các dịch vụ cơ bản cho chương trình ứng dụng thông qua cơ chế system call. [2] Hoạt động ở mức đặc quyền cao nhất, kernel có quyền truy cập trực tiếp đến CPU, bộ nhớ, thiết bị ngoại vi và hệ thống lưu trữ, từ đó kiểm soát toàn bộ hoạt động của hệ thống và đảm bảo các tiến trình vận hành ổn định, an toàn và hiệu quả.

Linux sử dụng kiến trúc Monolithic Kernel, trong đó các thành phần như driver, memory manager, scheduler và file system được tích hợp trực tiếp trong kernel space, giúp đạt hiệu năng cao nhờ khả năng giao tiếp trực tiếp giữa các thành phần mà không qua nhiều lớp trung gian. Kernel hỗ trợ nhiều loại driver như Character Device Driver, Block Device Driver và Network Device Driver. Nhờ cơ chế module hóa, các driver có thể được nạp hoặc gỡ khỏi kernel trong thời gian chạy mà không cần biên dịch lại toàn bộ hệ điều hành, giúp tăng tính linh hoạt và khả năng mở rộng, đặc biệt trong các hệ thống Linux Embedded.

Ngoài ra, Linux Kernel còn đảm nhận việc quản lý hệ thống tập tin, cung cấp giao diện thống nhất để truy cập dữ liệu trên nhiều loại thiết bị lưu trữ như HDD, SSD, eMMC hay thẻ nhớ SD. Linux hỗ trợ nhiều định dạng filesystem khác nhau như EXT4, FAT32, NTFS, SquashFS và JFFS2, phù hợp cho cả hệ thống máy tính thông thường lẫn Linux Embedded. Kernel kiểm soát việc đọc, ghi và tổ chức dữ liệu trên thiết bị lưu trữ nhằm đảm bảo tính toàn vẹn và hiệu quả truy xuất; trong các hệ thống nhúng, việc lựa chọn filesystem phù hợp đóng vai trò quan trọng trong việc tối ưu tốc độ boot, độ bền bộ nhớ flash và hiệu năng tổng thể của hệ thống.

2.2.2 Linux Device Driver

Trong hệ điều hành Linux, Device Driver là thành phần phần mềm đóng vai trò trung gian giữa Linux Kernel và phần cứng hệ thống, cho phép kernel giao tiếp và điều khiển các thiết bị ngoại vi thông qua giao diện chuẩn của hệ điều hành. [3] Có thể xem Device Driver như một lớp trừu tượng giúp ứng dụng người dùng truy cập phần cứng mà không cần thao tác trực tiếp lên thanh ghi hoặc tín hiệu vật lý của thiết bị. Trong Linux, mỗi thiết bị phần cứng thường được biểu diễn dưới dạng file thiết bị trong thư mục /dev; khi ứng dụng thực hiện thao tác đọc hoặc ghi, yêu cầu sẽ được chuyển đến driver tương ứng trong kernel space để xử lý, qua đó chuẩn hóa phương thức giao tiếp với phần cứng và tăng tính độc lập giữa application và hardware platform.

Linux Device Driver chịu trách nhiệm khởi tạo thiết bị phần cứng, cấu hình thanh ghi điều khiển, truyền và nhận dữ liệu, quản lý interrupt, đồng bộ truy cập thiết bị và cung cấp giao diện giao tiếp giữa kernel space với user space. Thông qua driver, Linux Kernel có thể quản lý nhiều loại thiết bị phần cứng khác nhau mà không cần ứng dụng người dùng hiểu chi tiết cấu trúc bên trong của thiết bị, từ đó tăng tính linh hoạt và khả năng mở rộng của hệ điều hành. Trong Linux, Device Driver thường được chia thành ba nhóm chính gồm Character Device Driver, Block Device Driver và Network Device Driver, mỗi loại được thiết kế để phục vụ một kiểu giao tiếp phần cứng khác nhau, như được trình bày trong Bảng 2.4.

Loại Driver	Ví dụ	Đặc điểm
Character Device	UART	Truyền dữ liệu theo từng byte, hỗ trợ các system call như open(), read(), write() và ioctl().
Block Device	SSD	Truy cập dữ liệu theo block cố định và hỗ trợ truy cập ngẫu nhiên trên thiết bị lưu trữ.
Network Device	Ethernet	Phục vụ giao tiếp mạng và truyền nhận packet thông qua network stack của Linux Kernel.

Bảng 2.4: Các loại Linux Device Driver

2.2.3 Device Tree

Trong các hệ thống Linux Embedded hiện đại, Device Tree là cơ chế được sử dụng để mô tả cấu trúc phần cứng của hệ thống một cách độc lập với Linux Kernel. Device Tree cho phép kernel nhận biết và cấu hình các thiết bị phần cứng mà không cần chỉnh sửa trực tiếp mã nguồn kernel. Đây là thành phần quan trọng trong các nền tảng nhúng sử dụng kiến trúc ARM, RISC-V và nhiều hệ thống SoC hiện đại khác. [4]

Device Tree mô tả phần cứng dưới dạng cấu trúc cây (Tree Structure). Mỗi node trong cây đại diện cho một thành phần phần cứng như CPU, bộ nhớ, UART, SPI, I2C, GPIO hoặc các thiết bị ngoại vi khác. Các node này chứa thông tin cấu hình cần thiết để kernel và driver có thể giao tiếp với phần cứng.

Thông thường, Device Tree được viết dưới dạng file .dts (Device Tree Source). Sau đó, file này được biên dịch thành file nhị phân .dtb (Device Tree Blob) để kernel sử dụng trong quá trình boot.

Quá trình hoạt động của Device Tree được mô tả như sau:

Bootloader → Device Tree Blob (.dtb) → Linux Kernel
→ Device Driver → Hardware

Trong quá trình khởi động hệ thống, bootloader sẽ nạp Linux Kernel cùng với file Device Tree Blob vào bộ nhớ. Kernel sau đó phân tích Device Tree để xác định cấu hình phần cứng của hệ thống và tự động khởi tạo các driver tương ứng.

Một Device Tree thường bao gồm các thông tin như:

Thành phần	Mô tả
CPU Architecture	Kiến trúc bộ xử lý của hệ thống
RAM	Dung lượng bộ nhớ RAM
Register Address	Địa chỉ thanh ghi phần cứng
GPIO Configuration	Cấu hình chân GPIO
UART/SPI/I2C	Thông số các giao tiếp ngoại vi
IRQ	Thông tin Interrupt Request
Clock & Power	Quản lý xung nhịp và nguồn

Bảng 2.5: Các thông tin trong Device Tree

Ví dụ cấu hình UART trong Device Tree:

```
uart0 {  
    status = "okay";  
    current-speed = <115200>;  
};
```

Trong ví dụ trên, node `uart0` đại diện cho bộ điều khiển UART của hệ thống. Thuộc tính `status = "okay"` cho biết thiết bị được kích hoạt, trong khi `current-speed` xác định tốc độ baudrate sử dụng cho giao tiếp UART.

Trong Linux Embedded, Device Tree thường được sử dụng kết hợp với Device Driver thông qua cơ chế `compatible string`. Mỗi node phần cứng sẽ khai báo thuộc tính `compatible` để kernel xác định driver phù hợp cho thiết bị.

Ví dụ:

```
compatible = "vendor,uart-device";
```

Khi kernel khởi động, hệ thống driver model của Linux sẽ tìm driver có `compatible string` tương ứng và tự động bind driver với thiết bị phần cứng.

Trong Yocto Project, Device Tree thường được build cùng với Linux Kernel và được tích hợp trực tiếp vào Embedded Linux Image. Nhờ đó, hệ thống Linux nhúng có thể khởi động và nhận diện phần cứng một cách tự động trên nền tảng target device.

2.2.4 Kernel Module

Trong Linux Kernel, Kernel Module là thành phần phần mềm có thể được nạp hoặc gỡ khỏi kernel trong thời gian chạy mà không cần biên dịch lại hoặc khởi động lại toàn bộ hệ điều hành. Cơ chế này giúp Linux tăng tính linh hoạt trong việc mở rộng chức năng hệ thống, đặc biệt trong các môi trường Linux Embedded và hệ thống nhúng công nghiệp. [5]

Kernel Module thường được sử dụng để triển khai các thành phần như Device Driver, File System, Network Protocol hoặc các chức năng mở rộng của kernel. Thay vì tích hợp toàn bộ

chức năng trực tiếp vào Linux Kernel, các module có thể được quản lý độc lập và chỉ nạp vào hệ thống khi cần thiết. Điều này giúp giảm kích thước kernel và tối ưu tài nguyên hệ thống.

Trong Linux, kernel module sau khi biên dịch thường có phần mở rộng `.ko` (Kernel Object). File module chứa mã máy có thể được Linux Kernel nạp động vào kernel space trong quá trình runtime. Linux cung cấp nhiều công cụ để quản lý kernel module trong hệ thống như `insmod`, `rmmmod`, `modprobe` và `lsmod`.

- `insmod`: Nạp kernel module vào Linux Kernel.
- `rmmmod`: Gỡ kernel module khỏi hệ thống.
- `modprobe`: Nạp module và tự động xử lý dependency liên quan.
- `lsmod`: Hiển thị danh sách các kernel module đang hoạt động.

Và trong kernel, mỗi module thường bao gồm hai thành phần chính là hàm khởi tạo và hàm giải phóng tài nguyên. Cấu trúc cơ bản của kernel module được mô tả như sau:

- `module_init()` được gọi khi nạp module.
- `module_exit()` được gọi khi gỡ module.

Hàm `module_init()` được sử dụng để thực hiện các thao tác khởi tạo khi module được load vào kernel. Các công việc phổ biến trong hàm này bao gồm đăng ký device driver, cấp phát bộ nhớ, khởi tạo hardware hoặc thiết lập interrupt handler. Ngược lại, hàm `module_exit()` chịu trách nhiệm giải phóng tài nguyên khi module bị gỡ khỏi hệ thống. Điều này bao gồm giải phóng bộ nhớ, unregister driver hoặc đóng các kết nối phần cứng liên quan.

Một kernel module thông thường còn bao gồm nhiều thành phần metadata khác nhằm cung cấp thông tin cho Linux Kernel như tên tác giả module, license sử dụng, phiên bản module và mô tả chức năng của module.

Ví dụ:

```
MODULE_LICENSE("GPL");  
MODULE_AUTHOR("Author Name");  
MODULE_DESCRIPTION("Linux Kernel Module");
```

Các metadata này giúp kernel quản lý module và kiểm tra tính tương thích giữa module với phiên bản Linux Kernel đang sử dụng.

2.2.5 Cơ chế tự động nạp module

Trong Linux, kernel module có thể được nạp thủ công bằng các lệnh như `insmod` hoặc `modprobe`. Tuy nhiên, trong các hệ thống Linux Embedded và tự động hóa công nghiệp, nhiều driver cần được khởi tạo tự động ngay khi hệ thống khởi động nhằm đảm bảo các thiết bị phần cứng sẵn sàng hoạt động mà không yêu cầu thao tác từ người dùng.

Để hỗ trợ cơ chế này, Yocto Project cung cấp khả năng tự động nạp kernel module thông qua metadata trong recipe. Khi image Linux được build, Yocto sẽ tự động tạo cấu hình để kernel nạp module tương ứng trong quá trình boot hệ thống.

Cơ chế tự động nạp module trong Yocto thường được khai báo bằng biến:

```
KERNEL_MODULE_AUTOLOAD += "my_driver"
```

Biến `KERNEL_MODULE_AUTOLOAD` được sử dụng để chỉ định danh sách kernel module cần tự động load khi hệ thống khởi động. Khi module được khai báo trong biến này, Yocto sẽ tạo các cấu hình cần thiết trong root filesystem nhằm đảm bảo module được kernel nạp tự động trong runtime. [6]

Cơ chế hoạt động của quá trình tự động nạp module có thể được mô tả như sau:

Bootloader → Linux Kernel Boot → Root Filesystem Initialization → Load Kernel Module → Driver Ready

Trong quá trình khởi động hệ thống, Linux Kernel trước tiên sẽ khởi tạo các thành phần cơ bản như memory manager, scheduler và filesystem. Sau khi root filesystem được mount hoàn tất, hệ thống init sẽ thực hiện quá trình load các kernel module được cấu hình tự động.

Các module thường được nạp thông qua công cụ `modprobe`. Công cụ này không chỉ load module chính mà còn tự động kiểm tra và nạp các dependency cần thiết trước khi module hoạt động. Điều này giúp đảm bảo driver được khởi tạo đầy đủ và ổn định trong hệ thống.

2.2.6 Xây dựng và tích hợp Driver trong Yocto

Trong Yocto Project, quá trình xây dựng hệ thống Linux Embedded được thực hiện theo mô hình pipeline tự động hóa thông qua BitBake và metadata của recipe. Toàn bộ quy trình từ mã nguồn driver đến image Linux hoàn chỉnh đều được quản lý thống nhất nhằm đảm bảo tính tái lập và khả năng mở rộng của hệ thống.

Quy trình build tổng quát trong Yocto được mô tả như sau:

Source Code → Recipe (.bb) → BitBake → Cross Compilation → Kernel Module (.ko) → Root Filesystem → Embedded Linux Image

Trong quy trình này, source code và metadata của driver được định nghĩa thông qua recipe BitBake. BitBake sau đó phân tích dependency và thực hiện pipeline build bao gồm các giai đoạn fetch, configure, compile, install và package nhằm tạo ra kernel module tương ứng.

Pipeline build của Yocto có thể được mô tả như sau:

Fetch → Configure → Compile → Install → Package → RootFS → Image

Trong đó, BitBake sẽ tự động tải source code, cấu hình môi trường cross-compilation, biên dịch kernel module và tích hợp package vào root filesystem của Embedded Linux Image. Để BitBake có thể nhận diện recipe và metadata của driver, layer tùy chỉnh cần được tích hợp vào môi trường build bằng lệnh:

```
bitbake-layers add-layer ../meta-my-driver
```

Sau khi layer được thêm vào hệ thống, các recipe trong layer sẽ được BitBake phân tích và đưa vào dependency graph của quá trình build. Driver được tích hợp vào Embedded Linux Image thông qua biến:

```
IMAGE_INSTALL:append = " my-driver"
```

Biến này cho phép package của driver được tự động đưa vào root filesystem trong quá trình build image. Nhờ đó, hệ thống Linux Embedded sau khi boot sẽ có đầy đủ module và các thành phần cần thiết phục vụ quá trình runtime. Sau khi hệ thống khởi động, kernel module có thể được kiểm tra thông qua các lệnh quản lý module của Linux. Một số lệnh phổ biến được trình bày trong Bảng 2.6.

Lệnh	Chức năng
lsmod	Kiểm tra module đang nạp
dmesg	Kiểm tra kernel log
modinfo	Xem thông tin module

Bảng 2.6: Các lệnh kiểm tra Kernel Module

Lệnh `lsmod` cho phép kiểm tra danh sách kernel module đang hoạt động trong hệ thống. Lệnh `dmesg` được sử dụng để xem thông điệp kernel log nhằm xác nhận trạng thái hoạt động của driver. Trong khi đó, `modinfo` cung cấp thông tin metadata của kernel module như phiên bản, license và vị trí file module trong filesystem.

2.3 Edge AI trong hệ thống nhúng

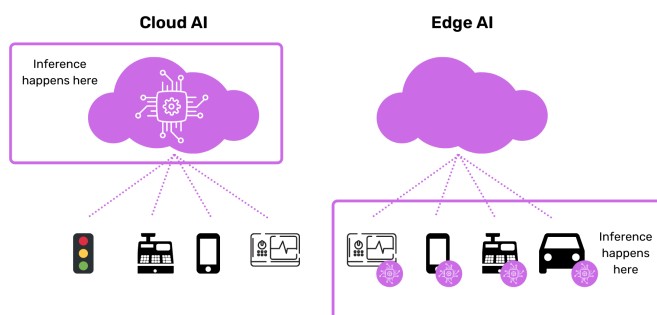
2.3.1 Khái niệm Edge AI và lợi ích so với Cloud AI

Trí tuệ nhân tạo (Artificial Intelligence – AI) là công nghệ cho phép máy tính mô phỏng các hành vi thông minh của con người như học tập, suy luận và nhận dạng. Trong các hệ thống truyền thống, mô hình AI thường được triển khai trên nền tảng điện toán đám mây (Cloud AI), nơi dữ liệu được gửi từ thiết bị đầu cuối lên máy chủ để xử lý.

Edge AI là xu hướng triển khai các mô hình trí tuệ nhân tạo trực tiếp trên thiết bị biên (edge device) như camera thông minh, gateway công nghiệp hoặc hệ thống nhúng. Thay vì truyền toàn bộ dữ liệu lên đám mây, quá trình suy luận (inference) được thực hiện ngay tại thiết bị thu thập dữ liệu.

So với Cloud AI, Edge AI mang lại nhiều lợi ích:

- Giảm độ trễ xử lý do không phụ thuộc vào kết nối mạng.
- Tăng tính ổn định của hệ thống trong môi trường công nghiệp.
- Nâng cao tính bảo mật do dữ liệu không cần truyền ra ngoài.
- Phù hợp cho các ứng dụng giám sát và xử lý thời gian thực.



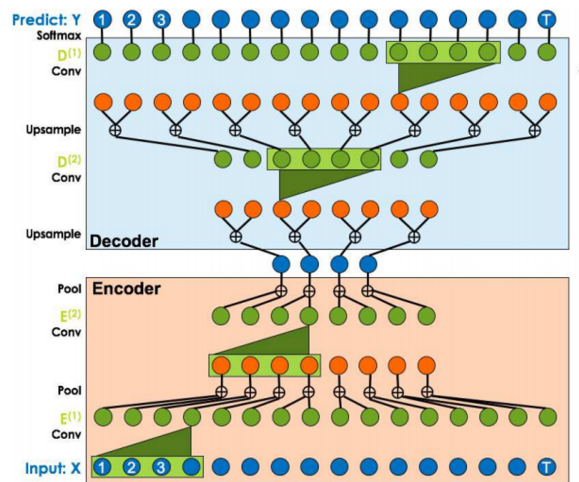
Hình 2.2: Cloud AI và Edge AI

Trong đề tài này, Edge AI được triển khai như một module phân tích dữ liệu chuỗi thời gian (Time Series), đảm nhiệm chức năng dự báo sớm nồng độ bụi mịn PM2.5 dựa trên các thông số môi trường thực tế ngay tại hiện trường.

2.3.2 Tổng quan mô hình TCN (Temporal Convolutional Network) và lý do lựa chọn TCN

2.3.2.1 Tổng quan về kiến trúc TCN

Temporal Convolutional Network (TCN) là một kiến trúc mạng nơ-ron được thiết kế chuyên biệt cho bài toán xử lý dữ liệu chuỗi thời gian. Mô hình này kết hợp ưu điểm của mạng tích chập (Convolutional Neural Network – CNN) với khả năng học các phụ thuộc dài hạn tương tự như mạng hồi tiếp (Recurrent Neural Network – RNN) [7,8].



Hình 2.3: Kiến trúc mô hình TCN

Kiến trúc TCN được xây dựng dựa trên hai nguyên lý cốt lõi:

- **Causal Convolution (Tích chập nhân quả):** Đảm bảo rằng tại thời điểm t , mô hình chỉ sử dụng các thông tin từ các thời điểm trước đó và hiện tại, tức là từ 0 đến t . Điều này giúp ngăn chặn hiện tượng “rò rỉ” thông tin từ tương lai vào quá trình dự báo.
- **Dilated Convolution (Tích chập giãn cách):** Các phần tử trong bộ lọc được áp dụng với khoảng cách giãn d , giúp mở rộng trường thụ cảm (*receptive field*) theo cấp số nhân mà không làm tăng đáng kể số lượng tham số của mô hình.

Trong đề tài này, mô hình được cấu hình với các hệ số giãn cách:

$$d = \{1, 2, 4, 8\}$$

Cấu hình này cho phép mô hình quan sát hiệu quả các biến động của nồng độ bụi mịn PM2.5 trong khoảng thời gian 30 phút trước đó, từ đó nâng cao khả năng dự báo cho các bước thời gian tiếp theo.

2.3.2.2 Lý do lựa chọn TCN thay vì LSTM/RNN

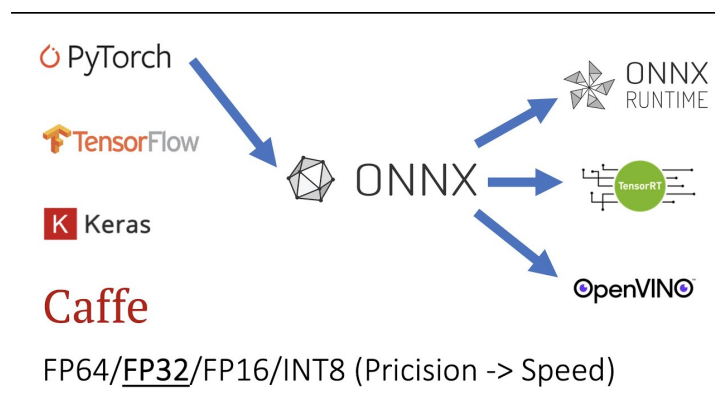
Sau quá trình thực nghiệm và tối ưu hóa cho hệ thống Edge AI, trong luận văn nhóm đã lựa chọn kiến trúc TCN thay thế cho các mô hình LSTM và RNN nhờ nhiều ưu điểm về hiệu năng và khả năng triển khai trên thiết bị nhúng. Khác với RNN và LSTM phải xử lý dữ liệu tuần tự theo từng bước thời gian, TCN sử dụng phép tích chập trên toàn bộ chuỗi dữ liệu, cho phép tận dụng khả năng tính toán song song của phần cứng nhằm giảm đáng kể thời gian huấn luyện và suy luận. Ngoài ra, cấu trúc tích chập phân tầng kết hợp với residual connection giúp TCN giảm hiện tượng mất mát đạo hàm và học ổn định hơn khi xử lý các phụ thuộc dài hạn trong chuỗi thời gian.

Bên cạnh đó, TCN cho phép điều chỉnh linh hoạt trường thụ cảm thông qua các tham số như `kernel_size` và `dilations`, giúp kiểm soát hiệu quả độ dài chuỗi dữ liệu được sử dụng để dự báo. Nhiều nghiên cứu gần đây cũng cho thấy TCN đạt hiệu quả cao trong các bài toán dự báo chất lượng không khí và nồng độ PM2.5 [7–9]. Ngoài hiệu quả dự báo, kiến trúc TCN còn có cấu trúc gọn nhẹ, số lượng tham số thấp và dễ dàng chuyển đổi sang định dạng TensorFlow Lite Float16. Trong đề tài này, mô hình sau khi lượng tử hóa đã giảm khoảng 50% dung lượng lưu trữ nhưng vẫn duy trì độ chính xác dự báo ở mức cao, phù hợp để triển khai trên các thiết bị Edge AI như Raspberry Pi 4.

2.3.3 Định dạng mô hình trung gian và tối ưu hóa trên thiết bị Edge

2.3.3.1 Định dạng mô hình ONNX (Open Neural Network Exchange)

ONNX (Open Neural Network Exchange) là một định dạng mở được thiết kế để biểu diễn các mô hình học máy, cho phép chuyển đổi linh hoạt giữa nhiều framework khác nhau như PyTorch, TensorFlow và Keras.



Hình 2.4: Định dạng ONNX

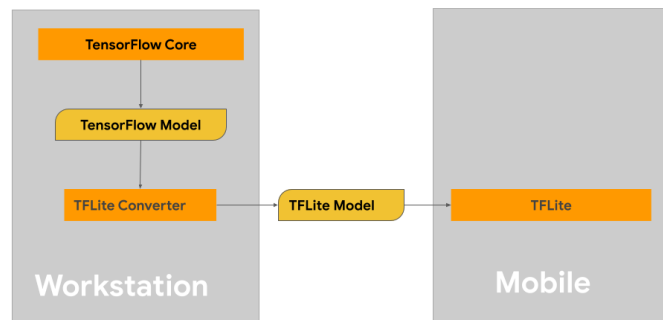
Trong đề tài này, ONNX đóng vai trò như một “ngôn ngữ chung” cho mô hình học sâu. Việc chuyển đổi mô hình sang định dạng ONNX giúp hệ thống có thể tận dụng các công cụ tối ưu

hóa và các bộ tăng tốc phần cứng khác nhau mà không phụ thuộc vào môi trường huấn luyện ban đầu.

Một ưu điểm quan trọng của ONNX là khả năng tối ưu hóa đồ thị tính toán (*graph optimization*), bao gồm việc loại bỏ các nút dư thừa, hợp nhất các phép toán liên tiếp và đơn giản hóa cấu trúc mô hình nhằm cải thiện tốc độ suy luận.

2.3.3.2 Định dạng TensorFlow Lite (TFLite) và tối ưu hóa Float16

TensorFlow Lite (TFLite) là thư viện mã nguồn mở được phát triển nhằm tối ưu hóa quá trình suy luận (*inference*) trên các thiết bị di động và hệ thống nhúng có tài nguyên phần cứng hạn chế.



Hình 2.5: Định dạng TensorFlow Lite

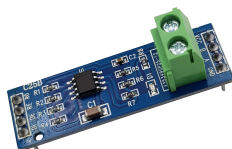
Trong hệ thống được xây dựng, kỹ thuật *Float16 Quantization* được áp dụng trong quá trình chuyển đổi mô hình (thực hiện trong file `convert.py`). Cụ thể, các trọng số của mô hình được chuyển từ định dạng số thực dấu phẩy động 32-bit (Float32) sang 16-bit (Float16).

Việc lượng tử hóa Float16 mang lại các lợi ích sau:

- **Giảm dung lượng mô hình:** Kích thước tệp mô hình giảm khoảng 50% so với mô hình gốc, giúp tiết kiệm bộ nhớ lưu trữ trên thiết bị nhúng.
- **Tăng tốc độ suy luận:** Nhiều bộ xử lý hiện đại hỗ trợ tính toán Float16 hiệu quả, giúp giảm độ trễ (*latency*) khi dự báo chỉ số PM2.5 theo thời gian thực.
- **Duy trì độ chính xác cao:** So với lượng tử hóa Int8, định dạng Float16 thường giữ được độ chính xác gần như tương đương với mô hình gốc, đồng thời ít gây sai lệch kết quả dự báo.

2.4 Giao tiếp phần cứng công nghiệp

2.4.1 Chuẩn vật lý RS485



Hình 2.6: RS485 TTL

Chuẩn RS485 (Recommended Standard 485) là một chuẩn truyền thông ở tầng vật lý, được Hiệp hội Công nghiệp Điện tử (EIA) ban hành nhằm phục vụ các hệ thống truyền dữ liệu nối tiếp trong môi trường công nghiệp. RS485 được thiết kế để đáp ứng yêu cầu truyền dữ liệu ổn định trên khoảng cách xa, chống nhiễu tốt và hỗ trợ nhiều thiết bị cùng kết nối trên một đường truyền, do đó được sử dụng rộng rãi trong các hệ thống điều khiển, giám sát và tự động hóa công nghiệp.

Về mặt nguyên lý hoạt động, RS485 sử dụng phương pháp truyền tín hiệu vi sai (differential signaling) thông qua hai dây tín hiệu A và tín hiệu B. Dữ liệu được biểu diễn bằng sự chênh lệch điện áp giữa hai dây này thay vì so với mass (hoặc GND) như các chuẩn truyền đơn cực. Cách truyền vi sai giúp RS485 có khả năng chống nhiễu điện từ cao, đặc biệt phù hợp với môi trường công nghiệp nơi tồn tại nhiều nguồn nhiễu như động cơ, biến tần và thiết bị công suất lớn.

Nhờ các đặc tính nổi bật như khả năng truyền xa, chống nhiễu tốt, hỗ trợ nhiều thiết bị và chi phí triển khai thấp, chuẩn RS485 vẫn giữ vai trò quan trọng trong các hệ thống công nghiệp hiện đại. Mặc dù đã xuất hiện nhiều chuẩn truyền thông mới, RS485 vẫn là lựa chọn ưu tiên trong các ứng dụng yêu cầu độ tin cậy cao, cấu trúc đơn giản và khả năng hoạt động ổn định trong môi trường khắc nghiệt.

Trong hệ thống IoT và Linux Embedded, RS485 thường được kết hợp với các giao thức truyền thông công nghiệp như Modbus RTU nhằm hỗ trợ trao đổi dữ liệu giữa gateway và các cảm biến hoặc thiết bị điều khiển.

2.4.2 Giao thức Modbus RTU và cấu trúc khung tin

2.4.2.1 Giao thức Modbus RTU

Modbus RTU (Remote Terminal Unit) là một giao thức truyền thông công nghiệp phổ biến, được phát triển bởi Modicon vào năm 1979. Đây là giao thức dạng master-slave (hiện nay còn

gọi là client-server), được thiết kế nhằm cho phép các thiết bị điều khiển và giám sát trong công nghiệp trao đổi dữ liệu với nhau một cách đơn giản, tin cậy và hiệu quả. Nhờ tính mở, dễ triển khai và khả năng tương thích cao, Modbus RTU hiện vẫn được sử dụng rộng rãi trong các hệ thống SCADA, PLC, cảm biến và thiết bị đo lường công nghiệp.

Trong mô hình hoạt động của Modbus RTU, thiết bị master đóng vai trò khởi tạo và điều khiển toàn bộ quá trình truyền thông, trong khi các thiết bị slave chỉ phản hồi khi nhận được yêu cầu hợp lệ từ master. Mỗi thiết bị slave được gán một địa chỉ duy nhất trên bus, giúp master xác định đúng thiết bị cần giao tiếp. Cơ chế này giúp tránh xung đột dữ liệu và đảm bảo tính tuần tự trong quá trình truyền thông.

2.4.2.2 Cấu trúc khung tin

Modbus RTU truyền dữ liệu dưới dạng các khung tin (frame) được mã hóa theo dạng nhị phân, cho phép tiết kiệm băng thông và nâng cao hiệu quả truyền thông. Mỗi khung tin Modbus RTU bao gồm các trường dữ liệu được sắp xếp theo thứ tự xác định, đảm bảo thiết bị nhận có thể giải mã chính xác nội dung thông tin.

Một khung tin Modbus RTU tiêu chuẩn bao gồm các thành phần chính: địa chỉ thiết bị, mã hàm, vùng dữ liệu và mã kiểm tra lỗi CRC. Khung tin được phân tách với các khung khác bằng một khoảng im lặng trên đường truyền (silent interval), có độ dài tối thiểu tương đương 3.5 ký tự, giúp thiết bị nhận xác định ranh giới giữa các khung dữ liệu.

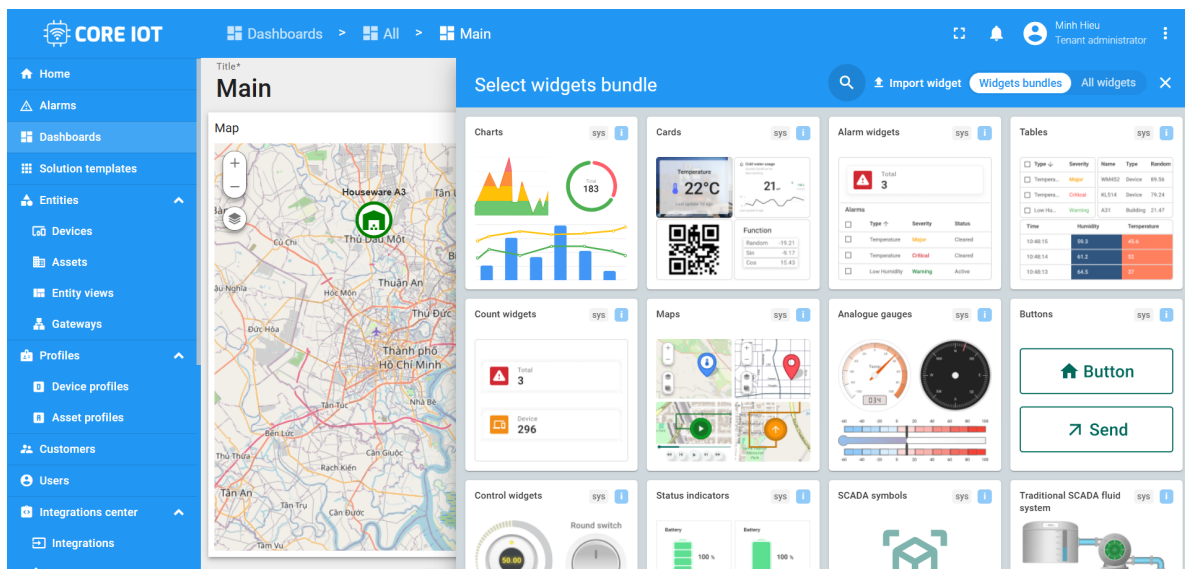
Trường địa chỉ thiết bị có độ dài 1 byte, dùng để xác định slave mà master muốn giao tiếp. Địa chỉ hợp lệ nằm trong khoảng từ 1 đến 247, trong khi địa chỉ 0 được sử dụng cho chế độ broadcast, cho phép master gửi lệnh đến tất cả các slave mà không yêu cầu phản hồi.

Mỗi khung tin Modbus RTU kết thúc bằng trường CRC (Cyclic Redundancy Check) dài 2 byte. CRC được sử dụng để kiểm tra lỗi trong quá trình truyền dữ liệu, giúp phát hiện các lỗi do nhiễu hoặc sai lệch tín hiệu trên đường truyền. Slave hoặc master sẽ tự động tính toán CRC của khung tin nhận được và so sánh với giá trị CRC đi kèm để xác định tính toàn vẹn của dữ liệu.

Mặc dù đã xuất hiện nhiều giao thức truyền thông công nghiệp hiện đại hơn, Modbus RTU vẫn giữ vai trò quan trọng trong thực tế nhờ độ ổn định, tính phổ biến và khả năng tương thích rộng rãi. Việc hiểu rõ nguyên lý hoạt động và cấu trúc khung tin Modbus RTU là nền tảng quan trọng cho việc thiết kế, triển khai và vận hành các hệ thống giám sát và điều khiển công nghiệp.

2.5 Kết nối Cloud và IoT

2.5.1 Tổng quan về CoreIoT



Hình 2.7: Tổng quan về CoreIoT

CoreIoT là một nền tảng Internet of Things (IoT) được phát triển và xây dựng dựa trên định hướng tương tự các nền tảng IoT mã nguồn mở phổ biến như ThingsBoard, nhằm phục vụ nhu cầu giảng dạy, nghiên cứu và triển khai các bài toán IoT trong môi trường học thuật và đào tạo kỹ thuật. Nền tảng được tùy biến và phát triển lại để phù hợp với điều kiện sử dụng, ngôn ngữ và đối tượng người dùng là sinh viên, giảng viên và các nhóm nghiên cứu tại nhà trường.

Về mặt kiến trúc tổng thể, CoreIoT được thiết kế theo mô hình nền tảng trung tâm, cho phép thu thập, xử lý, lưu trữ và hiển thị dữ liệu từ các thiết bị IoT phân tán. Hệ thống đóng vai trò trung gian giữa các thiết bị đầu cuối và người dùng, hỗ trợ quản lý thiết bị, giám sát dữ liệu theo thời gian thực và xây dựng các ứng dụng IoT hoàn chỉnh. Cách tiếp cận này giúp người học dễ dàng tiếp cận các khái niệm cốt lõi của IoT mà không cần triển khai toàn bộ hạ tầng từ đầu.

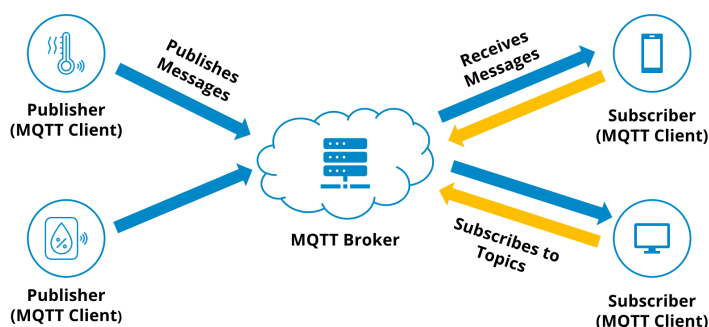
CoreIoT hỗ trợ kết nối với các thiết bị IoT thông qua các giao thức truyền thông phổ biến, đặc biệt phù hợp với các thiết bị nhúng và hệ thống công nghiệp. Dữ liệu từ cảm biến sau khi được thu thập tại thiết bị biên sẽ được truyền về nền tảng để xử lý và lưu trữ tập trung. Thông qua đó, CoreIoT cho phép theo dõi trạng thái thiết bị, phân tích dữ liệu và hiển thị thông tin trực quan trên dashboard, hỗ trợ người dùng giám sát hệ thống một cách hiệu quả.

Bên cạnh chức năng giám sát, CoreIoT còn hỗ trợ xây dựng các quy tắc xử lý dữ liệu và sự kiện thông qua cơ chế xử lý logic ở tầng nền tảng. Các quy tắc này cho phép hệ thống phản ứng tự động trước các điều kiện cụ thể, chẳng hạn như phát cảnh báo khi giá trị cảm biến vượt

ngưỡng hoặc ghi nhận sự kiện phục vụ phân tích sau này. Cách tiếp cận này giúp người học hiểu rõ hơn về mô hình xử lý dữ liệu và tự động hóa trong các hệ thống IoT hiện đại.

Với định hướng phục vụ đào tạo và nghiên cứu, CoreIoT không chỉ đóng vai trò là một công cụ giám sát mà còn là môi trường thực hành giúp sinh viên tiếp cận các khái niệm quan trọng như quản lý thiết bị, truyền thông IoT, xử lý dữ liệu thời gian thực và tích hợp hệ thống. Nền tảng tạo điều kiện thuận lợi cho việc triển khai các đề tài học tập, đồ án và nghiên cứu ứng dụng, đồng thời có khả năng mở rộng để đáp ứng các bài toán IoT phức tạp hơn trong tương lai.

2.5.2 Giao thức MQTT trong hệ thống nhúng



Hình 2.8: Giao thức MQTT trong hệ thống nhúng

MQTT (Message Queuing Telemetry Transport) là một giao thức truyền tin nhẹ, được thiết kế theo mô hình publish–subscribe, đặc biệt phù hợp cho các hệ thống IoT có tài nguyên hạn chế và yêu cầu truyền dữ liệu với độ trễ thấp. Trong mô hình này, các thiết bị không giao tiếp trực tiếp với nhau mà thông qua một thành phần trung gian gọi là broker. Thiết bị gửi dữ liệu sẽ đóng vai trò publisher, trong khi thiết bị nhận dữ liệu là subscriber, và broker chịu trách nhiệm phân phối thông điệp dựa trên các chủ đề (topic).

Một đặc điểm nổi bật của MQTT là kích thước gói tin nhỏ, giúp giảm băng thông và tiết kiệm năng lượng, rất phù hợp với các thiết bị nhúng và mạng truyền thông không ổn định. MQTT hỗ trợ nhiều mức chất lượng dịch vụ (Quality of Service – QoS), cho phép cân bằng giữa độ tin cậy và hiệu năng truyền thông. Bên cạnh đó, giao thức còn hỗ trợ cơ chế giữ kết nối (keep-alive), thông báo trạng thái thiết bị và khả năng mở rộng linh hoạt trong các hệ thống IoT quy mô lớn. Nhờ các đặc tính trên, MQTT thường được sử dụng trong các ứng dụng giám sát thời gian thực, thu thập dữ liệu cảm biến, hệ thống điều khiển từ xa và các nền tảng IoT trung tâm như CoreIoT hoặc các nền tảng tương đương.

Chương 3

Thiết kế hệ thống

Chương này tập trung vào việc phân tích bài toán quản lý cảnh báo trong nhà kho, xác định các yêu cầu của hệ thống và xây dựng thiết kế các chức năng cho hệ thống giám sát.

3.1 Phân tích yêu cầu

3.2 Thiết kế nền tảng

3.3 Thiết kế khối xử lý

3.1 Phân tích yêu cầu

3.1.1 Mô tả bài toán

Trong lĩnh vực quản lý kho hàng, đặc biệt đối với các kho lưu trữ lương thực, thực phẩm và nguyên vật liệu công nghiệp, các điều kiện môi trường như nhiệt độ và độ ẩm đóng vai trò rất quan trọng trong việc đảm bảo chất lượng hàng hóa. Nếu các thông số này vượt quá ngưỡng cho phép, hàng hóa có thể bị hư hỏng, giảm chất lượng hoặc gây tổn thất kinh tế lớn. Vì vậy, hệ thống cần có khả năng giám sát liên tục các thông số môi trường và đưa ra cảnh báo kịp thời khi phát hiện bất thường.

Bên cạnh đó, trong mỗi kho thường tồn tại nhiều thiết bị và máy móc như quạt hút gió, động cơ, dây chuyền sản xuất hoặc hệ thống điện công suất lớn. Các thiết bị này tiềm ẩn nguy cơ xảy ra sự cố như quá nhiệt, chập điện hoặc cháy nổ nếu không được theo dõi và kiểm soát chặt chẽ. Ngoài ra, một khu vực có thể bao gồm nhiều kho hoạt động đồng thời, dẫn đến nhu cầu quản lý tập trung và giám sát từ xa trở nên cần thiết nhằm đảm bảo an toàn và nâng cao hiệu quả vận hành.

Xuất phát từ thực tế đó, bài toán đặt ra là xây dựng một hệ thống nhúng IoT có khả năng thu thập dữ liệu từ các cảm biến nhiệt độ, độ ẩm, khói và khí gas để giám sát môi trường kho theo thời gian thực. Hệ thống cần hỗ trợ cảnh báo tức thời khi phát hiện nguy cơ cháy nổ hoặc điều kiện môi trường vượt ngưỡng an toàn, đồng thời cho phép quản lý nhiều kho hàng tập trung thông qua nền tảng giám sát từ xa. Ngoài chức năng giám sát, hệ thống còn cần đảm bảo tính ổn định, khả năng mở rộng và bảo mật nhằm đáp ứng yêu cầu vận hành liên tục trong môi trường công nghiệp.



Hình 3.1: Mô tả bài toán

3.1.2 Yêu cầu hệ thống

3.1.2.1 Yêu cầu chức năng

Bảng 3.1: Yêu cầu chức năng của hệ thống

Mã	Chức năng	Yêu cầu
FR-01	Thu thập dữ liệu	Hệ thống phải thu thập dữ liệu cảm biến nhiệt độ, độ ẩm, PM2.5 và ánh sáng với chu kỳ tối đa 5 giây/lần.
FR-02	Quản lý driver	Hệ thống phải hỗ trợ giao tiếp I2C và RS485 thông qua driver Linux nhúng.
FR-03	Tiền xử lý dữ liệu	Dữ liệu cảm biến phải được chuẩn hóa và kiểm tra ngưỡng trước khi xử lý.
FR-04	Edge Computing	Gateway phải xử lý dữ liệu và đưa ra quyết định cục bộ với thời gian phản hồi dưới 2 giây.
FR-05	Phân tích cảnh báo	Hệ thống phải phân loại trạng thái thành NORMAL, WARNING, CRITICAL và EMERGENCY_FIRE.
FR-06	Điều khiển thiết bị	Hệ thống phải hỗ trợ điều khiển relay và quạt làm mát từ Dashboard hoặc Rule Chain.
FR-07	Giám sát hệ thống	Dashboard phải cập nhật telemetry và trạng thái thiết bị theo thời gian thực dưới 5 giây.
FR-08	Bảo mật hệ thống	Hệ thống phải triển khai SELinux, dm-verity và xác thực package trên Yocto Linux.
FR-09	Build và custom Yocto	Hệ thống phải hỗ trợ tự động build và custom image Yocto Linux.

3.1.2.2 Yêu cầu phi chức năng

Bảng 3.2: Yêu cầu phi chức năng của hệ thống

Mã	Yêu cầu	Mô tả
NFR-01	Bảo mật	Dữ liệu MQTT phải được mã hóa bằng SSL/TLS và hệ thống phải giới hạn truy cập trái phép.
NFR-02	Hiệu năng	Hệ thống phải xử lý và gửi telemetry với độ trễ nhỏ hơn 5 giây.
NFR-03	Độ ổn định	Gateway phải hoạt động liên tục tối thiểu 24 giờ mà không bị treo hoặc mất dữ liệu.
NFR-04	Khả năng mở rộng	Hệ thống phải hỗ trợ mở rộng tối thiểu 10 node IoT hoạt động đồng thời.
NFR-05	Tính module	Driver, Rule Chain và các service phải hoạt động độc lập và dễ dàng mở rộng.
NFR-06	Độ tin cậy	Hệ thống phải duy trì kết nối ổn định với MQTT Broker trong môi trường công nghiệp.

3.2 Kiến trúc tổng quan hệ thống

3.3 Thiết kế phần cứng hệ thống

3.3.1 Raspberry Pi4

Trong phạm vi đề tài, Raspberry Pi 4 Model B phiên bản 8GB được lựa chọn làm nền tảng phần cứng trung tâm cho hệ thống Linux Embedded. Việc lựa chọn này dựa trên nhiều yếu tố liên quan đến hiệu năng xử lý, khả năng mở rộng phần cứng, mức độ hỗ trợ hệ điều hành và tính phù hợp đối với các ứng dụng IoT công nghiệp.

Trước hết, Raspberry Pi 4 được trang bị bộ vi xử lý Broadcom BCM2711 kiến trúc ARM Cortex-A72 64-bit với 4 nhân xử lý, cho phép đáp ứng đồng thời nhiều tác vụ như xử lý driver kernel, truyền dữ liệu MQTT, hiển thị giao diện dashboard và thực thi mô-đun AI phân tích dữ liệu. Phiên bản RAM 8GB cũng cung cấp dung lượng bộ nhớ đủ lớn để triển khai các thành phần AI và các dịch vụ Linux Embedded mà không gây thiếu hụt tài nguyên hệ thống.

Các nền tảng vi điều khiển khác như Arduino hoặc ESP32 có ưu điểm về chi phí thấp, mức tiêu thụ năng lượng nhỏ và dễ triển khai đối với các ứng dụng điều khiển đơn giản. Tuy nhiên, các nền tảng này không hỗ trợ đầy đủ hệ điều hành Linux, khả năng đa nhiệm còn hạn chế và khó đáp ứng các yêu cầu xử lý phức tạp như giao tiếp mạng, triển khai AI hoặc tích hợp đồng thời nhiều dịch vụ hệ thống. Do đó, nhóm vi điều khiển này chưa phù hợp với mục tiêu xây dựng một nền tảng Linux Embedded hoàn chỉnh trong đề tài.

Bên cạnh đó, một số bo mạch nhúng hỗ trợ Linux như BeagleBone Black, ODROID-C4 hoặc NanoPi NEO2 cũng được xem xét trong quá trình lựa chọn phần cứng. Mặc dù các nền tảng này hỗ trợ nhiều giao tiếp ngoại vi và có khả năng chạy Yocto Project, tuy nhiên hiệu năng xử lý, dung lượng bộ nhớ và hệ sinh thái phát triển vẫn còn những hạn chế nhất định. Ngoài ra, mức độ phổ biến của các nền tảng này không cao bằng Raspberry Pi, dẫn đến việc thiếu tài liệu kỹ thuật, cộng đồng hỗ trợ và các package phần mềm tối ưu cho Linux Embedded.

Sau quá trình đánh giá, Raspberry Pi 4 Model B được lựa chọn làm nền tảng trung tâm của hệ thống nhờ khả năng cân bằng giữa hiệu năng xử lý, tài nguyên phần cứng, khả năng hỗ trợ Yocto Project và mức độ hoàn thiện của hệ sinh thái phần mềm.

Bảng 3.3: So sánh các nền tảng phần cứng hỗ trợ Yocto Project

Board	Yocto	CPU	RAM	Đánh giá tổng quan
Raspberry Pi 4	Tốt	A72	2–8 GB	Hỗ trợ Yocto ổn định, cộng đồng lớn, nhiều giao tiếp ngoại vi, phù hợp cho Linux Embedded và AIoT
BeagleBone Black	Khá	A8	512 MB	Hỗ trợ giao tiếp công nghiệp tốt nhưng hạn chế về hiệu năng và dung lượng RAM
ODROID-C4	Khá	A55	4 GB	Hiệu năng CPU tốt, hỗ trợ Yocto nhưng hệ sinh thái và tài liệu chưa phong phú
NanoPi NEO2	Trung bình	A53	512 MB–1 GB	Chi phí thấp, kích thước nhỏ gọn nhưng hạn chế khả năng mở rộng hệ thống

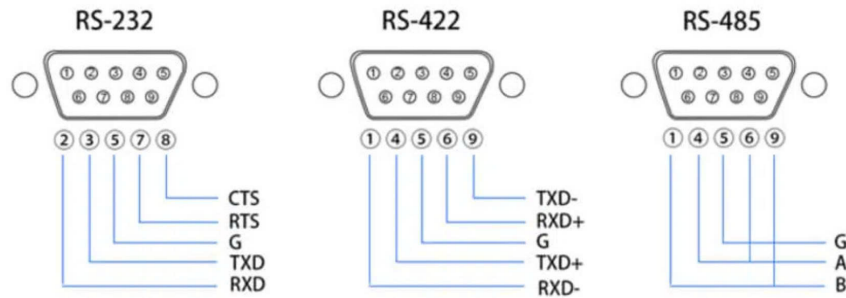
Từ các kết quả đánh giá và so sánh, có thể thấy Raspberry Pi 4 Model B đáp ứng tốt các yêu cầu của hệ thống Linux Embedded trong đề tài, bao gồm khả năng xử lý đa nhiệm, hỗ trợ giao tiếp phần cứng, tích hợp AIoT và khả năng tùy biến thông qua Yocto Project.

3.3.2 Chuẩn giao tiếp vật lý RS485

Trong các hệ thống nhúng và tự động hóa công nghiệp, việc lựa chọn chuẩn truyền thông đóng vai trò quan trọng đối với độ ổn định và khả năng mở rộng của hệ thống. Trong phạm vi đề tài, chuẩn truyền thông RS485 được lựa chọn để kết nối giữa Raspberry Pi 4 và cảm biến môi trường thay vì các chuẩn truyền thông nối tiếp khác như RS232 hoặc RS422.

Chuẩn RS232 có ưu điểm là đơn giản và dễ triển khai, tuy nhiên khoảng cách truyền ngắn, khả năng chống nhiễu thấp và chỉ hỗ trợ giao tiếp point-to-point. Điều này khiến RS232 không phù hợp với môi trường công nghiệp, nơi thường tồn tại nhiễu điện từ và yêu cầu truyền dữ liệu trên khoảng cách lớn.

Trong khi đó, RS422 hỗ trợ truyền xa và chống nhiễu tốt hơn RS232 nhờ cơ chế truyền vi sai. Tuy nhiên, chuẩn này chủ yếu hỗ trợ giao tiếp một chiều giữa một master và nhiều slave, đồng thời khả năng giao tiếp đa điểm còn hạn chế.



Hình 3.2: Các chuẩn giao tiếp vật lý công nghiệp

So với các chuẩn trên, RS485 hỗ trợ cơ chế truyền vi sai giúp tăng khả năng chống nhiễu và đảm bảo độ ổn định trong môi trường công nghiệp. Chuẩn này cho phép nhiều thiết bị cùng kết nối trên một bus truyền thông, hỗ trợ khoảng cách truyền lớn và có khả năng hoạt động ổn định trong điều kiện nhiễu điện từ cao. Ngoài ra, RS485 cũng được sử dụng phổ biến trong các giao thức công nghiệp như Modbus RTU, giúp thuận lợi cho việc tích hợp cảm biến và mở rộng hệ thống trong tương lai.

Bên cạnh đó, RS485 có cấu trúc phần cứng tương đối đơn giản, dễ tích hợp với UART của Raspberry Pi thông qua IC chuyển đổi MAX485. Điều này giúp giảm chi phí triển khai nhưng vẫn đảm bảo hiệu quả truyền nhận dữ liệu trong hệ thống Linux Embedded.

Từ các đặc điểm trên, RS485 được đánh giá là phù hợp hơn cho bài toán giám sát môi trường công nghiệp của đề tài, đặc biệt trong các yêu cầu về độ ổn định, khả năng chống nhiễu và hỗ trợ mở rộng hệ thống IoT.

3.4 Thiết kế nền tảng nhúng

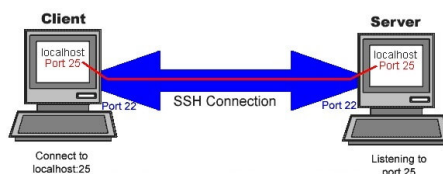
3.4.1 Thiết kế hệ thống bảo mật cho Linux nhúng

3.4.1.1 Xây dựng cơ chế kiểm soát truy cập và phân quyền

Hệ thống thiết kế cơ chế phân quyền người dùng bằng sudo nhằm giới hạn phạm vi thao tác trên thiết bị Yocto Linux. Tài khoản gatewayadmin được cấp quyền thực thi và quản lý các ứng dụng cần thiết của hệ thống, tuy nhiên không được phép thực hiện các thao tác ảnh hưởng đến toàn bộ hệ thống như chỉnh sửa file hệ thống, reboot thiết bị hoặc thay đổi cấu hình bảo mật.

Các quyền được cấu hình thông qua file `/etc/sudoers.d/` nhằm kiểm soát chính xác các lệnh mà người dùng được phép thực thi. Cơ chế này giúp tăng cường bảo mật và hạn chế các thao tác trái phép trên hệ thống Linux nhúng.

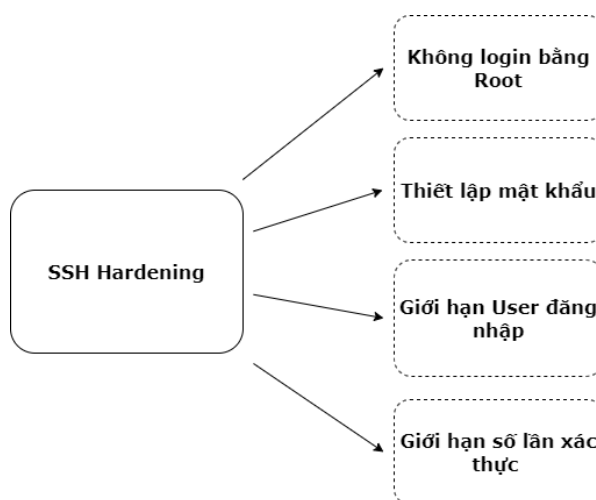
3.4.1.2 Tăng cường bảo mật dịch vụ SSH



Hình 3.3: Cơ chế SSH

SSH (Secure Shell) là giao thức được sử dụng để truy cập và quản trị hệ thống Linux từ xa. Trong các hệ thống Linux nhúng xây dựng bằng Yocto Linux, SSH đóng vai trò quan trọng trong quá trình cấu hình, giám sát và bảo trì thiết bị. Tuy nhiên, dịch vụ SSH cũng là một trong những mục tiêu tấn công phổ biến do thường được mở trên môi trường mạng.

Để nâng cao mức độ an toàn cho thiết bị, Nhóm tiến hành thiết kế cơ chế SSH Hardening nhằm tăng cường khả năng bảo mật và kiểm soát truy cập đối với dịch vụ SSH. Kiến trúc tổng quan của cơ chế bảo mật SSH được thể hiện trong Hình 3.4.



Hình 3.4: Thiết kế cơ chế tăng cường bảo mật SSH

Nhóm thiết lập cơ chế SSH Hardening được thiết kế bao gồm các chức năng chính như sau:

Thứ nhất, nhóm thiết kế vô hiệu hóa chức năng đăng nhập trực tiếp bằng tài khoản root thông qua giao thức SSH. Người dùng không được phép truy cập trực tiếp với quyền quản trị cao nhất mà phải đăng nhập bằng tài khoản thông thường trước khi sử dụng cơ chế nâng quyền. Điều này giúp hạn chế nguy cơ tấn công trực tiếp vào tài khoản quản trị hệ thống và giảm thiểu khả năng khai thác trái phép từ bên ngoài mạng.

Thứ hai, nhóm thiết kế cơ chế thiết lập chính sách mật khẩu nhằm tăng cường mức độ an toàn cho tài khoản người dùng khi đăng nhập SSH. Chính sách mật khẩu được thiết kế với

các yêu cầu về độ dài tối thiểu, mức độ phức tạp và thời gian sử dụng mật khẩu. Các quy tắc mật khẩu được tích hợp thông qua PAM (Pluggable Authentication Modules) trên Yocto Linux nhằm đảm bảo tính linh hoạt và khả năng quản lý tập trung trong hệ thống.

Thứ ba, nhóm thiết kế cơ chế kiểm soát danh sách người dùng được phép truy cập SSH. Chỉ các tài khoản nằm trong danh sách cho phép mới có quyền đăng nhập từ xa vào thiết bị. Cơ chế này giúp kiểm soát quyền truy cập, hạn chế việc sử dụng các tài khoản không hợp lệ và giảm bề mặt tấn công của hệ thống.

Thứ tư, để tăng cường khả năng chống lại các cuộc tấn công brute-force, hệ thống thiết kế cơ chế giới hạn số lần xác thực đăng nhập SSH. Khi số lần nhập sai vượt quá ngưỡng cho phép, kết nối đăng nhập sẽ bị từ chối hoặc tạm thời khóa trong một khoảng thời gian nhất định. Ngoài ra, hệ thống cũng thiết lập thời gian giới hạn cho quá trình xác thực nhằm tránh việc duy trì kết nối SSH không cần thiết trong thời gian dài.

3.4.1.3 Thiết kế hệ thống quản lý package an toàn

Trong các hệ thống Linux nhúng sử dụng Yocto Linux, cơ chế quản lý package đóng vai trò quan trọng trong việc cập nhật chức năng, bảo trì và mở rộng hệ thống. Tuy nhiên, nếu không có cơ chế kiểm soát phù hợp, người dùng có thể cài đặt các package không đáng tin cậy hoặc package đã bị thay đổi nội dung, từ đó ảnh hưởng đến tính toàn vẹn và mức độ an toàn của thiết bị.

Trong luận văn này, nhóm thực hiện đề xuất kiến trúc quản lý package theo hướng kiểm soát tập trung nhằm tăng cường khả năng xác thực và kiểm soát quá trình cài đặt package trên thiết bị nhúng. Hệ thống được thiết kế để mọi package trước khi cài đặt đều phải trải qua các bước kiểm tra bảo mật và xác thực nguồn gốc.

Lựa chọn package manager cho hệ thống

Trên các hệ thống Linux nhúng hiện nay, một số package manager phổ biến bao gồm DPKG, RPM và OPKG. Mỗi cơ chế quản lý package có đặc điểm riêng về kiến trúc, hiệu năng và mức độ phù hợp đối với các hệ thống nhúng có tài nguyên hạn chế. Trong quá trình nghiên cứu và thiết kế hệ thống, nhóm đã tiến hành đánh giá các package manager phổ biến nhằm lựa chọn giải pháp phù hợp cho nền tảng Yocto Linux được sử dụng trong luận văn.

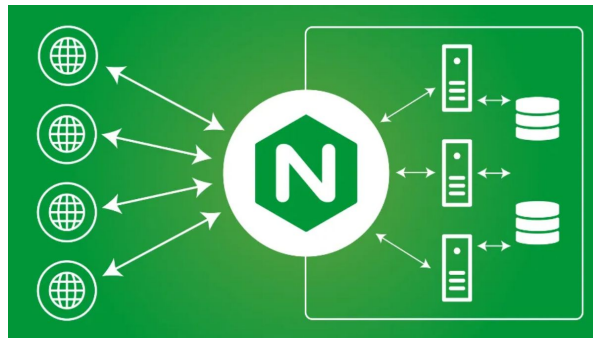
Bảng 3.4 trình bày so sánh giữa DPKG, RPM và OPKG trong môi trường Linux nhúng.

Bảng 3.4: So sánh các package manager trên Linux

Tiêu chí	DPKG	RPM	OPKG
Định dạng package	.deb	.rpm	.ipk
Dung lượng hệ thống	Trung bình	Lớn	Nhẹ
Hiệu năng Embedded	Trung bình	Trung bình	Tốt
Quản lý dependency	Đầy đủ	Đầy đủ	Đơn giản
Mức phù hợp Embedded	Khá phù hợp	Hệ thống lớn	Rất phù hợp
Hệ thống phổ biến	Ubuntu	Fedora	Yocto

Từ kết quả đánh giá, nhóm lựa chọn OPKG làm package manager chính cho hệ thống do có kiến trúc gọn nhẹ, khả năng xử lý nhanh và phù hợp với các thiết bị Linux nhúng có tài nguyên phần cứng hạn chế như Raspberry Pi 4.

Thiết kế repository quản lý package

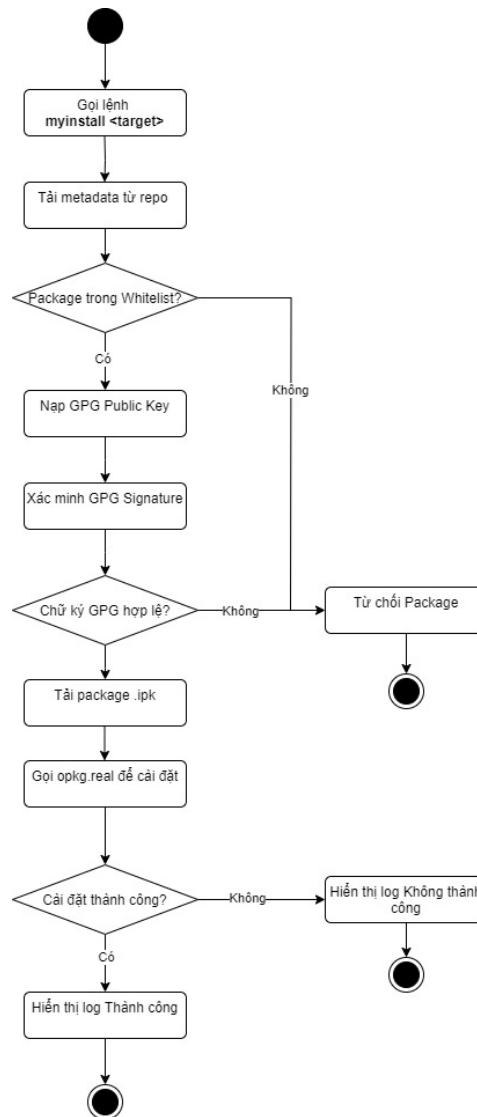


Hình 3.5: Nginx repository server cho quản lý package an toàn

Trong kiến trúc được đề xuất, nhóm sử dụng Nginx như một repository server nội bộ để lưu trữ và phân phối package đến thiết bị nhúng. Nginx là web server có hiệu năng cao, khả năng xử lý nhẹ và phù hợp cho việc cung cấp các tệp tĩnh thông qua giao thức HTTP. Repository của hệ thống bao gồm các package .ipk, file metadata Packages.gz, chữ ký số Packages.gz.asc và khóa công khai public.key.

Tổng quan thiết kế hệ thống quản lý package an toàn

Trong luận văn này, nhóm đề xuất cơ chế cài đặt package thông qua lớp trung gian thay vì cho phép người dùng truy cập trực tiếp package manager của hệ thống. Kiến trúc tổng quan của cơ chế quản lý package được mô tả như sau:



Hình 3.6: Kiến trúc quản lý package an toàn

Trong kiến trúc này, lớp trung gian `myinstall` được nhóm thiết kế nhằm thực hiện các bước xác thực package trước khi cho phép cài đặt thực tế trên hệ thống. Cách tiếp cận này giúp tăng cường khả năng kiểm soát package và hạn chế nguy cơ cài đặt package trái phép hoặc package không đáng tin cậy.

Để kiểm soát danh sách package được phép cài đặt, nhóm đề xuất cơ chế whitelist nhằm giới hạn các package hợp lệ trên hệ thống. Theo thiết kế, chỉ các package tồn tại trong danh sách cho phép mới được tiếp tục quá trình xác thực và cài đặt. Điều này giúp giảm bề mặt tấn công và hạn chế nguy cơ cài đặt các package không thuộc phạm vi quản lý của hệ thống.

Bên cạnh cơ chế whitelist, nhóm cũng thiết kế cơ chế xác thực chữ ký số GPG nhằm đảm bảo tính toàn vẹn của repository package. Ở đây, metadata repository sẽ được ký bằng khóa bí mật trên máy host và được xác thực bằng khóa công khai trên thiết bị nhúng trước khi cho phép cài đặt package. Điều này giúp hệ thống phát hiện các trường hợp package hoặc metadata

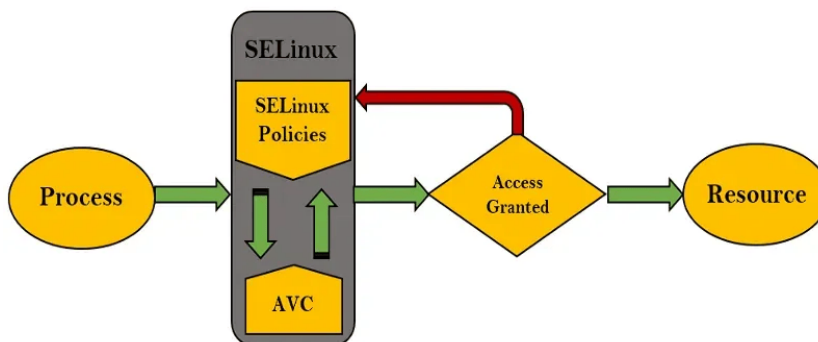
bị thay đổi nội dung trong quá trình truyền tải, từ đó hạn chế nguy cơ giả mạo repository hoặc tấn công chèn mã độc vào package.

Trong hệ thống được đề xuất, nhóm không cho phép người dùng sử dụng trực tiếp package manager mặc định của hệ thống. Thay vào đó, mọi thao tác cài đặt package phải đi qua lớp kiểm soát trung gian do nhóm xây dựng. Điều này giúp đảm bảo toàn bộ quá trình cài đặt luôn tuân theo chính sách xác thực và kiểm tra bảo mật đã được thiết kế trong hệ thống. Ngoài ra, backend package manager thực tế cũng được nhóm giới hạn quyền truy cập nhằm giảm nguy cơ khai thác trái phép hoặc bỏ qua cơ chế kiểm soát bảo mật của hệ thống.

3.4.1.4 Triển khai cơ chế bảo mật SELinux

SELinux (Security-Enhanced Linux) là cơ chế bảo mật dành cho Linux dựa trên mô hình Mandatory Access Control (MAC), cho phép kiểm soát quyền truy cập giữa tiến trình và tài nguyên hệ thống thông qua các policy bảo mật. Trong hệ thống Yocto Linux, SELinux được tích hợp thông qua layer meta-selinux nhằm tăng cường khả năng bảo vệ cho hệ điều hành nhúng.

Trong quá trình thiết kế hệ thống, đề tài sử dụng SELinux nhằm giới hạn quyền truy cập của tiến trình và giảm thiểu ảnh hưởng khi dịch vụ bị khai thác. SELinux hoạt động dựa trên security context và các policy được định nghĩa sẵn, theo nguyên tắc mặc định từ chối truy cập nếu không được cho phép rõ ràng.



Hình 3.7: Bảo mật quyền truy cập hệ thống với SELinux trong Yocto

SELinux cung cấp nhiều loại policy khác nhau phục vụ cho từng mục đích triển khai hệ thống. Bảng 3.5 trình bày một số policy phổ biến của SELinux.

Bảng 3.5: So sánh các policy trong SELinux

Policy	Đặc điểm	Mục đích sử dụng
targeted	Chỉ bảo vệ các dịch vụ và tiến trình quan trọng	Phù hợp cho hệ thống thực tế, dễ triển khai và tương thích cao
minimum	Chỉ chứa các policy tối thiểu cần thiết	Dùng cho hệ thống nhúng hoặc hệ thống yêu cầu tối giản
mls	Hỗ trợ Multi-Level Security với nhiều mức bảo mật	Sử dụng trong hệ thống yêu cầu bảo mật nghiêm ngặt

Trong đề tài này, hệ thống lựa chọn policy targeted do đây là policy phổ biến và phù hợp với môi trường Linux nhúng trên Yocto. Policy này tập trung bảo vệ các dịch vụ quan trọng như SSH, systemd hoặc web server, đồng thời vẫn đảm bảo khả năng tương thích và dễ triển khai trên thiết bị nhúng.

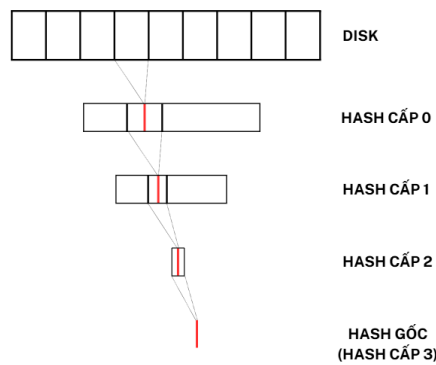
SELinux được cấu hình ở chế độ enforcing nhằm đảm bảo mọi truy cập không hợp lệ sẽ bị từ chối theo policy đã định nghĩa trước. Việc sử dụng policy mặc định giúp giảm độ phức tạp trong quá trình phát triển hệ thống, đồng thời vẫn đảm bảo tăng cường bảo mật cho thiết bị Yocto Linux.

3.4.1.5 Bảo vệ tính toàn vẹn hệ thống với dm-verity

Trong các hệ thống Linux nhúng, việc bảo vệ tính toàn vẹn của phân vùng hệ thống tập tin gốc (root filesystem) là một yêu cầu quan trọng nhằm ngăn chặn các hành vi chỉnh sửa trái phép trên thiết bị lưu trữ. Đặc biệt, trong các kịch bản tấn công offline, kẻ tấn công có thể can thiệp trực tiếp vào thẻ nhớ hoặc bộ nhớ ngoài để thay đổi dữ liệu hệ thống mà không cần truy cập vào hệ điều hành đang chạy. Để giải quyết vấn đề này, trong luận văn nhóm đã nghiên cứu và hiện thực cơ chế dm-verity trên nền tảng Linux nhúng.

dm-verity (Device Mapper Verity) là một cơ chế xác thực dữ liệu ở mức khối, được tích hợp trong Linux kernel thông qua Device Mapper. Cơ chế này sử dụng cấu trúc cây băm Merkle (Merkle Hash Tree) để kiểm tra tính toàn vẹn của từng block dữ liệu trong phân vùng rootfs. Mỗi block dữ liệu được băm để tạo ra các giá trị hash ở các cấp thấp hơn; các giá trị này tiếp tục được băm lên nhiều cấp cho đến khi thu được một hash gốc duy nhất. Hash gốc này đại diện cho toàn bộ nội dung của hệ thống tập tin và được sử dụng làm cơ sở để xác minh dữ liệu trong quá trình khởi động và truy cập hệ thống.

Hình 3.8 minh họa cấu trúc cây Merkle Hash Tree mà nhóm đã sử dụng trong quá trình nghiên cứu và hiện thực cơ chế xác thực toàn vẹn cho rootfs. Trong mô hình này, khi một block dữ liệu bị thay đổi, giá trị hash tương ứng sẽ không còn khớp với các hash ở cấp cao hơn, từ đó hệ thống có thể phát hiện sớm sự can thiệp trái phép và ngăn chặn việc sử dụng dữ liệu đã bị chỉnh sửa.



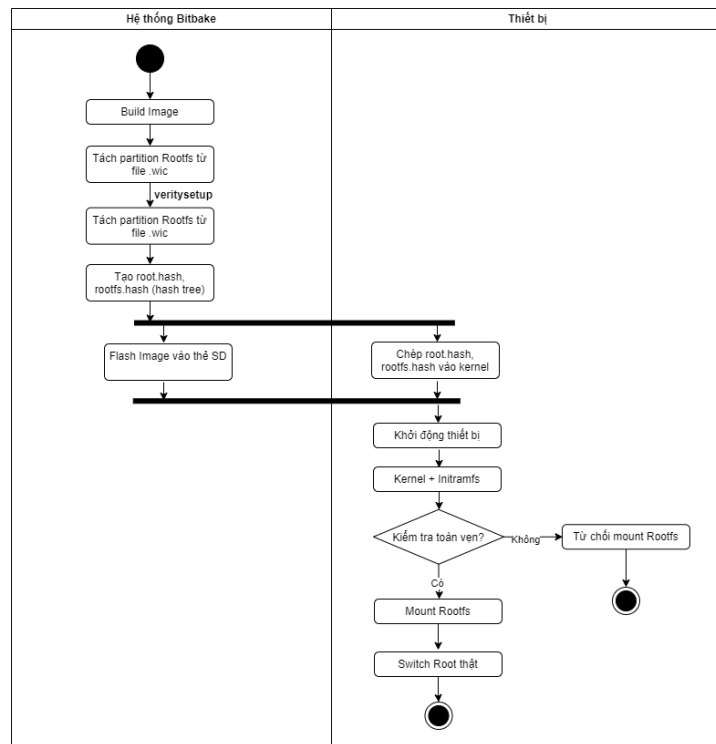
Hình 3.8: Cấu trúc Merkle Hash Tree trong cơ chế dm-verity

Trong luận văn, nhóm đã thiết kế hệ thống dm-verity theo hướng kết hợp giữa quá trình tạo ảnh hệ thống và quá trình kiểm tra tính toàn vẹn khi thiết bị khởi động. Kiến trúc tổng thể của hệ thống gồm hai giai đoạn chính: giai đoạn chuẩn bị image trên máy phát triển và giai đoạn xác thực rootfs trên thiết bị đích.

Ở giai đoạn chuẩn bị image, sau khi hoàn tất quá trình build hệ điều hành, nhóm đã tiến hành tách phân vùng rootfs từ file `.wic` và sử dụng công cụ `veritysetup` để tạo ra cây hash tương ứng. Từ đó, hệ thống sinh ra hai thành phần quan trọng là `root.hash` và `rootfs.hash`. Trong đó, `rootfs.hash` chứa dữ liệu hash tree phục vụ xác thực các block dữ liệu, còn `root.hash` là giá trị hash gốc được dùng để xác minh toàn bộ cấu trúc. Sau khi tạo xong, image được flash vào thẻ SD và giá trị hash gốc được tích hợp vào kernel hoặc `initramfs` nhằm đảm bảo tính tin cậy trong quá trình khởi động.

Ở giai đoạn khởi động thiết bị, kernel và `initramfs` sẽ khởi tạo cơ chế dm-verity trước khi mount phân vùng rootfs. Hệ thống sẽ sử dụng giá trị hash gốc đã được chuẩn bị từ trước để kiểm tra chuỗi hash của toàn bộ phân vùng. Nếu tất cả các giá trị băm đều khớp, rootfs sẽ được mount bình thường và hệ điều hành tiếp tục khởi động. Ngược lại, nếu phát hiện có sự sai lệch tại bất kỳ cấp hash nào, hệ thống sẽ từ chối mount rootfs và dừng quá trình khởi động nhằm ngăn chặn việc sử dụng dữ liệu không còn đảm bảo tính toàn vẹn.

Hình 3.9 mô tả activity diagram của hệ thống mà nhóm đã hiện thực trong luận văn. Sơ đồ cho thấy luồng xử lý bắt đầu từ bước build image, tạo hash tree, flash image vào thẻ SD, sau đó thiết bị khởi động và tiến hành kiểm tra toàn vẹn trước khi mount rootfs. Cách tổ chức này cho phép hệ thống chỉ chấp nhận image hợp lệ, đồng thời tăng cường khả năng chống lại các hình thức tấn công can thiệp offline vào thiết bị lưu trữ.



Hình 3.9: Activity diagram thiết kế dm-verity

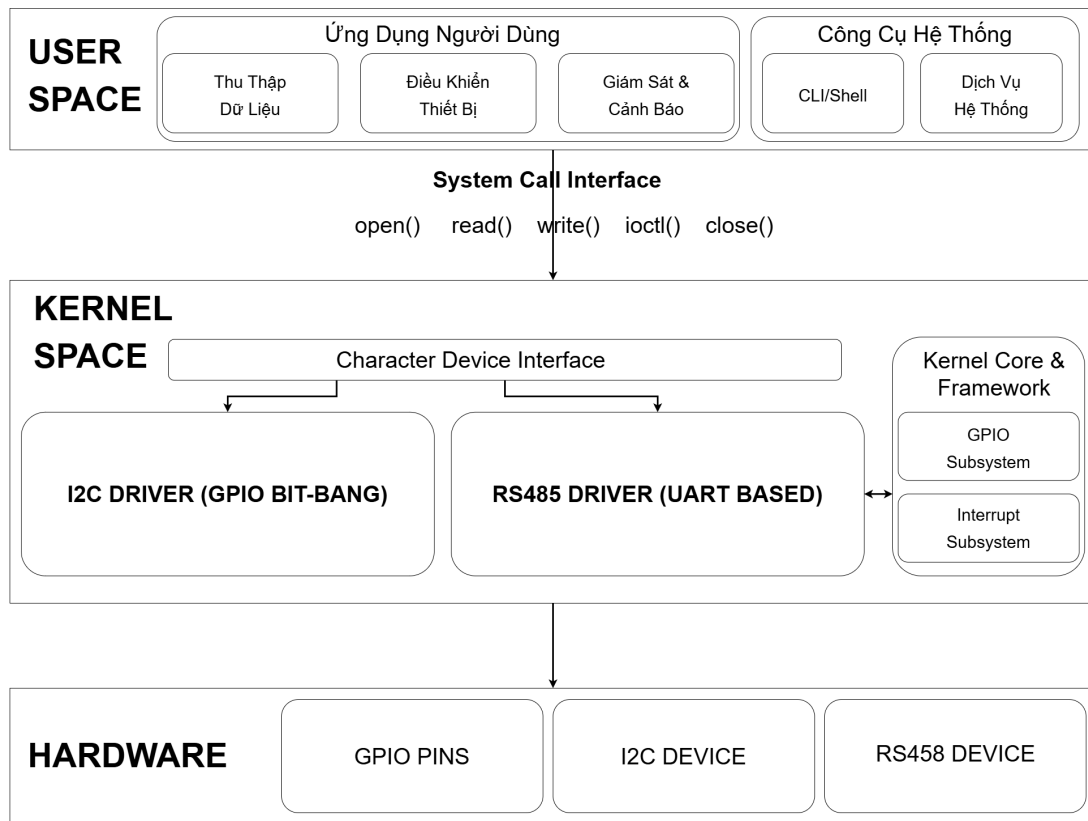
3.4.2 Phát triển Driver cho Linux nhúng

3.4.2.1 Kiến trúc tổng quan Driver trong hệ thống Yocto Linux

Trong hệ thống Linux nhúng sử dụng Yocto Project, driver đóng vai trò là lớp trung gian giữa tầng ứng dụng và phần cứng ngoại vi. Driver chịu trách nhiệm quản lý giao tiếp phần cứng, cung cấp interface chuẩn cho user space và thực hiện điều khiển thiết bị ở mức thấp thông qua Linux kernel.

Trong đề tài, nhóm không sử dụng các driver giao tiếp mặc định của hệ thống mà tự xây dựng các custom driver cho giao thức I2C và RS485 nhằm tăng tính tùy biến và phù hợp với yêu cầu của hệ thống tự động hóa công nghiệp.

Kiến trúc tổng quan của hệ thống driver được mô tả theo mô hình phân lớp như Hình 3.10.



Hình 3.10: Kiến trúc tổng quan driver trong hệ thống Yocto Linux

Trong kiến trúc này, tầng **User Space** bao gồm các ứng dụng điều khiển, dashboard giám sát và các chương trình xử lý dữ liệu cảm biến. Các ứng dụng này giao tiếp với driver thông qua các system call chuẩn của Linux như `open()`, `read()`, `write()` và `ioctl()`.

Tầng **Kernel Space** bao gồm các custom character device driver do nhóm phát triển. Các driver này chịu trách nhiệm xử lý logic giao tiếp, điều khiển GPIO, quản lý timing truyền dữ liệu và tạo các tín hiệu giao tiếp phần cứng.

Driver I2C được hiện thực theo cơ chế software bit-banging nhằm điều khiển trực tiếp hai đường tín hiệu SDA và SCL thông qua GPIO. Driver RS485 được xây dựng để quản lý truyền nhận UART và điều khiển chiều truyền dữ liệu của module RS485 phục vụ giao tiếp Modbus RTU.

Các driver được đóng gói thành kernel module và tích hợp vào hệ thống Yocto thông qua custom layer và BitBake recipe. Sau khi build image, các module driver sẽ được nạp vào Linux kernel và tạo device file trong thư mục `/dev` để user space có thể truy cập.

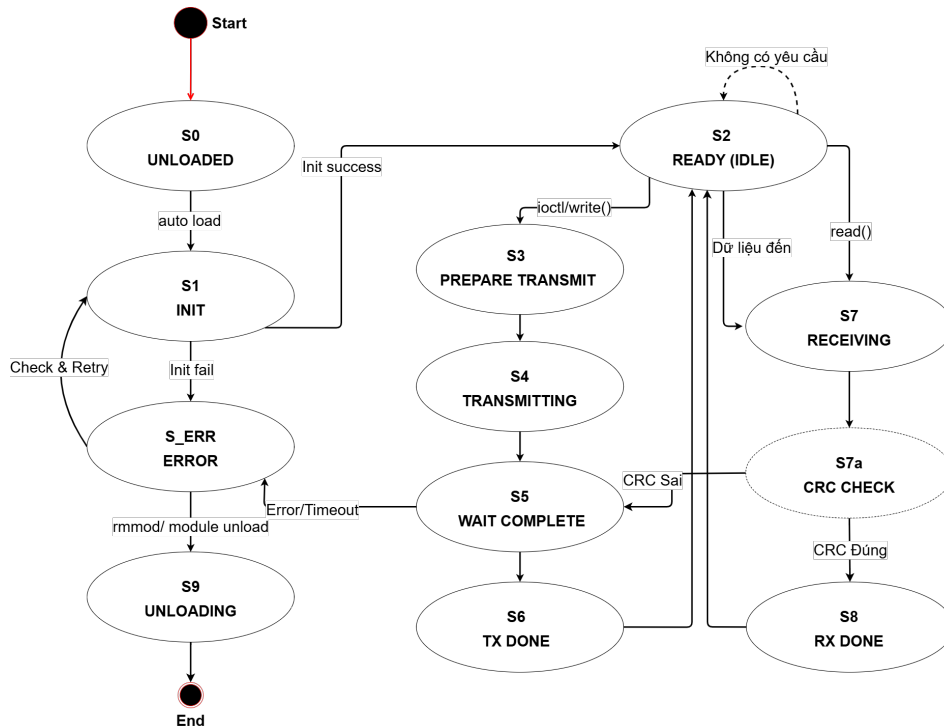
Ở tầng thấp nhất, driver giao tiếp trực tiếp với phần cứng ngoại vi như cảm biến, màn hình OLED, relay và các thiết bị công nghiệp sử dụng chuẩn RS485.

Kiến trúc này giúp hệ thống có tính mô-đun cao, dễ mở rộng và phù hợp với các ứng dụng Linux nhúng yêu cầu khả năng tùy biến giao tiếp phần cứng.

3.4.2.2 Driver RS485

Mô hình FSM của driver RS485

Để quản lý quá trình truyền nhận dữ liệu và xử lý lỗi trong môi trường Linux Kernel, driver được thiết kế theo mô hình máy trạng thái (Finite State Machine – FSM). Mô hình này giúp tổ chức luồng hoạt động của driver một cách rõ ràng, đồng thời hỗ trợ việc kiểm soát các trạng thái truyền nhận và xử lý lỗi trong quá trình giao tiếp RS485 Modbus RTU.



Hình 3.11: Máy trạng thái cho driver RS458

FSM của driver RS485 bao gồm các trạng thái chính như mô tả ở Hình 3.11. Sau khi hệ thống khởi động, driver bắt đầu tại trạng thái UNLOADED. Khi kernel module được tự động nạp thông qua cơ chế autoload, driver chuyển sang trạng thái INIT để thực hiện khởi tạo UART, GPIO điều khiển DE/RE và các tài nguyên nội bộ của kernel module.

Nếu quá trình khởi tạo thành công, driver chuyển sang trạng thái READY (IDLE) và chờ yêu cầu truyền nhận dữ liệu từ User Space hoặc kernel thread polling. Trong trường hợp khởi tạo thất bại, driver sẽ chuyển sang trạng thái ERROR, đồng thời thực hiện cơ chế retry hoặc chờ unload module khỏi hệ thống.

Khi có yêu cầu đọc hoặc ghi dữ liệu, driver chuyển sang trạng thái PREPARE TRANSMIT để xây dựng khung Modbus RTU và tính toán CRC16 cho gói tin. Sau đó dữ liệu được gửi qua UART trong trạng thái TRANSMITTING. Khi quá trình truyền hoàn tất, driver chuyển sang trạng thái WAIT COMPLETE để chờ phản hồi từ thiết bị slave.

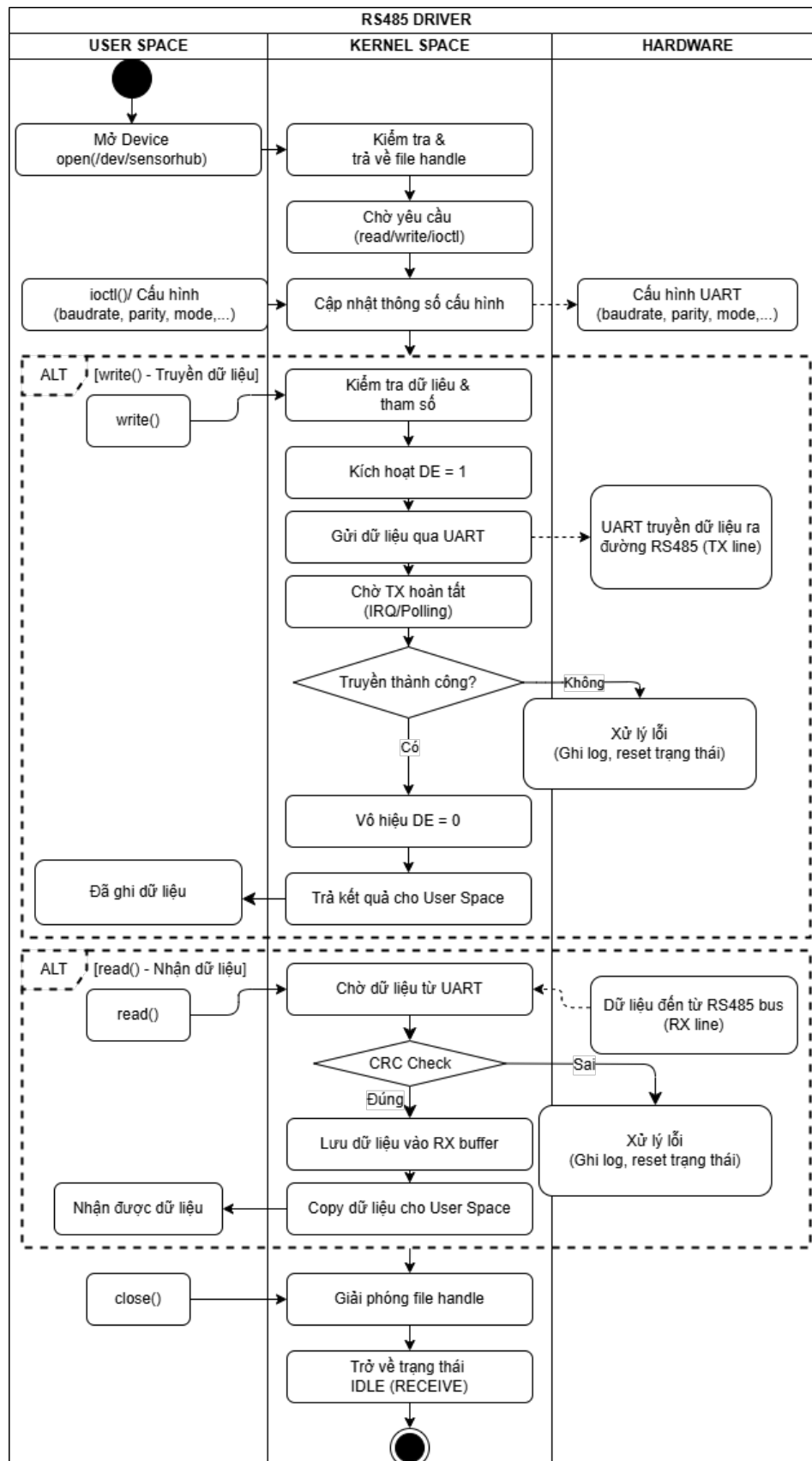
Sau khi nhận được dữ liệu phản hồi, driver thực hiện kiểm tra CRC trong trạng thái CRC CHECK. Nếu CRC hợp lệ, driver tiếp tục chuyển sang trạng thái RECEIVING để giải mã dữ liệu

thanh ghi Modbus, sau đó chuyển sang trạng thái RX DONE nhằm cập nhật cache dữ liệu và cung cấp dữ liệu cho User Space. Ngược lại, nếu CRC không hợp lệ hoặc xảy ra timeout trong quá trình truyền nhận, driver sẽ chuyển sang trạng thái ERROR để xử lý lỗi truyền thông.

Cuối cùng, khi kernel module bị gỡ khỏi hệ thống thông qua lệnh `rmmmod`, driver chuyển sang trạng thái UNLOADING để giải phóng tài nguyên và kết thúc vòng đời hoạt động của module trong Linux Kernel.

Activity Diagram của driver RS485

Trong khi FSM tập trung mô tả các trạng thái nội bộ của driver, quá trình tương tác thực tế giữa User Space, Kernel Space và phần cứng UART được thể hiện thông qua Activity Diagram. Sơ đồ này mô tả tuần tự các bước xử lý khi ứng dụng thực hiện các thao tác đọc, ghi và cấu hình thiết bị thông qua system call trong Linux.



Hình 3.12: Activity Diagram của driver RS485

Hình 3.12 mô tả luồng hoạt động tổng quát của driver RS485 trong môi trường Linux Em-

bedded, bao gồm sự tương tác giữa User Space, Kernel Space và phần cứng UART/RS485. Quá trình hoạt động bắt đầu khi ứng dụng ở User Space thực hiện lời gọi hệ thống `open()` để truy cập device file `/dev/sensorhub`. Khi đó, driver trong Kernel Space sẽ kiểm tra và khởi tạo file handle trước khi chuyển sang trạng thái chờ các yêu cầu xử lý như `read()`, `write()` hoặc `ioctl()`.

Đối với cơ chế cấu hình thiết bị, ứng dụng có thể sử dụng `ioctl()` để thay đổi các tham số truyền thông như baudrate, parity hoặc chế độ hoạt động UART. Driver sau đó cập nhật lại cấu hình và đồng bộ các tham số này xuống phần cứng UART thông qua Serdev Framework.

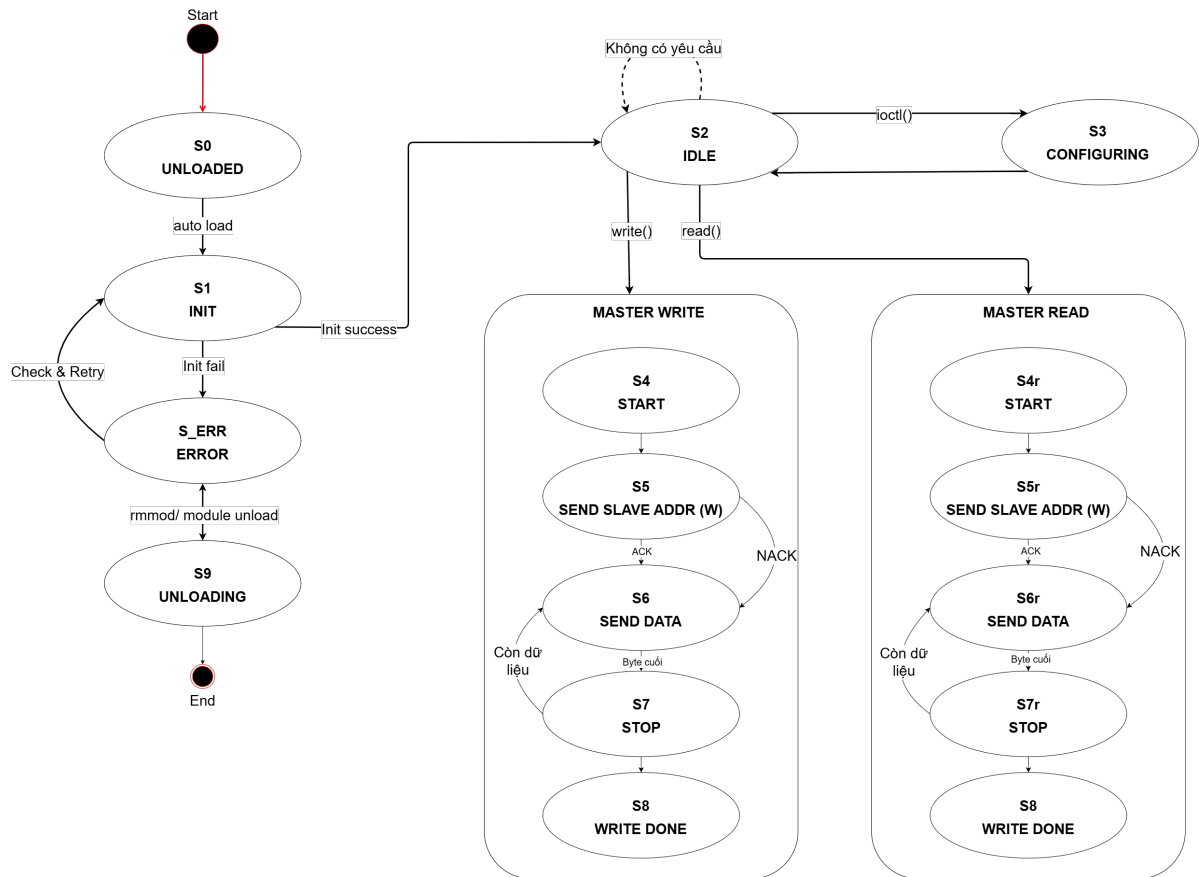
Trong quá trình truyền dữ liệu, ứng dụng sử dụng lời gọi `write()` để gửi dữ liệu xuống driver. Driver sẽ kiểm tra dữ liệu đầu vào, kích hoạt chân DE nhằm chuyển transceiver RS485 sang chế độ truyền, sau đó gửi dữ liệu qua UART. Khi quá trình truyền hoàn tất, driver chờ tín hiệu xác nhận TX hoàn thành bằng cơ chế polling hoặc interrupt. Nếu truyền thành công, driver vô hiệu hóa tín hiệu DE để quay về chế độ nhận và trả kết quả cho User Space. Ngược lại, nếu xảy ra lỗi truyền thông hoặc timeout, driver sẽ thực hiện cơ chế xử lý lỗi như ghi log và reset trạng thái hoạt động.

Đối với cơ chế nhận dữ liệu, ứng dụng sử dụng lời gọi `read()` để yêu cầu đọc dữ liệu từ thiết bị. Driver sẽ chờ dữ liệu đến từ bus RS485 thông qua UART RX line. Sau khi nhận dữ liệu, driver thực hiện kiểm tra CRC nhằm xác minh tính toàn vẹn của khung Modbus RTU. Nếu CRC hợp lệ, dữ liệu sẽ được lưu vào RX buffer và sao chép sang User Space thông qua cơ chế `copy_to_user()`. Trong trường hợp CRC không hợp lệ hoặc xảy ra lỗi nhận dữ liệu, driver sẽ chuyển sang nhánh xử lý lỗi nhằm đảm bảo tính ổn định của hệ thống.

Sau khi hoàn tất quá trình truyền nhận, ứng dụng gọi `close()` để giải phóng file handle. Driver khi đó sẽ giải phóng tài nguyên liên quan và đưa hệ thống trở về trạng thái chờ (IDLE) để tiếp tục phục vụ các yêu cầu truyền nhận tiếp theo.

3.4.2.3 Driver I2C

Mô hình FSM của driver I2C Để quản lý quá trình truyền nhận dữ liệu trên bus I2C, driver được thiết kế theo mô hình máy trạng thái hữu hạn (Finite State Machine – FSM). Mô hình này giúp driver kiểm soát rõ ràng các trạng thái hoạt động trong quá trình khởi tạo, cấu hình, truyền dữ liệu và xử lý lỗi khi giao tiếp với thiết bị slave thông qua giao thức I2C.



Hình 3.13: Máy trạng thái cho driver I2C

Quá trình hoạt động của driver bắt đầu tại trạng thái UNLOADED. Khi kernel module được nạp vào hệ thống thông qua cơ chế autoloader hoặc lệnh insmod, driver chuyển sang trạng thái INIT để khởi tạo GPIO, cấu hình SDA/SCL và đăng ký character device với Linux Kernel. Nếu quá trình khởi tạo thành công, driver chuyển sang trạng thái IDLE để chờ các yêu cầu từ User Space. Ngược lại, nếu xảy ra lỗi khởi tạo, driver sẽ chuyển sang trạng thái ERROR để thực hiện cơ chế retry hoặc chờ unload module.

Trong trạng thái IDLE, driver có thể tiếp nhận các yêu cầu cấu hình thông qua lời gọi hệ thống `ioctl()`. Khi đó, driver chuyển sang trạng thái CONFIGURING nhằm cập nhật các tham số như địa chỉ slave, tốc độ truyền hoặc chế độ hoạt động I2C trước khi quay trở lại trạng thái chờ.

Khi ứng dụng thực hiện thao tác ghi dữ liệu thông qua `write()`, driver chuyển sang nhánh MASTER WRITE. Tại đây, driver lần lượt thực hiện các trạng thái bao gồm tạo tín hiệu START, gửi địa chỉ slave ở chế độ ghi, kiểm tra tín hiệu ACK và truyền dữ liệu lên bus I2C. Quá trình truyền tiếp tục cho đến khi byte cuối cùng được gửi thành công, sau đó driver tạo tín hiệu STOP và chuyển sang trạng thái WRITE DONE. Trong trường hợp không nhận được ACK từ slave device, driver sẽ thực hiện cơ chế xử lý lỗi hoặc retry truyền dữ liệu.

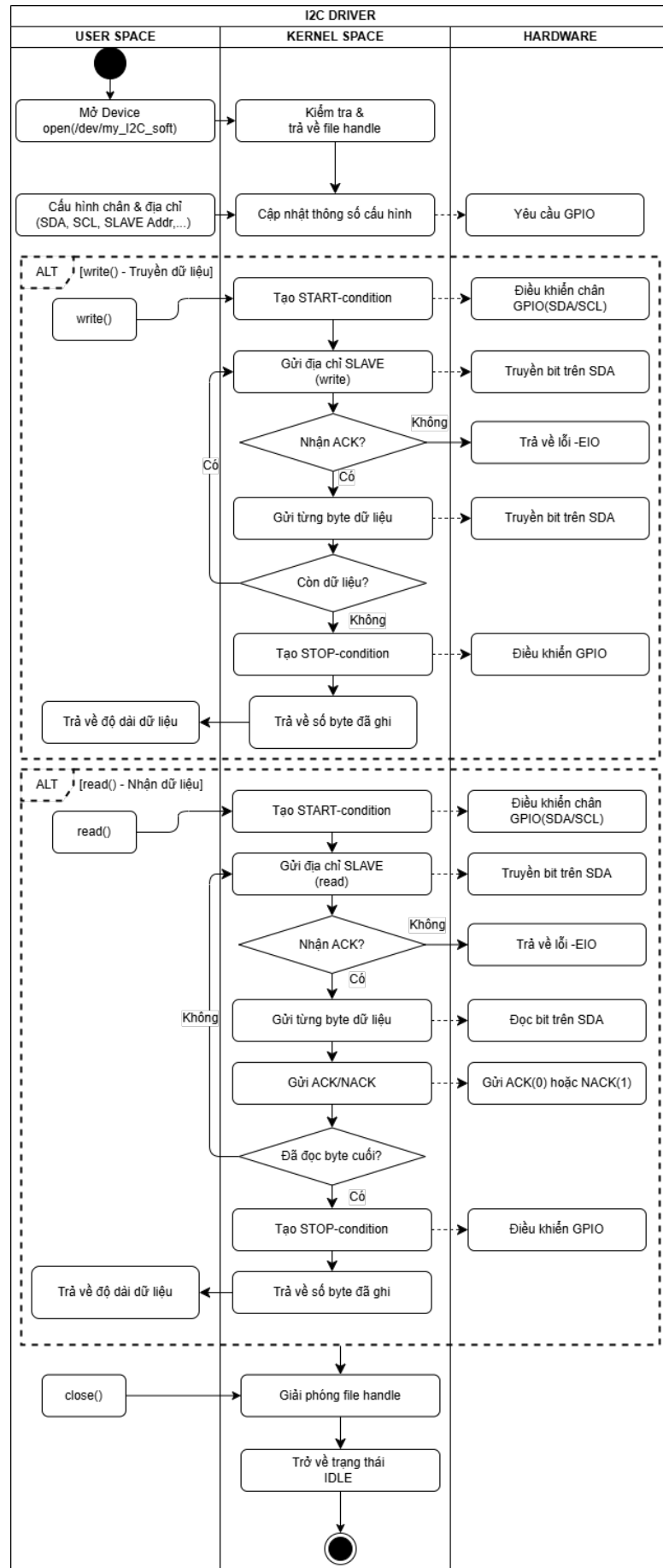
Tương tự, khi ứng dụng thực hiện thao tác đọc dữ liệu bằng `read()`, driver chuyển sang nhánh MASTER READ. Driver sẽ tạo tín hiệu START, gửi địa chỉ slave và nhận dữ liệu từ thiết

bị ngoại vi thông qua bus I2C. Sau khi toàn bộ dữ liệu được nhận hoàn tất, driver tạo tín hiệu STOP và cập nhật dữ liệu vào buffer nội bộ trước khi trả dữ liệu về User Space.

Cuối cùng, khi kernel module bị gỡ khỏi hệ thống thông qua lệnh `rmmmod`, driver chuyển sang trạng thái UNLOADING để giải phóng GPIO, tài nguyên kernel và character device trước khi kết thúc vòng đời hoạt động của module trong Linux Kernel. **Activity Diagram của driver I2C**

Driver I2C được hiện thực theo mô hình Character Device trong Linux Kernel, cho phép ứng dụng ở User Space giao tiếp với thiết bị ngoại vi thông qua device file `/dev/my_I2C_soft`. Quá trình hoạt động của driver bao gồm các thao tác cấu hình bus, truyền dữ liệu và nhận dữ liệu thông qua cơ chế điều khiển trực tiếp các chân GPIO SDA/SCL theo chuẩn I2C software bit-banging.

Hoạt động của driver được mô tả thông qua Activity Diagram trong Hình 3.14.



Hình 3.14: Activity Diagram của driver I2C

Trong giai đoạn khởi tạo, ứng dụng User Space thực hiện lời gọi `open()` để mở device file của driver. Linux Kernel sau đó kiểm tra file handle và khởi tạo các tài nguyên cần thiết phục vụ giao tiếp I2C. Các thông số như địa chỉ slave, chân SDA/SCL và tốc độ truyền có thể được cấu hình thông qua cơ chế `ioctl()`.

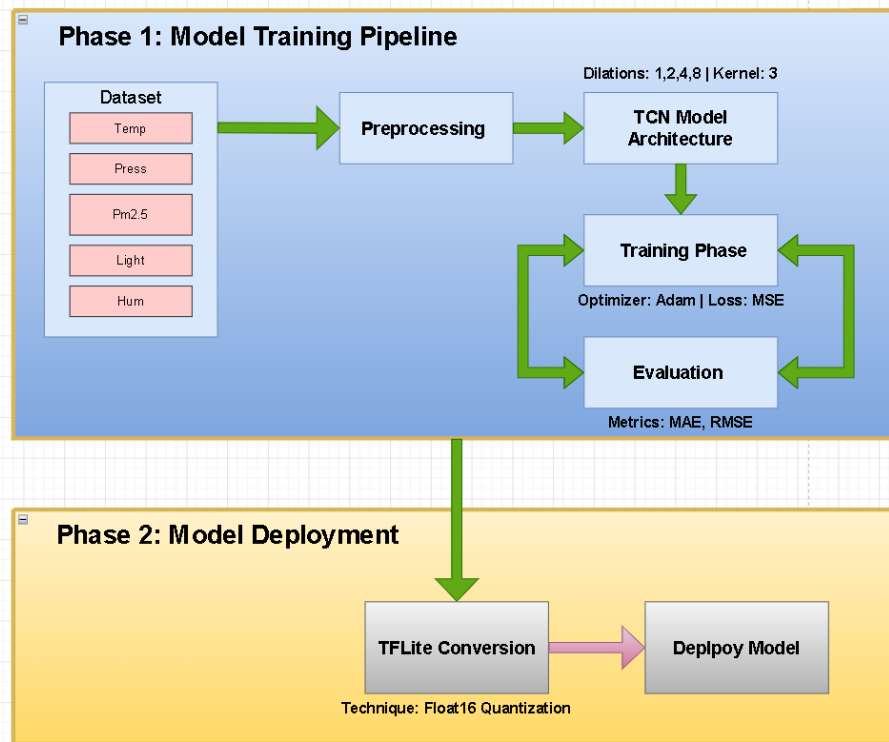
Đối với quá trình truyền dữ liệu (`write()`), driver tạo điều kiện START-condition và gửi địa chỉ slave theo chế độ ghi. Sau khi nhận tín hiệu ACK từ thiết bị ngoại vi, dữ liệu sẽ được truyền tuần tự từng byte thông qua chân SDA kết hợp tín hiệu clock trên chân SCL. Nếu xảy ra lỗi ACK hoặc mất kết nối thiết bị, driver sẽ trả về lỗi `-EIO` để thông báo cho User Space. Khi hoàn tất quá trình truyền, driver tạo STOP-condition và trả về số byte đã ghi thành công.

Trong quá trình nhận dữ liệu (`read()`), driver tiếp tục tạo START-condition và gửi địa chỉ slave theo chế độ đọc. Dữ liệu từ thiết bị ngoại vi được đọc tuần tự từng byte thông qua chân SDA, đồng thời driver gửi ACK hoặc NACK tương ứng để điều khiển quá trình nhận dữ liệu. Sau khi nhận byte cuối cùng, STOP-condition được tạo ra nhằm giải phóng bus I2C trước khi dữ liệu được trả về cho User Space.

Activity Diagram cho thấy driver được tổ chức theo hướng tách biệt giữa User Space, Kernel Space và Hardware Layer. Cách tiếp cận này giúp quá trình xử lý dữ liệu trở nên rõ ràng, đồng thời hỗ trợ việc mở rộng và bảo trì driver trong các hệ thống Linux Embedded sử dụng Yocto Project.

3.4.3 Thiết kế khối xử lý AI

Mục tiêu của khối xử lý AI là xây dựng một mô hình học sâu có khả năng khai thác các đặc trưng tiềm ẩn trong dữ liệu chuỗi thời gian từ hệ thống cảm biến môi trường, từ đó đưa ra dự báo nồng độ bụi mịn PM2.5 trong tương lai với độ chính xác cao. Khối này đóng vai trò là thành phần cốt lõi của hệ thống Edge AI, cho phép thiết bị Raspberry Pi 4 thực hiện suy luận trực tiếp tại hiện trường mà không phụ thuộc vào máy chủ đám mây.



Hình 3.15: Sơ đồ khối Module AI

3.4.3.1 Thiết kế dữ liệu đầu vào và đầu ra

Dữ liệu môi trường thu thập được có dạng chuỗi thời gian đa biến (*Multivariate Time Series*), trong đó mỗi mẫu dữ liệu bao gồm đồng thời nhiều thông số vật lý đo được tại cùng một thời điểm. Để chuyển đổi dữ liệu này thành dạng phù hợp cho mô hình học sâu, hệ thống sử dụng kỹ thuật cửa sổ trượt (*Sliding Window*) nhằm xây dựng các cặp dữ liệu đầu vào – đầu ra theo phương pháp học có giám sát.

- **Vector đầu vào (X):** Được biểu diễn dưới dạng ma trận có kích thước $(T \times F)$, trong đó:
 - $T = 30$: số bước thời gian quan sát trong quá khứ.
 - $F = 5$: số đặc trưng đầu vào, bao gồm Temperature, Humidity, Pressure, PM2.5 và Light .

Điều này có nghĩa là tại mỗi lần dự báo, mô hình sẽ sử dụng 30 mẫu dữ liệu gần nhất để phân tích xu hướng biến động của môi trường.

- **Giá trị đầu ra (y):** Là giá trị nồng độ PM2.5 tại thời điểm $t + 15$ phút. Việc lựa chọn khoảng thời gian dự báo 15 phút giúp hệ thống có đủ thời gian để phát hiện xu hướng ô nhiễm và kích hoạt các cơ chế điều khiển phù hợp.

3.4.3.2 Quy trình tiền xử lý và chuẩn hóa dữ liệu

Dữ liệu thu thập từ các cảm biến thường có đơn vị đo và miền giá trị khác nhau. Ví dụ, nhiệt độ được đo theo độ C, áp suất theo hPa, trong khi cường độ ánh sáng có thể dao động trên dải giá trị rất lớn. Nếu đưa trực tiếp các dữ liệu này vào mô hình, các đặc trưng có biên độ lớn sẽ chi phối quá trình tối ưu hóa và làm giảm hiệu quả học của mạng nơ-ron.

Để khắc phục vấn đề này, toàn bộ dữ liệu được chuẩn hóa về khoảng $[0, 1]$ bằng phương pháp Min-Max Scaling.

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

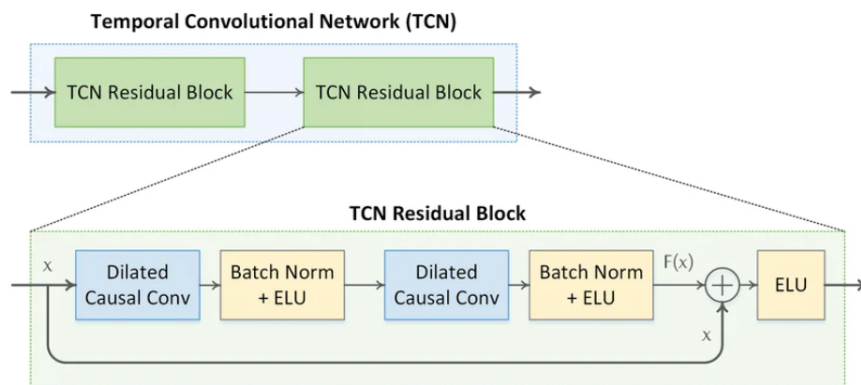
Trong đó:

- x : Giá trị thực tế của cảm biến.
- $x_{\min}$: Giá trị nhỏ nhất của đặc trưng trong tập huấn luyện.
- $x_{\max}$: Giá trị lớn nhất của đặc trưng trong tập huấn luyện.

Phương pháp chuẩn hóa này giúp các đặc trưng có cùng thang đo, từ đó cải thiện tốc độ hội tụ, tăng tính ổn định và nâng cao độ chính xác của mô hình.

3.4.3.3 Kiến trúc mạng nơ-ron TCN (Temporal Convolutional Network)

Mô hình được xây dựng dựa trên kiến trúc TCN, một cấu trúc mạng tích chập được thiết kế chuyên biệt cho bài toán chuỗi thời gian. So với các kiến trúc hồi tiếp như RNN hay LSTM, TCN có ưu điểm về khả năng xử lý song song, tốc độ huấn luyện nhanh và khả năng học phụ thuộc dài hạn hiệu quả.



Hình 3.16: mô hình TCN

Mô hình bao gồm ba thành phần cốt lõi sau.

3.4.3.3.1 a. Tích chập nhân quả (Causal Convolution)

Tích chập nhân quả đảm bảo rằng tại thời điểm dự báo t , mô hình chỉ được phép sử dụng thông tin từ quá khứ và hiện tại, tức là các phần tử $x_1, x_2, \dots, x_t$. Cơ chế này giúp ngăn chặn hiện tượng rò rỉ thông tin từ tương lai vào quá trình học.

3.4.3.3.2 b. Tích chập giãn cách (Dilated Convolution)

Để mở rộng trường thụ cảm mà không làm tăng đáng kể số lượng tham số, mô hình sử dụng các lớp tích chập với hệ số giãn cách d .

$$(F *_d x)(s) = \sum_{i=0}^{k-1} F(i) \cdot x(s - d \cdot i)$$

Trong đó:

- k : Kích thước kernel tích chập, được lựa chọn bằng 3.
- d : Hệ số giãn cách.

Trong đề tài này, các hệ số giãn cách được thiết kế theo cấp số nhân:

$$d = \{1, 2, 4, 8\}$$

Cấu hình này giúp mô hình mở rộng trường thụ cảm theo cấp số nhân, cho phép học hiệu quả các xu hướng dài hạn của chuỗi dữ liệu.

3.4.3.3.3 c. Kết nối tắt (Residual Connections)

Kết nối tắt giúp cải thiện quá trình lan truyền đạo hàm và hạn chế hiện tượng mất mát đạo hàm khi mạng trở nên sâu hơn.

$$o = \text{Activation}(x + \mathcal{F}(x))$$

Trong đó $\mathcal{F}(x)$ là hàm biến đổi phi tuyến của khối tích chập, còn x là tín hiệu đầu vào được cộng trực tiếp vào đầu ra của khối.

3.4.3.4 Thiết kế chiến lược huấn luyện

Quá trình huấn luyện mô hình được thực hiện với mục tiêu tối thiểu hóa sai số giữa giá trị dự báo và giá trị thực tế.

- **Hàm mất mát (Loss Function):** Sai số bình phương trung bình (Mean Squared Error – MSE) được lựa chọn do phù hợp với bài toán hồi quy.

$$MSE = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

Trong đó y_i là giá trị thực tế và $\hat{y}_i$ là giá trị dự báo của mô hình.

- **Thuật toán tối ưu:** Adam Optimizer được sử dụng với tốc độ học (*learning rate*) $\eta = 0.001$. Thuật toán này kết hợp ưu điểm của Momentum và RMSProp, giúp mô hình hội tụ nhanh và ổn định hơn so với phương pháp gradient descent truyền thống.

Ngoài ra, trong quá trình huấn luyện, tập dữ liệu được chia thành tập huấn luyện và tập kiểm thử để đánh giá khả năng tổng quát hóa của mô hình.

3.4.3.5 Thiết kế quy trình tối ưu hóa Edge AI (TFLite Quantization)

Sau khi huấn luyện hoàn tất, mô hình được chuyển đổi sang định dạng TensorFlow Lite để triển khai trên Raspberry Pi 4.

Kỹ thuật *Float16 Quantization* được áp dụng để nén các trọng số từ định dạng Float32 xuống Float16. Quá trình này giúp giảm đáng kể dung lượng mô hình trong khi vẫn duy trì độ chính xác gần như tương đương với mô hình gốc.

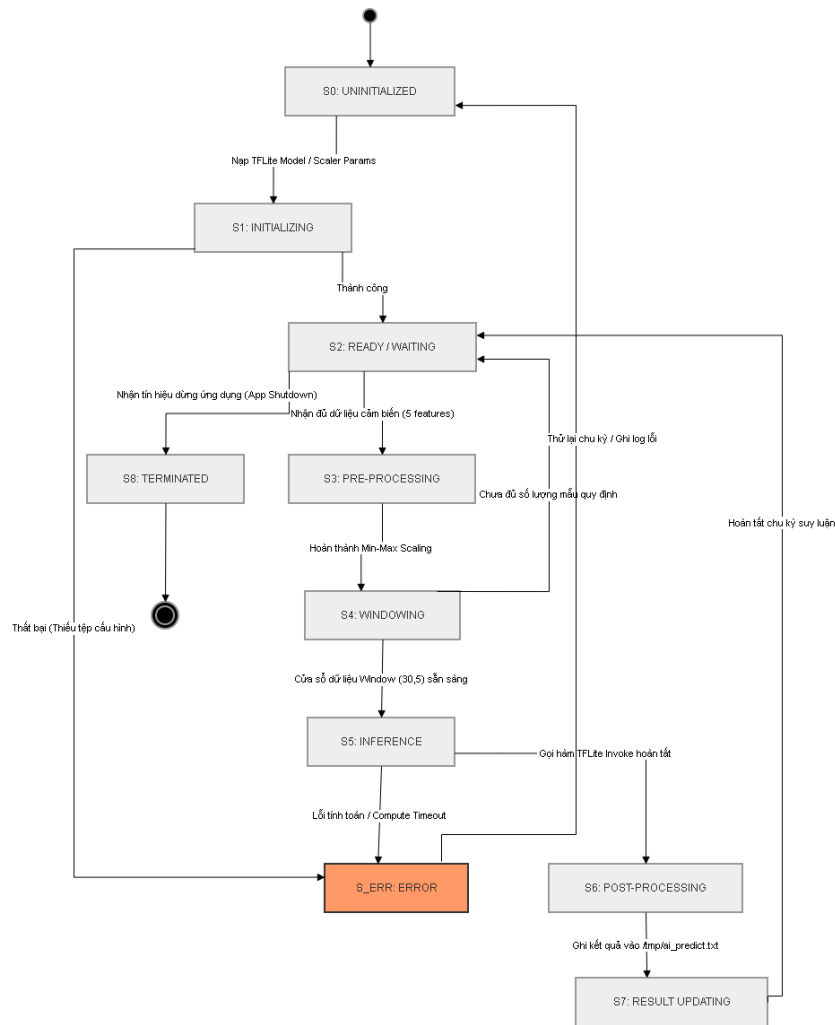
Các lợi ích chính của quá trình tối ưu hóa bao gồm:

- Giảm khoảng 50% kích thước tệp mô hình.
- Tăng tốc độ suy luận trên thiết bị biên.
- Giảm nhu cầu sử dụng bộ nhớ RAM.
- Duy trì độ chính xác ở mức cao.

Nhờ quá trình tối ưu hóa này, mô hình TCN có thể hoạt động hiệu quả trên Raspberry Pi 4, đáp ứng yêu cầu xử lý thời gian thực trong hệ thống giám sát và dự báo chất lượng không khí.

3.4.3.6 Mô hình FSM khối AI

Để quản lý toàn bộ vòng đời suy luận, tối ưu hóa tài nguyên RAM và đảm bảo tính hoạt động ổn định của mô hình mạng TCN (Temporal Convolutional Network) trong không gian người dùng (User Space), khối xử lý AI được cấu trúc vận hành dựa trên một máy trạng thái hữu hạn (FSM) bao gồm các trạng thái chính sau:



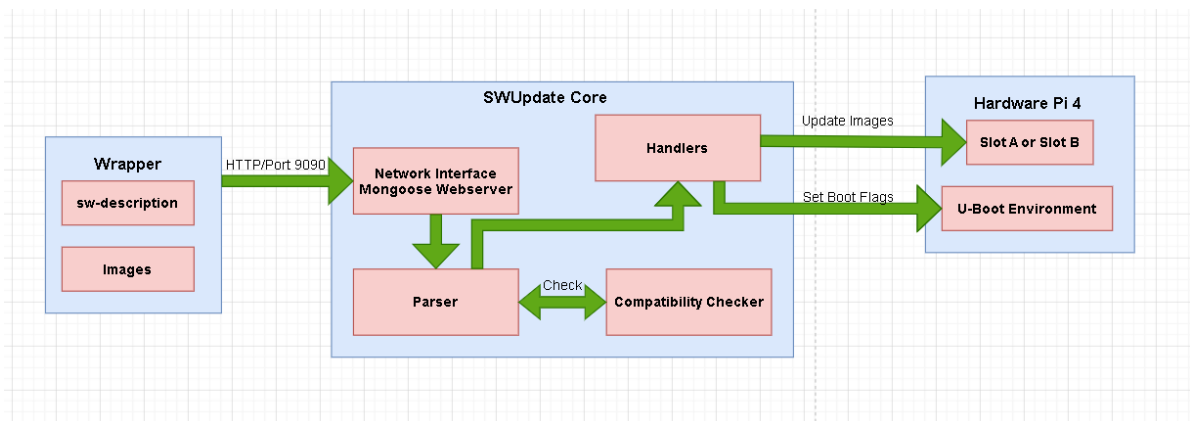
Hình 3.17: Mô hình FSM khối AI

- S0 (UNINITIALIZED): Trạng thái khởi đầu khi tiến trình AI Engine được kích hoạt, các vùng đệm dữ liệu và mô hình chưa được nạp vào bộ nhớ.
- S1 (INITIALIZING): Nạp mô hình `tcn_model_production.tflite`, khởi tạo TFLite Interpreter và đọc các tham số chuẩn hóa từ `scaler_params.json`.
- S2 (READY / WAITING): Trạng thái chờ dữ liệu cảm biến mới từ Gateway Service.
- S3 (PRE-PROCESSING): Thực hiện lọc trung bình trượt và chuẩn hóa 5 đặc trưng đầu vào về miền giá trị $[0, 1]$.
- S4 (WINDOWING): Đưa dữ liệu vào hàng đợi và kiểm tra đủ 30 mẫu liên tiếp để tạo tensor đầu vào kích thước 30×5 .
- S5 (INFERENCE): Gọi TFLite Interpreter để thực hiện suy luận với mô hình TCN.
- S6 (POST-PROCESSING): Thực hiện inverse scaling để chuyển kết quả dự báo về đơn vị PM2.5 gốc tại thời điểm $t + 15$ phút.

- S7 (RESULT UPDATING): Lưu kết quả vào `/tmp/ai_predict.txt`, giải phóng bộ đệm và quay lại trạng thái S2.
- S8 (TERMINATED): Giải phóng bộ nhớ và hủy TFLite Interpreter khi nhận tín hiệu dừng hệ thống.
- S_ERR (ERROR): Xử lý các lỗi như lỗi nạp mô hình, dữ liệu không hợp lệ hoặc timeout, ghi log và khởi động lại tiến trình từ trạng thái S0.

3.4.4 Thiết kế khối xử lý OTA

Khối OTA được thiết kế như một thực thể quản lý vòng đời cập nhật của hệ thống, từ giai đoạn tiếp nhận gói dữ liệu đến giai đoạn trao đổi phân vùng khởi động. Toàn bộ logic được đóng gói trong một quy trình khép kín nhằm đảm bảo tính nguyên tử (*Atomicity*) và khả năng tự phục hồi (*Recovery*).



Hình 3.18: Sơ đồ khối OTA

3.4.4.1 Mô tả chi tiết các thành phần hệ thống

- **Wrapper (Gói cập nhật .swu):** Là một trình bao gói chứa đựng tệp mô tả `sw-description` và các tệp ảnh hệ thống (`Images`). Thiết kế này đảm bảo tính nhất quán giữa chỉ thị cài đặt và dữ liệu thực thi.
- **Network Interface (Mongoose Webserver):** Được cấu hình hoạt động như một HTTP Server tại cổng 9090. Thành phần này chịu trách nhiệm tiếp nhận luồng dữ liệu (*Streaming*) từ người dùng và đẩy trực tiếp vào bộ phận phân tích mà không cần lưu trữ tạm thời, giúp tối ưu hóa tài nguyên RAM cho Raspberry Pi 4.
- **Parser (Bộ phân tích cú pháp Libconfig):** Đóng vai trò là trung tâm điều khiển của hệ thống. Khối này giải mã các chỉ thị từ `sw-description`, xác định logic cài đặt, vị trí phân vùng đích và các ràng buộc về phần cứng.

- **Compatibility Checker:** Thực hiện cơ chế kiểm tra chéo giữa *Hardware ID* trong gói cập nhật và thông tin phần cứng thực tế của thiết bị. Đây là chốt chặn an toàn nhằm ngăn ngừa các bản cập nhật không tương thích gây lỗi hệ thống (*brick*).
- **Handlers (Trình thực thi):** Gồm các module chuyên biệt (Raw/Archive Handler) có nhiệm vụ giải nén và ghi trực tiếp dữ liệu vào các phân vùng vật lý. Sau khi ghi hoàn tất, Handlers sẽ thực hiện kiểm tra mã băm (*Checksum*) để xác minh tính toàn vẹn của dữ liệu trên đĩa.

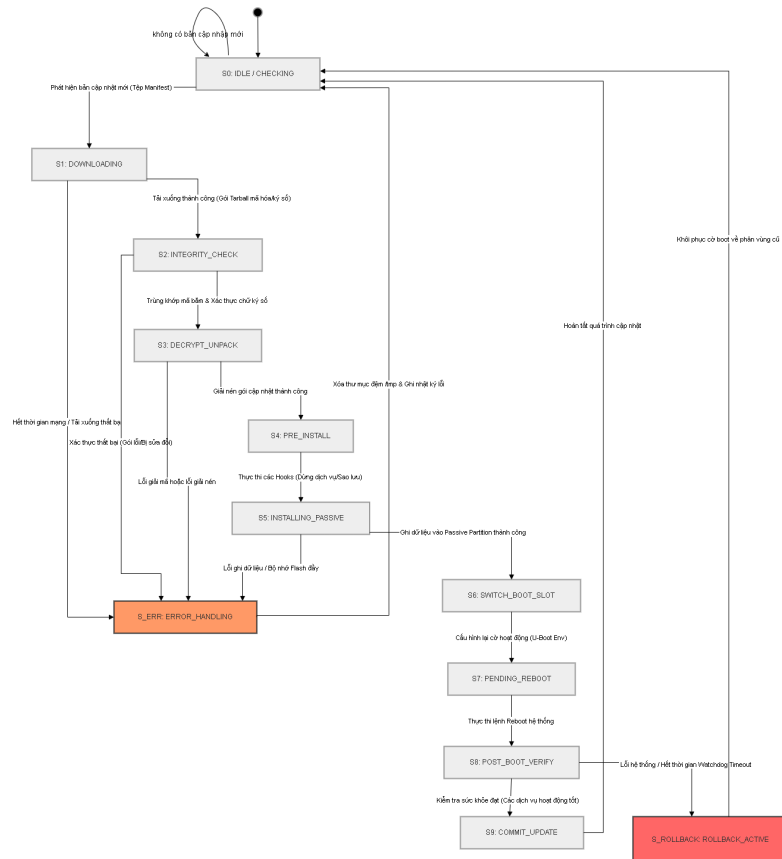
3.4.4.2 Luồng dữ liệu và logic xử lý (Data Flow Description)

Quy trình xử lý một bản cập nhật được thiết kế theo luồng liên kết chặt chẽ giữa các khối chức năng, mô tả như sau:

1. **Giai đoạn tiếp nhận và phân tích:** Luồng dữ liệu từ tệp `.swu` đi qua Network Interface và được chuyển tiếp ngay lập tức đến Parser. Parser thực hiện bóc tách các siêu dữ liệu (*Metadata*) để chuẩn bị cho các bước thẩm định tiếp theo.
2. **Giai đoạn thẩm định và ánh xạ:** Parser gửi tín hiệu đến Compatibility Checker để xác nhận ID phần cứng. Nếu khớp, hệ thống truy vấn U-Boot Environment để xác định trạng thái của Slot A và Slot B. Dựa trên cơ chế *Dual-bank*, hệ thống sẽ chọn phân vùng *Passive* (đang không hoạt động) làm mục tiêu cập nhật.
3. **Giai đoạn thực thi song song (Dual-action):** Sau khi Parser ra chỉ thị, khối Handlers thực hiện hai luồng hành động chính:
 - **Update Images:** Trích xuất và ghi trực tiếp tệp `rootfs.ext4.gz` vào phân vùng đã chọn (Slot A hoặc Slot B).
 - **Integrity Check:** Thực hiện xác minh Checksum sau khi ghi xong 100% dữ liệu để đảm bảo không có lỗi bit xảy ra trong quá trình I/O.
4. **Giai đoạn xác lập (Finalization):** Chỉ khi giai đoạn thực thi thành công hoàn toàn, Handlers mới thực hiện luồng tác động cuối cùng vào U-Boot Environment thông qua lệnh Set Boot Flags (ví dụ thiết lập biến `ustate=1`). Thao tác này đánh dấu bản cập nhật đã sẵn sàng, hoàn tất tính nguyên tử của quy trình.
5. **Tái khởi động:** Hệ thống phát lệnh khởi động lại để Bootloader thực hiện trao đổi Slot khởi động dựa trên các cờ hiệu đã thiết lập.

3.4.4.3 Mô hình FSM khối OTA

Nhằm đảm bảo tính an toàn, khả năng chịu lỗi cao (Fault Tolerance) và duy trì tính sẵn sàng liên tục của hệ thống theo cơ chế cập nhật Dual Partition (A/B Switching), tiến trình OTA trong không gian người dùng (User Space) được tổ chức dưới dạng máy trạng thái hữu hạn (Finite State Machine – FSM). Mô hình này kiểm soát toàn bộ vòng đời cập nhật firmware theo cơ chế nguyên tử (Atomicity), đồng thời hỗ trợ khả năng chống hỏng hệ thống (anti-bricking).



Hình 3.19: Mô hình FSM khối OTA

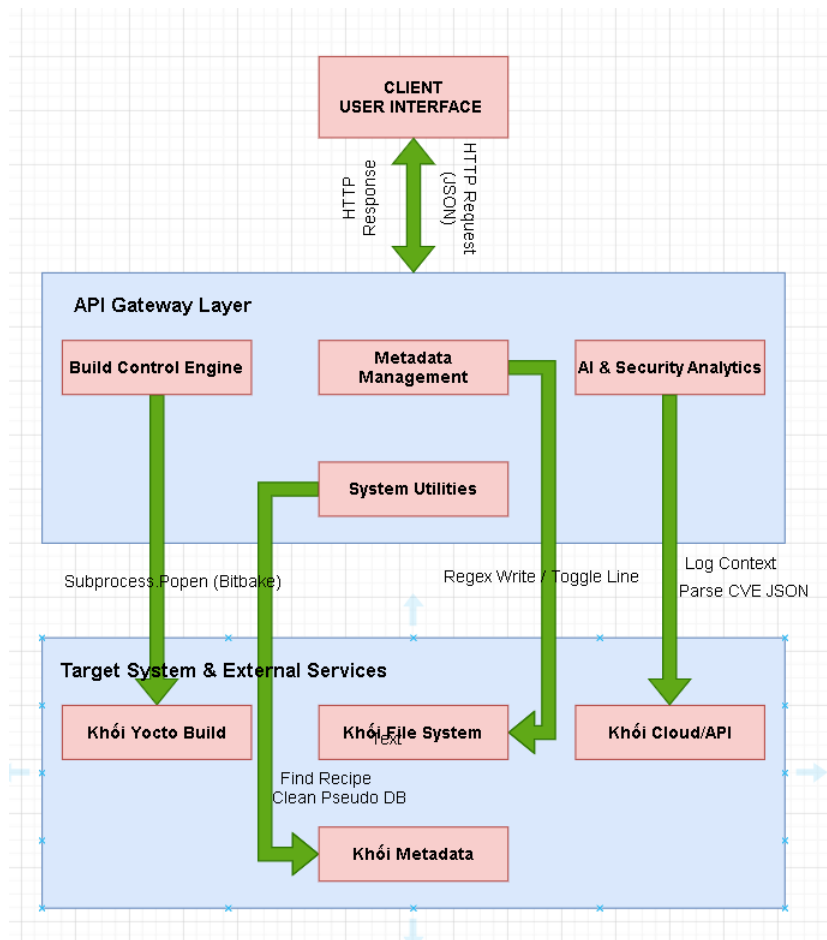
- S0 (IDLE / CHECKING): Trạng thái chờ và kiểm tra định kỳ. Gateway gửi yêu cầu HTTPS đến Deployment Server để tải tập Manifest và so sánh phiên bản hiện tại với phiên bản mới nhất.
 - Điều kiện chuyển dịch 1: Nếu không có bản cập nhật mới, hệ thống tiếp tục duy trì ở trạng thái S0.
 - Điều kiện chuyển dịch 2: Nếu phát hiện phiên bản mới, hệ thống chuyển sang trạng thái S1.
- S1 (DOWNLOADING): Trạng thái tải gói cập nhật (.swu hoặc .tar.gz) về thư mục tạm /tmp.

- Điều kiện chuyển dịch 1: Nếu tải xuống thành công, hệ thống chuyển sang trạng thái S2.
- Điều kiện chuyển dịch 2: Nếu xảy ra lỗi mạng hoặc timeout, hệ thống chuyển sang trạng thái S_ERR.
- S2 (INTEGRITY_CHECK): Trạng thái kiểm tra mã băm SHA-256 và xác thực chữ ký số của gói cập nhật.
 - Điều kiện chuyển dịch 1: Nếu mã băm trùng khớp và chữ ký hợp lệ, hệ thống chuyển sang trạng thái S3.
 - Điều kiện chuyển dịch 2: Nếu xác thực thất bại, hệ thống chuyển sang trạng thái S_ERR.
- S3 (DECRYPT_UNPACK): Trạng thái giải mã (nếu có) và giải nén các thành phần Kernel, DTB và RootFS.
 - Điều kiện chuyển dịch 1: Nếu giải nén thành công, hệ thống chuyển sang trạng thái S4.
 - Điều kiện chuyển dịch 2: Nếu xảy ra lỗi giải mã hoặc giải nén, hệ thống chuyển sang trạng thái S_ERR.
- S4 (PRE_INSTALL): Trạng thái thực thi các hook tiền cài đặt như dừng dịch vụ cảm biến, AI Engine và đồng bộ dữ liệu.
 - Điều kiện chuyển dịch: Sau khi hoàn tất các tác vụ chuẩn bị, hệ thống chuyển sang trạng thái S5.
- S5 (INSTALLING_PASSIVE): Trạng thái ghi firmware mới vào phân vùng không hoạt động (Passive Partition).
 - Điều kiện chuyển dịch 1: Nếu ghi dữ liệu thành công, hệ thống chuyển sang trạng thái S6.
 - Điều kiện chuyển dịch 2: Nếu xảy ra lỗi ghi dữ liệu hoặc thiếu dung lượng lưu trữ, hệ thống chuyển sang trạng thái S_ERR.
- S6 (SWITCH_BOOT_SLOT): Trạng thái cập nhật biến môi trường U-Boot để chọn phân vùng mới cho lần khởi động tiếp theo.
 - Điều kiện chuyển dịch: Sau khi cập nhật thành công, hệ thống chuyển sang trạng thái S7.

- S7 (PENDING_REBOOT): Trạng thái thực hiện sync và phát lệnh khởi động lại hệ thống.
 - Điều kiện chuyển dịch: Sau khi reboot, hệ thống chuyển sang trạng thái S8.
- S8 (POST_BOOT_VERIFY): Trạng thái kiểm tra sức khỏe hệ thống sau khởi động, bao gồm các dịch vụ MQTT, cảm biến và AI Engine.
 - Điều kiện chuyển dịch 1: Nếu hệ thống hoạt động ổn định, chuyển sang trạng thái S9.
 - Điều kiện chuyển dịch 2: Nếu phát hiện lỗi hoặc timeout, chuyển sang trạng thái S_ROLLBACK.
- S9 (COMMIT_UPDATE): Trạng thái xác nhận cập nhật bằng cách ghi cờ boot_successful = 1 vào U-Boot Env.
 - Điều kiện chuyển dịch: Sau khi hoàn tất, hệ thống quay trở lại trạng thái S0.
- S_ROLLBACK (ROLLBACK_ACTIVE): Trạng thái khôi phục cờ boot về phân vùng cũ ổn định và tự động khởi động lại.
 - Điều kiện chuyển dịch: Sau khi rollback thành công, hệ thống ghi log lỗi và quay về trạng thái S0.
- S_ERR (ERROR_HANDLING): Trạng thái xử lý ngoại lệ, xóa dữ liệu tạm trong /tmp, giải phóng tài nguyên và ghi nhật ký lỗi.
 - Điều kiện chuyển dịch: Sau khi hoàn tất dọn dẹp, hệ thống quay trở lại trạng thái S0.

3.4.5 Thiết kế khôi xử lý Tool

Hệ thống Secure-Yocto Builder được thiết kế dựa trên mô hình kiến trúc phân lớp (*Layered Architecture*), chia thành ba tầng chức năng riêng biệt nhằm đảm bảo tính module hóa, dễ dàng mở rộng và tối ưu hóa luồng xử lý dữ liệu.



Hình 3.20: Sơ đồ khối Tool

3.4.5.1 Lớp Giao diện Người dùng (Client User Interface)

Lớp này đóng vai trò là bề mặt tương tác (*Frontend*) của hệ thống, được xây dựng trên nền tảng Web với phong cách Terminal UI (TUI).

- **Giao diện tương tác:** Cho phép người dùng tùy chỉnh các thông số cấu hình như thiết bị đích (Target), các tính năng bảo mật (SELinux, dm-verity) và nạp thư viện bổ sung.
- **Giao thức truyền thông:** Sử dụng phương thức gửi yêu cầu HTTP Request (định dạng JSON) tới máy chủ và nhận về các phản hồi HTTP Response để cập nhật trạng thái hệ thống theo thời gian thực.

3.4.5.2 Lớp Cửa ngõ API (API Gateway Layer)

Đây là lớp xử lý trung tâm (*Backend*) sử dụng Flask Framework, đóng vai trò điều phối và thực thi các logic nghiệp vụ của ứng dụng. Lớp này bao gồm 4 module cốt lõi:

- **Build Control Engine:** Điều khiển tiến trình biên dịch thông qua kỹ thuật gọi tiến trình con (*subprocess*), cho phép khởi chạy lệnh bitbake mà không làm gián đoạn luồng xử

lý chính của ứng dụng.

- **Metadata Management:** Chuyên trách việc thao tác dữ liệu cấu hình thông qua biểu thức chính quy (Regex), thực hiện cơ chế bật/tắt (*Toggle*) các dòng mã trong file cấu hình Yocto.
- **AI & Security Analytics:** Tích hợp trí tuệ nhân tạo thông qua Gemini API để phân tích ngữ cảnh lỗi (Log Context) và thực hiện phân tích cú pháp dữ liệu lỗ hổng CVE từ các tệp JSON bảo mật.
- **System Utilities:** Cung cấp các công cụ phụ trợ hệ thống như kiểm tra sự tồn tại của Recipe và dọn dẹp cơ sở dữ liệu cache (Pseudo DB).

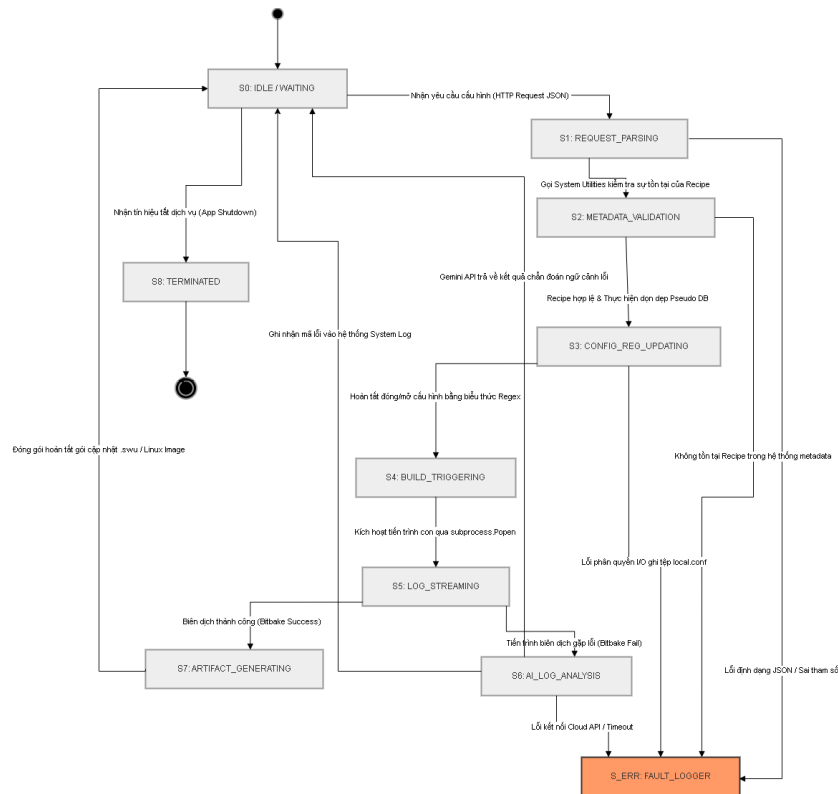
3.4.5.3 Lớp Hệ thống Đích và Dịch vụ Ngoại vi (Target System & External Services)

Đây là tầng thực thi cuối cùng, nơi chứa các tài nguyên hệ thống và các dịch vụ bên thứ ba:

- **Khối Yocto Build:** Môi trường thực thi lệnh biên dịch BitBake và lưu trữ các tệp nhật ký tiến trình (Terminal Log).
- **Khối File System:** Lưu trữ các tệp cấu hình cốt lõi của Yocto Project như `local.conf` và `bblayers.conf`.
- **Khối Metadata:** Hệ thống quản lý các layer và các recipe phục vụ cho quá trình đóng gói hệ điều hành.
- **Khối Cloud/API:** Điểm kết nối với nền tảng điện toán đám mây để sử dụng dịch vụ AI phân tích thông minh và truy xuất dữ liệu bảo mật từ các cơ sở dữ liệu quốc gia.

3.4.5.4 Mô hình FSM khối Tool

Nhằm tự động hóa quá trình cấu hình hệ thống, quản lý tập trung các lớp metadata và tích hợp trí tuệ nhân tạo hỗ trợ phân tích lỗi biên dịch, khối Tool (Secure-Yocto Builder) được thiết kế dưới dạng máy trạng thái hữu hạn (Finite State Machine – FSM). Mô hình này điều phối luồng dữ liệu giữa giao diện người dùng và công cụ BitBake thông qua các trạng thái xử lý sau.



Hình 3.21: Mô hình FSM khối Tool

- S0 (IDLE / WAITING): Trạng thái chờ của hệ thống. Backend Flask API lắng nghe và tiếp nhận các yêu cầu cấu hình từ giao diện người dùng.
 - Điều kiện chuyển dịch 1: Khi nhận được HTTP Request chứa dữ liệu JSON cấu hình, hệ thống chuyển sang trạng thái S1.
 - Điều kiện chuyển dịch 2: Khi nhận tín hiệu tắt ứng dụng, hệ thống chuyển sang trạng thái S8.
- S1 (REQUEST_PARSING): Trạng thái phân tích cú pháp dữ liệu JSON, bao gồm thiết bị mục tiêu, các tùy chọn bảo mật và danh sách Recipe cần bổ sung.
 - Điều kiện chuyển dịch 1: Nếu dữ liệu hợp lệ, hệ thống chuyển sang trạng thái S2.
 - Điều kiện chuyển dịch 2: Nếu JSON sai định dạng hoặc thiếu tham số, hệ thống chuyển sang trạng thái S_ERR.
- S2 (METADATA_VALIDATION): Trạng thái kiểm tra sự tồn tại của các Recipe và thực hiện dọn dẹp Pseudo DB.
 - Điều kiện chuyển dịch 1: Nếu các Recipe hợp lệ, hệ thống chuyển sang trạng thái S3.

- Điều kiện chuyển dịch 2: Nếu Recipe không tồn tại, hệ thống chuyển sang trạng thái S_ERR.
- S3 (CONFIG_REG_UPDATING): Trạng thái cập nhật các tệp `local.conf` và `bblayers.conf` bằng biểu thức chính quy.
 - Điều kiện chuyển dịch 1: Nếu cập nhật thành công, hệ thống chuyển sang trạng thái S4.
 - Điều kiện chuyển dịch 2: Nếu xảy ra lỗi ghi tệp hoặc lỗi phân quyền, hệ thống chuyển sang trạng thái S_ERR.
- S4 (BUILD_TRIGGERING): Trạng thái thiết lập môi trường và khởi chạy tiến trình biên dịch BitBake bằng `subprocess.Popen`.
 - Điều kiện chuyển dịch: Sau khi tiến trình biên dịch được tạo thành công, hệ thống chuyển sang trạng thái S5.
- S5 (LOG_STREAMING): Trạng thái đọc và truyền log thời gian thực từ BitBake về giao diện người dùng.
 - Điều kiện chuyển dịch 1: Nếu biên dịch thành công, hệ thống chuyển sang trạng thái S7.
 - Điều kiện chuyển dịch 2: Nếu biên dịch thất bại, hệ thống chuyển sang trạng thái S6.
- S6 (AI_LOG_ANALYSIS): Trạng thái gửi log lỗi đến Gemini API để phân tích nguyên nhân và đề xuất hướng khắc phục.
 - Điều kiện chuyển dịch 1: Nếu nhận được kết quả phân tích, hệ thống hiển thị kết quả và quay về trạng thái S0.
 - Điều kiện chuyển dịch 2: Nếu lỗi kết nối API hoặc timeout, hệ thống chuyển sang trạng thái S_ERR.
- S7 (ARTIFACT_GENERATING): Trạng thái thu thập và đóng gói các tệp đầu ra như `.swu`, RootFS, Kernel Image và các artifact liên quan.
 - Điều kiện chuyển dịch: Sau khi đóng gói hoàn tất, hệ thống giải phóng tài nguyên và quay về trạng thái S0.
- S8 (TERMINATED): Trạng thái dừng an toàn. Hệ thống đóng Flask Backend, dừng các tiến trình con đang chạy và giải phóng tài nguyên trước khi kết thúc.

- S_ERR (FAULT_LOGGER): Trạng thái xử lý lỗi, ghi nhận nhật ký hệ thống và gửi thông báo lỗi tới giao diện người dùng.
 - Điều kiện chuyển dịch: Sau khi xử lý lỗi hoàn tất, hệ thống quay trở lại trạng thái S0.

3.5 Thiết kế dịch vụ đám mây và IoT

3.5.1 Thiết kế cơ chế truyền dữ liệu giữa Gateway và CoreIoT

Trong hệ thống được xây dựng, Raspberry Pi 4 đóng vai trò gateway trung tâm, thực hiện chức năng thu thập dữ liệu từ các cảm biến, xử lý trạng thái hệ thống và truyền dữ liệu lên nền tảng quản lý IoT CoreIoT. Để đáp ứng yêu cầu truyền dữ liệu theo thời gian thực với mức tiêu thụ tài nguyên thấp, hệ thống sử dụng giao thức MQTT làm cơ chế giao tiếp chính giữa gateway và cloud server.

Gateway thực hiện kết nối đến MQTT Broker tại địa chỉ `App.coreiot.io` thông qua cổng 1883. Quá trình xác thực sử dụng access token của thiết bị làm username cho phiên kết nối MQTT. Sau khi kết nối thành công, thiết bị định kỳ publish dữ liệu lên topic `v1/devices/me/telemetry` với chu kỳ 5 giây nhằm phục vụ việc giám sát và lưu trữ dữ liệu trên nền tảng cloud.

Dữ liệu được truyền lên hệ thống được đóng gói dưới dạng JSON nhằm đảm bảo tính linh hoạt và khả năng mở rộng trong quá trình xử lý phía server. Payload bao gồm các thông số môi trường được thu thập từ hệ thống cảm biến như nhiệt độ, độ ẩm, áp suất, cường độ ánh sáng và nồng độ bụi mịn PM2.5. Các giá trị này lần lượt được đọc từ các file trong hệ thống file của Linux như:

- `/sys/class/sensorhub/hub0/temp`: dữ liệu nhiệt độ.
- `/sys/class/sensorhub/hub0/hum`: dữ liệu độ ẩm.
- `/sys/class/sensorhub/hub0/pressure`: dữ liệu áp suất môi trường.
- `/sys/class/sensorhub/hub0/lux`: dữ liệu cường độ ánh sáng.
- `/sys/class/sensorhub/hub0/pm25`: dữ liệu bụi mịn PM2.5.

Ngoài dữ liệu cảm biến thời gian thực, hệ thống còn tích hợp giá trị dự báo nồng độ PM2.5 trong 15 phút tiếp theo thông qua file tạm `/tmp/ai_predict.txt`. Giá trị này được sử dụng nhằm hỗ trợ cơ chế cảnh báo sớm và nâng cao khả năng giám sát môi trường trong hệ thống kho thông minh.

Bên cạnh dữ liệu cảm biến, payload còn chứa các trường bổ trợ như mã thiết bị (node), chế độ hoạt động (mode) và timestamp thời gian thực nhằm hỗ trợ đồng bộ và lưu trữ dữ liệu trên nền tảng cloud. Trước khi gửi lên MQTT Broker, gateway thực hiện tiền xử lý dữ liệu để chuẩn hóa các giá trị cảm biến, trong đó nhiệt độ được chia cho 10 và áp suất được chia cho 100 để đưa về đơn vị thực tế.

Sau quá trình tiền xử lý, gateway tiến hành đánh giá trạng thái hoạt động của hệ thống dựa trên các ngưỡng cảnh báo đã được thiết lập. Nếu nhiệt độ vượt quá 45°C hoặc giá trị dự báo PM2.5 lớn hơn 100, hệ thống sẽ chuyển sang trạng thái EMERGENCY_FIRE và kích hoạt quạt làm mát ở mức cao nhất nhằm đảm bảo an toàn cho môi trường vận hành.

Điều kiện	Trạng thái hệ thống	Mức quạt
Nhiệt độ < 30°C	NORMAL	LOW
30°C ≤ Nhiệt độ < 40°C	WARNING	MEDIUM
Nhiệt độ ≥ 40°C	CRITICAL	HIGH
Nhiệt độ > 45°C hoặc PM2.5 dự báo > 100	EMERGENCY_FIRE	HIGH

Bảng 3.6: Cơ chế xác định trạng thái hệ thống

Bên cạnh cơ chế điều khiển theo ngưỡng nhiệt độ, hệ thống còn triển khai cơ chế giám sát trạng thái kéo dài nhằm phát hiện các tình huống bất thường trong quá trình vận hành. Nếu quạt duy trì ở mức HIGH liên tục trong hơn 30 giây, gateway sẽ chuyển hệ thống sang trạng thái EMERGENCY_SYSTEM. Cơ chế này giúp nâng cao khả năng giám sát và tăng mức độ an toàn cho hệ thống trong môi trường thực tế.

Song song với quá trình điều khiển quạt, relay cũng được điều khiển tự động dựa trên nhiệt độ môi trường. Relay sẽ được kích hoạt khi nhiệt độ vượt quá 25°C và tự động tắt khi nhiệt độ giảm xuống dưới ngưỡng quy định.

Sau khi hoàn tất quá trình xử lý và đánh giá trạng thái, gateway tiến hành tổng hợp toàn bộ dữ liệu cảm biến và trạng thái hệ thống thành một payload hoàn chỉnh trước khi gửi lên nền tảng CoreIoT thông qua giao thức MQTT. Payload được xây dựng dưới dạng JSON nhằm đảm bảo khả năng mở rộng và dễ dàng tích hợp với hệ thống backend.

Cấu trúc dữ liệu được gửi lên hệ thống được mô tả trong Bảng 3.7.

Trường dữ liệu	Ý nghĩa	Ví dụ
temperature	Nhiệt độ môi trường	32.5
humidity	Độ ẩm môi trường	75
light_intensity	Cường độ ánh sáng	420
pm25	Nồng độ bụi mịn PM2.5	18
stat	Trạng thái hệ thống	WARNING
node	Mã thiết bị gateway	ESP_01
mode	Chế độ hoạt động	day
timestamp	Thời gian gửi dữ liệu	1715259000

Bảng 3.7: Cấu trúc payload dữ liệu gửi lên CoreIoT

Từ cấu trúc trên, payload JSON được gửi lên MQTT Broker có dạng như sau:

```
{  
  "temperature": 32.5,  
  "humidity": 75,  
  "light_intensity": 420,  
  "pm25": 18,  
  "stat": "WARNING",  
  "node": "ESP_01",  
  "mode": "day",  
  "timestamp": "1715259000"  
}
```

3.5.2 Thiết kế Rule Chain xử lý dữ liệu IoT

Nhóm đã nghiên cứu và hiện thực cơ chế Rule Chain nhằm xử lý telemetry và điều khiển thiết bị IoT theo hướng tự động hóa. Rule Chain được triển khai trên cloud server và đóng vai trò như một pipeline xử lý dữ liệu trung tâm, cho phép tiếp nhận telemetry từ các local rule chain trên thiết bị Linux nhúng, sau đó phân tích dữ liệu, thực hiện điều khiển và phát sinh cảnh báo tương ứng.

Các trường dữ liệu chính rule chain xử lý logic:

```
{  
  "stat": "WARNING",  
  "node": "ESP_01",  
  "mode": "day"  
}
```

Trong đó, trường stat biểu diễn trạng thái hoạt động hoặc mức độ cảnh báo của thiết bị, trường node dùng để định danh thiết bị gửi dữ liệu và trường mode biểu diễn chế độ hoạt động

hiện tại của hệ thống như chế độ ngày hoặc đêm. Các telemetry này được local rule chain xử lý sơ bộ trước khi gửi lên cloud để giảm tải cho hệ thống trung tâm và tối ưu khả năng phản hồi.

Sau khi nhận telemetry, Rule Chain trên cloud sẽ sử dụng các script node để kiểm tra giá trị của từng trường dữ liệu và quyết định hướng xử lý tương ứng. Ví dụ, nếu trường mode có giá trị day, hệ thống sẽ kích hoạt các rule dành cho chế độ hoạt động ban ngày. Tương tự, khi trường stat mang giá trị WARNING hoặc CRITICAL, Rule Chain sẽ chuyển dữ liệu đến các node alarm để tạo cảnh báo trên Dashboard giám sát. Việc xử lý dựa trên telemetry giúp hệ thống có khả năng phản ứng tự động theo trạng thái thực tế của thiết bị.

Bên cạnh chức năng xử lý telemetry, nhóm còn tích hợp cơ chế RPC (Remote Procedure Call) nhằm hỗ trợ điều khiển thiết bị từ xa thông qua cloud server. Khi người dùng thao tác trên Dashboard, Rule Chain sẽ tạo RPC request và gửi trực tiếp đến node tương ứng dựa trên giá trị node trong telemetry. Thiết bị sau khi nhận RPC sẽ thực thi các tác vụ như bật hoặc tắt thiết bị, thay đổi chế độ hoạt động hoặc cập nhật trạng thái hệ thống, sau đó gửi phản hồi ngược về cloud để cập nhật trạng thái theo thời gian thực.

Chương 4

Hiện thực nền tảng nhúng

Chương này trình bày quá trình cài đặt, tích hợp và kết quả kiểm thử từng thành phần kỹ thuật của nền tảng.

- 4.1 Hiện thực Bảo mật Hệ thống**
- 4.2 Hiện thực Device Driver**
- 4.3 Hiện thực Khối xử lý AI**
- 4.4 Hiện thực hệ thống cập nhập từ xa OTA**
- 4.5 Hiện thực Tool Build Yocto**

4.1 Hiện thực bảo mật hệ thống

4.1.1 Hiện thực giảm thiểu bề mặt tấn công

Để giảm thiểu việc tấn công bề mặt cho hệ thống Yocto, không thể chỉ dừng lại ở việc cấu hình hệ thống mà còn bao gồm việc kiểm soát chặt chẽ các thiết lập mặc định và thành phần của hệ thống. Khi tạo một dự án mới, mặc định là hệ thống sẽ cung cấp một file `local.conf` ban đầu bao gồm các tính năng như cho phép mật khẩu rỗng hoặc đăng nhập root trực tiếp bằng cách sử dụng biến `EXTRA_IMAGE_FEATURES` với dòng lệnh:



```
Open ▾ [1]
local.conf
~/.yocto/poky/build/conf

DISTRO_FEATURES:append = " systemd"
DISTRO_FEATURES:remove = " sysvinit"

VIRTUAL-RUNTIME_init_manager = "systemd"
VIRTUAL-RUNTIME_init_scripts = ""

DISTRO_FEATURES:append = " security"

#
# =====
# HARDENING
# =====
EXTRA_IMAGE_FEATURES:remove = "allow-empty-password empty-root-password allow-root-login"
```

Hình 4.1: Giảm thiểu bề mặt tấn công trong Yocto Project

Để đảm bảo an toàn cho hệ thống, các lựa chọn “allowempty-password”, “allowrootlogin” hoặc “emptyrootpassword” cần đảm bảo không phải là một trong những `IMAGE_FEATURES`. Ngoài ra, chỉ giữ lại các thành phần thực sự cần thiết cho hệ thống Yocto thông qua các biến cấu hình như `DISTRO_FEATURES` và `IMAGE_FEATURES`, nhằm giảm thiểu bề mặt tấn công. Bên cạnh đó, cần xem xét và tinh chỉnh cấu hình nhân hệ điều hành để đảm bảo chỉ bao gồm các trình điều khiển và tính năng cần thiết, tránh đưa vào những thành phần dư thừa có thể gây rủi ro bảo mật thông qua việc tùy chỉnh các thành phần trong kernel như `CONFIG_DEVMEM` và `STRICT_DEVMEM` nhằm hạn chế khả năng truy cập trực tiếp vào bộ nhớ thiết bị từ không gian người dùng. Điều này góp phần ngăn chặn các hành vi khai thác ở mức thấp, từ đó nâng cao mức độ an toàn tổng thể của hệ thống.

Một biện pháp quan trọng khác nhằm giảm thiểu rủi ro bị tấn công bề mặt là hạn chế hoặc không cho phép người dùng cài đặt thêm phần mềm sau khi thiết bị đã được triển khai. Trong các hệ thống sử dụng Yocto, phần mềm thường được build sẵn vào image, và hệ thống không được thiết kế như các bản phân phối Linux thông thường với trình quản lý gói mở cho người dùng. Việc cho phép cài đặt thêm package có thể làm mất kiểm soát cấu hình hệ thống, dẫn đến nguy cơ xuất hiện các phần mềm không được kiểm chứng hoặc chứa lỗ hổng bảo mật. Để giảm thiểu các rủi ro này, các developer không sử dụng các định dạng package như `ipk`, `rpm` hoặc `deb` trong quá trình build, đồng thời loại bỏ các thành phần package manager khỏi image. Ngoài ra, việc loại bỏ feature `package-management` và thiết lập hệ thống file ở chế độ chỉ đọc (`read-only root filesystem`) sẽ giúp ngăn chặn việc cài đặt hoặc thay đổi phần mềm trong quá trình vận hành. Việc tinh chỉnh cấu hình kernel, loại bỏ các driver và tính năng không cần thiết,

cũng như kích hoạt các tùy chọn bảo mật, giúp hạn chế khả năng khai thác từ các lỗ hổng mức thấp. Những biện pháp này góp phần xây dựng một hệ thống an toàn, ổn định và phù hợp với môi trường nhúng.

4.1.2 Hiện thực Selinux

Nhóm thực hiện bổ sung layer meta-selinux vào môi trường build:

```
meta-selinux/  
├── classes/  
├── conf/  
├── recipes-core/  
├── recipes-kernel/  
├── recipes-security/  
│   ├── retpolicy/  
│   ├── selinux/  
│   ├── packagegroups/  
│   └── images/  
└── recipes-support/
```

Hình 4.2: Cấu trúc thư mục chính của layer meta-selinux

Hình 4.2 mô tả cấu trúc thư mục chính của layer meta-selinux trong Yocto Project. Layer này cung cấp các thành phần cần thiết để tích hợp cơ chế bảo mật SELinux vào hệ thống Linux nhúng. Trong đó, thư mục `classes` chứa các lớp hỗ trợ quá trình build và cấu hình SELinux; thư mục `conf` lưu trữ các thiết lập cấu hình của layer. Các thư mục `recipes-core`, `recipes-kernel` và `recipes-support` chứa các recipe bổ sung hoặc chỉnh sửa cho các thành phần hệ thống liên quan đến SELinux.

Đặc biệt, thư mục `recipes-security` đóng vai trò trung tâm trong việc triển khai cơ chế bảo mật. Bên trong thư mục này, `retpolicy` chứa các policy chuẩn của SELinux, `selinux` cung cấp các gói và công cụ hỗ trợ SELinux, `packagegroups` định nghĩa các nhóm package cần thiết, trong khi `images` hỗ trợ tích hợp SELinux trực tiếp vào image được build. Cấu trúc phân lớp này giúp việc quản lý, mở rộng và tùy chỉnh chính sách bảo mật trở nên linh hoạt và thuận tiện hơn trong quá trình phát triển hệ thống nhúng sử dụng Yocto Project.

SELinux hỗ trợ nhiều loại policy khác nhau nhằm đáp ứng các yêu cầu bảo mật và mức độ kiểm soát truy cập khác nhau của hệ thống. Trong đó, hai policy phổ biến nhất là `retpolicy-targeted` và `retpolicy-mls`. Mỗi policy được thiết kế cho các mục đích sử dụng riêng, từ các hệ thống Linux thông thường đến các môi trường yêu cầu bảo mật nghiêm ngặt. Việc lựa chọn policy phù hợp có ảnh hưởng trực tiếp đến mức độ bảo vệ, hiệu năng và độ phức tạp trong quá trình quản trị hệ thống.

Bảng 4.1: So sánh các policy phổ biến của SELinux

Policy	Đặc điểm	Mức độ bảo mật	Trường hợp sử dụng
refpolicy-targeted	Chỉ áp dụng cơ chế kiểm soát truy cập cho các tiến trình và dịch vụ quan trọng	Trung bình đến cao	Hệ thống nhúng, máy chủ thông thường, môi trường cân cân bằng giữa bảo mật và hiệu năng
refpolicy-mls	Áp dụng cơ chế Multi-Level Security (MLS), kiểm soát truy cập theo nhiều cấp độ bảo mật	Rất cao	Hệ thống quân sự, chính phủ hoặc các môi trường yêu cầu phân cấp dữ liệu nghiêm ngặt
minimum	Policy tối giản với số lượng rule hạn chế, chỉ bảo vệ các thành phần cơ bản	Cơ bản	Hệ thống tài nguyên hạn chế hoặc môi trường thử nghiệm

Trong phạm vi đề tài, policy `refpolicy-targeted` được lựa chọn để triển khai trên hệ thống Yocto Linux. Nguyên nhân là do policy này cung cấp mức độ bảo mật phù hợp cho các hệ thống nhúng IoT nhưng vẫn đảm bảo tính ổn định và hiệu năng của hệ thống. So với `refpolicy-mls`, policy `targeted` có cấu hình đơn giản hơn, ít tiêu tốn tài nguyên và dễ triển khai trong môi trường thiết bị nhúng có giới hạn về bộ nhớ và năng lực xử lý. Đồng thời, policy này vẫn đủ khả năng bảo vệ các dịch vụ quan trọng khỏi các hành vi truy cập trái phép hoặc leo thang đặc quyền trong quá trình vận hành hệ thống.

Để tích hợp SELinux vào hệ thống Yocto, trước tiên cần bổ sung layer `meta-selinux` vào danh sách layer trong file `bblayers.conf`. Layer này cung cấp các recipe, policy và công cụ cần thiết để hỗ trợ cơ chế Mandatory Access Control (MAC) trên hệ thống Linux nhúng. Đồng thời, do `meta-selinux` phụ thuộc vào các layer từ repository `meta-openembedded`, bao gồm `meta-oe` và `meta-python`, các layer này cũng cần được thêm vào môi trường build để đảm bảo đầy đủ thư viện và thành phần hỗ trợ liên quan.

Sau khi thêm layer, hệ thống được cấu hình để kích hoạt SELinux thông qua việc bổ sung `selinux` vào biến `DISTRO_FEATURES` trong file `local.conf`. Việc này giúp Yocto tự động tích hợp các thành phần SELinux vào image trong quá trình build. Ngoài ra, các package phụ trợ như `acl`, `xattr` và `pam` cũng được thêm vào nhằm đảm bảo SELinux có đầy đủ cơ chế quản lý quyền truy cập, extended attributes và xác thực người dùng.

Tiếp theo, hệ thống cần lựa chọn chính sách bảo mật phù hợp thông qua biến `PREFERRED_PROVIDER_virtual/refpolicy` sử dụng policy `refpolicy-targeted`.

Sau khi hoàn tất các bước cấu hình, hệ thống được build lại để tạo ra image tích hợp SELinux.

```
# =====  
# SELinux  
# =====  
DISTRO_FEATURES:append = " pam selinux acl xattr"  
PREFERRED_PROVIDER_virtual/refpolicy ?= "refpolicy-targeted"  
FIRST_BOOT_RELABEL = "1"  
RPI_EXTRA_CMDLINE:append = "selinux=1 enforcing=1"S
```

Hình 4.3: Selinux

Trong quá trình vận hành, SELinux có thể hoạt động ở ba chế độ chính: enforcing, permissive và disabled. Ở chế độ enforcing, SELinux thực thi nghiêm ngặt các chính sách bảo mật, mọi hành vi truy cập không được phép sẽ bị chặn. Ở chế độ permissive, hệ thống không chặn các hành vi vi phạm mà chỉ ghi lại nhật ký, giúp nhà phát triển theo dõi và điều chỉnh chính sách phù hợp trong giai đoạn kiểm thử. Trong khi đó, chế độ disabled sẽ vô hiệu hóa hoàn toàn SELinux, hệ thống quay về cơ chế phân quyền truyền thống. Việc kiểm tra trạng thái hoạt động của SELinux có thể được thực hiện thông qua các công cụ như `getenforce` hoặc `sestatus`. Thông qua đó, có thể xác định chế độ hiện tại của SELinux và đánh giá hiệu quả của các chính sách bảo mật đã được áp dụng trong hệ thống.

4.1.3 Hiện thực dm-verity

Quá trình triển khai dm-verity nhằm bảo vệ tính toàn vẹn của hệ thống được thực hiện qua các giai đoạn từ xây dựng hình ảnh hệ thống (*system image*) tại môi trường phát triển cho đến quá trình thực thi kiểm chứng tại thiết bị đích. Cụ thể:

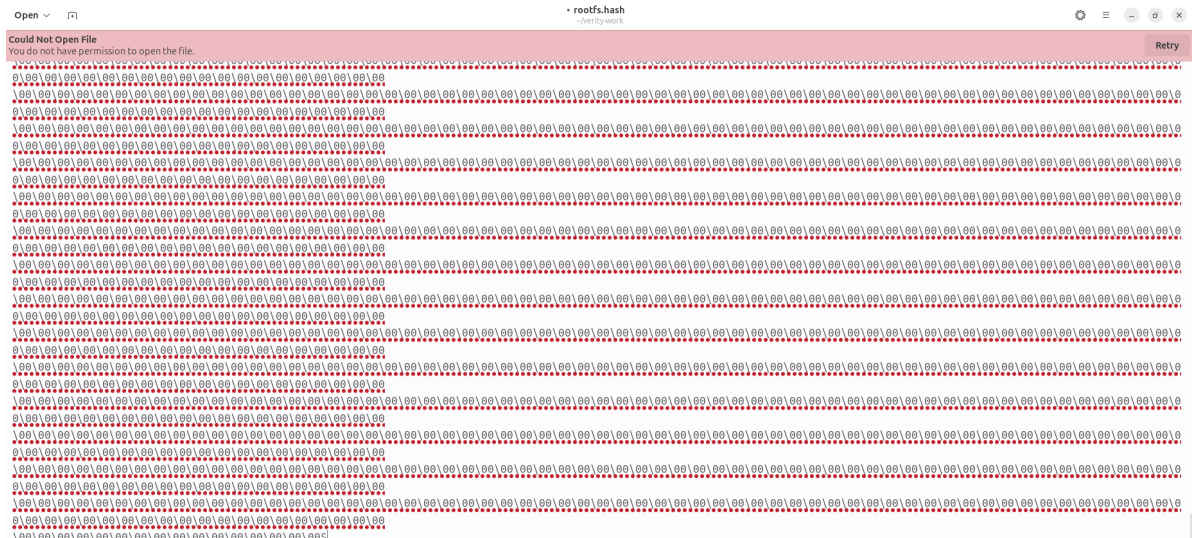
4.1.3.1 Giai đoạn tiền xử lý và tách phân vùng dữ liệu

Tiến trình bắt đầu bằng việc xây dựng image hệ thống gốc được đóng gói dưới định dạng `.wic`. Từ image này, phân vùng *rootfs* tương ứng với phân vùng p2 được tách riêng biệt để làm dữ liệu đầu vào cho quá trình tạo dữ liệu (*metadata*) của dm-verity. Đây là bước then chốt nhằm đảm bảo toàn bộ thông tin băm (*hash*) được sinh ra từ chính xác nội dung cuối cùng của *rootfs* sau khi biên dịch, tạo sự đồng nhất tuyệt đối giữa dữ liệu tính toán và dữ liệu thực tế trên thiết bị.

4.1.3.2 Khởi tạo cấu trúc cây băm (Hash Tree) và xác thực metadata

Sử dụng công cụ `veritysetup`, hệ thống tiến hành chia phân vùng p2 thành các khối dữ liệu (*blocks*) có kích thước cố định. Tại đây, mỗi khối dữ liệu được xử lý qua thuật toán băm SHA-256 để tạo ra các giá trị băm tương ứng. Các giá trị này được tổ chức theo cấu trúc cây băm nhiều mức (*Merkle Tree*), dẫn đến hai thành phần đầu ra quan trọng:

- `rootfs.hash`: tệp tin chứa toàn bộ cấu trúc cây băm của phân vùng.



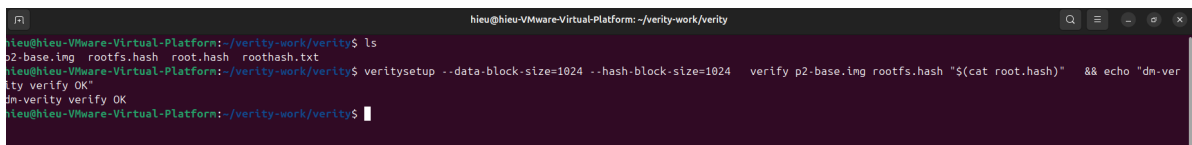
Hình 4.4: Cấu trúc cây băm của rootfs

- `root.hash`: giá trị băm gốc (*root hash*), đại diện duy nhất cho tính toàn vẹn của toàn bộ phân vùng *rootfs*.



Hình 4.5: Giá trị băm gốc (root hash)

Để đảm bảo độ tin cậy, lệnh `veritysetup verify` được thực thi nhằm kiểm tra tính nhất quán giữa dữ liệu gốc và các thành phần *metadata* vừa khởi tạo trước khi tiến hành nạp vào thiết bị.



Hình 4.6: Kết quả xác thực metadata của dm-verity

4.1.3.3 Triển khai và cấu hình giai đoạn khởi động sớm

Sau khi xác thực thành công, hình ảnh hệ thống được nạp vào thẻ nhớ SD. Trong cấu trúc này, phân vùng p1 đảm nhiệm vai trò khởi động (*boot partition*), trong khi p2 chứa *rootfs* cần được bảo vệ. Các tệp tin `root.hash` và `rootfs.hash` được lưu trữ vào thư mục `/verity` trên phân vùng p1, cho phép hệ thống tệp trong bộ nhớ tạm (*initramfs*) có thể truy xuất trong giai đoạn khởi động sớm.

4.1.3.4 Cơ chế kiểm chứng thực thi tại thiết bị đích

Tại thiết bị đích, tiến trình khởi động được điều khiển bởi Kernel và *initramfs*. Tập lệnh */init* trong *initramfs* thực hiện gắn kết (*mount*) phân vùng boot p1, truy xuất các tệp tin băm và sử dụng lệnh *veritysetup open* để thiết lập một thiết bị ánh xạ (*mapping device*) cho phân vùng p2. Lớp ánh xạ này đóng vai trò là một tầng trung gian kiểm soát giữa hệ điều hành và dữ liệu vật lý. Nội dung của file */init* được trình bày dưới đây:

```
1  #!/bin/sh
2
3  export PATH=/sbin:/bin:/usr/sbin:/usr/bin
4
5  panic() {
6      echo "INITRAMFS ERROR: $" > /dev/console
7      exec sh
8  }
9
10 echo "Starting dm-verity initramfs..." > /dev/console
11
12 # Mount filesystem
13 mount -t proc proc /proc || panic "mount proc failed"
14 mount -t sysfs sysfs /sys || panic "mount sys failed"
15 mount -t devtmpfs devtmpfs /dev || panic "mount dev failed"
16
17 # Load module
18 modprobe dm_mod
19 modprobe dm_verity
20 modprobe loop
21 modprobe ext4
22 modprobe vfat
23
24 # Tao thu muc chua file hash
25 mkdir -p /mnt/boot /newroot
26
27 # Mount boot partition
28 mount -t vfat -o ro /dev/mmcblk0p1 /mnt/boot \
29     || panic "boot mount failed"
30
31 # Gan file hash vao loop device
```

```
32 losetup -r /dev/loop0 /mnt/boot/rootfs.hash \  
33     || panic "loop setup failed"  
34  
35 # Kích hoạt dm-verity  
36 veritysetup open /dev/mmcblk0p2 vroot /dev/loop0 \  
37     --root-hash-file /etc/verity/root.hash \  
38     || panic "verity open failed"  
39  
40 # Mount verified rootfs  
41 mount -t ext4 -o ro /dev/mapper/vroot /newroot \  
42     || panic "rootfs mount failed"  
43  
44 # Chuyển sang rootfs thật  
45 exec switch_root /newroot /sbin/init
```

File `init` trong `initramfs` có vai trò khởi tạo môi trường tối thiểu trước khi hệ điều hành chuyển sang *root filesystem* chính đã được bảo vệ bằng cơ chế `dm-verity`. Đầu tiên, script tiến hành mount các filesystem cần thiết như `/proc`, `/sys` và `/dev` để cung cấp môi trường hoạt động cơ bản cho kernel và userspace. Sau đó, các kernel module liên quan như `dm_mod`, `dm_verity`, `loop`, `ext4` và `vfat` được nạp để hỗ trợ cơ chế xác thực filesystem và mount các phân vùng lưu trữ.

Tiếp theo, script mount phân vùng boot ở chế độ chỉ đọc để truy cập file `rootfs.hash`, đây là Merkle Hash Tree dùng cho cơ chế xác thực toàn vẹn dữ liệu. File hash này được ánh xạ vào loop device thông qua `losetup`, cho phép `dm-verity` sử dụng như một block device phục vụ quá trình kiểm tra integrity. Sau đó, lệnh `veritysetup open` được sử dụng để tạo thiết bị ánh xạ `/dev/mapper/vroot`. Từ thời điểm này, mọi block dữ liệu được đọc từ root filesystem sẽ được kernel tự động kiểm tra hash nhằm phát hiện các thay đổi trái phép.

Sau khi quá trình xác thực thành công, root filesystem đã được bảo vệ sẽ được mount ở chế độ chỉ đọc vào thư mục `/newroot`. Cuối cùng, lệnh `switch_root` được thực thi để chuyển quyền điều khiển sang hệ thống chính và tiếp tục quá trình khởi động Linux thông qua `/sbin/init`.

Mọi yêu cầu truy cập dữ liệu từ *rootfs* sẽ được đối chiếu theo thời gian thực với cấu trúc cây băm và giá trị băm gốc đã thiết lập.

- **Trường hợp dữ liệu hợp lệ:** Nếu dữ liệu trên p2 hoàn toàn khớp với *metadata*, hệ thống sẽ tiến hành gắn kết *rootfs* ở chế độ chỉ đọc (*read-only*) và thực hiện lệnh `switch_root` để chuyển sang hệ điều hành chính thức.
- **Trường hợp dữ liệu bị can thiệp:** Nếu xuất hiện bất kỳ sự sai lệch nào giữa dữ liệu thực tế và cây băm, `dm-verity` sẽ ngay lập tức phát hiện, từ chối quyền truy cập và ngăn chặn

quá trình gắn kết hệ thống. Lúc này, thiết bị sẽ chuyển sang trạng thái lỗi hoặc chế độ phục hồi (*recovery mode*).

4.2 Hiện thực Device Driver

4.2.1 Tổng quan kiến trúc Driver

Trong hệ thống, nhóm triển khai hai device driver chính bao gồm driver giao tiếp I2C và driver giao tiếp RS485 Modbus RTU trên nền tảng Linux nhúng Yocto. Các driver này đóng vai trò trung gian giữa tầng ứng dụng (user space) và phần cứng ngoại vi.

Driver được hiện thực dưới dạng character device driver, cho phép các ứng dụng user space có thể giao tiếp với phần cứng thông qua các device file trong thư mục /dev. Việc tự xây dựng driver giúp hệ thống có tính tùy biến cao, đồng thời hỗ trợ tối ưu cho các ứng dụng tự động hóa công nghiệp.

Kiến trúc tổng quát của hệ thống driver được mô tả như sau:

User Application → Device Driver → Hardware Interface → Peripheral Device

4.2.2 Hiện thực driver I2C

Trong đề tài, nhóm xây dựng một driver I2C dạng software bit-banging trên nền tảng Linux nhúng Yocto nhằm hỗ trợ giao tiếp với các thiết bị ngoại vi như màn hình OLED SSD1306 và các cảm biến môi trường. Driver được hiện thực dưới dạng character device driver cho phép ứng dụng ở user space có thể giao tiếp với phần cứng thông qua device file trong thư mục /dev.

Khác với việc sử dụng bộ điều khiển I2C phần cứng mặc định của Raspberry Pi 4, nhóm lựa chọn hiện thực giao thức I2C bằng phần mềm thông qua GPIO nhằm tăng tính tùy biến và chủ động trong việc điều khiển giao thức. Cách tiếp cận này cho phép hệ thống dễ dàng thay đổi chân SDA và SCL trong quá trình runtime, đồng thời hỗ trợ nghiên cứu cơ chế hoạt động của giao thức I2C ở mức thấp trong Linux kernel.

4.2.2.1 Kiến trúc driver

Driver được xây dựng theo mô hình character device driver của Linux kernel. Sau khi module được nạp vào hệ thống, driver sẽ đăng ký major number và tạo device file /dev/my_i2c_soft để các ứng dụng user space có thể truy cập. Luồng hoạt động của driver được mô tả như sau:

User Application → Character Device Driver → GPIO Bit-banging → I2C Device

Trong mô hình này, tầng user application gửi dữ liệu xuống kernel thông qua các system call như `read()`, `write()` và `ioctl()`. Driver sẽ xử lý dữ liệu và điều khiển trực tiếp các chân GPIO để tạo tín hiệu SDA và SCL theo chuẩn giao thức I2C.

4.2.2.2 Nguyên lý giao thức I2C

Giao thức I2C sử dụng hai đường tín hiệu bao gồm SDA (Serial Data) và SCL (Serial Clock). Quá trình truyền dữ liệu được đồng bộ bằng xung clock từ master device. Driver sử dụng cơ chế điều khiển trực tiếp GPIO để mô phỏng giao thức I2C thông qua phương pháp bit-banging. Trong driver, nhóm hiện thực các tín hiệu cơ bản của giao thức I2C bao gồm:

- START condition.
- STOP condition.
- Truyền dữ liệu theo từng bit.
- Nhận ACK từ slave device.
- Đọc dữ liệu từ slave.

4.2.2.3 Các chức năng chính của driver

Driver I2C Software được xây dựng nhằm mô phỏng cơ chế giao tiếp I2C thông qua điều khiển GPIO bằng phần mềm trong Linux Kernel. Để đảm bảo tín hiệu giao tiếp ổn định và đáp ứng đúng yêu cầu timing của giao thức I2C, driver sử dụng hàm `udelay()` để tạo các khoảng trễ ngắn giữa các cạnh xung clock trong quá trình truyền nhận dữ liệu.

Trong quá trình hoạt động, driver đảm nhiệm nhiều chức năng quan trọng liên quan đến điều khiển bus I2C và giao tiếp giữa kernel space với user space. Các chức năng chính của driver bao gồm:

- Khởi tạo, cấu hình và giải phóng các chân GPIO được sử dụng cho hai tín hiệu SDA và SCL.
- Hiện thực cơ chế tạo tín hiệu START và STOP theo đúng chuẩn giao thức I2C nhằm bắt đầu và kết thúc quá trình truyền dữ liệu.
- Hỗ trợ cơ chế truyền và nhận dữ liệu theo từng byte thông qua quá trình điều khiển tín hiệu clock và data bằng phần mềm.
- Kiểm tra và xử lý tín hiệu ACK từ slave device sau mỗi lần truyền dữ liệu nhằm đảm bảo tính chính xác của quá trình giao tiếp.
- Cung cấp khả năng đọc và ghi dữ liệu từ user space thông qua các system call như `read()` và `write()`.
- Hỗ trợ cấu hình động các chân GPIO và địa chỉ slave device thông qua cơ chế `ioctl()`, giúp tăng tính linh hoạt trong quá trình sử dụng driver.

Thông qua các chức năng trên, driver cho phép hệ thống Linux nhúng thực hiện giao tiếp với các thiết bị ngoại vi I2C mà không phụ thuộc hoàn toàn vào phần cứng I2C controller tích hợp sẵn trên vi xử lý. Điều này giúp tăng tính linh hoạt trong quá trình phát triển và kiểm thử các hệ thống nhúng sử dụng Linux Kernel.

4.2.2.4 Thiết kế cơ chế IOCTL

Driver sử dụng cơ chế `ioctl()` để cho phép ứng dụng user space cấu hình động chân SDA, SCL và địa chỉ slave device trong quá trình runtime. Các IOCTL command được định nghĩa như sau:

```
1 #define SET_SDA_PIN      _IOW(I2C_IOCTL_MAGIC, 1, int)
2 #define SET_SCL_PIN      _IOW(I2C_IOCTL_MAGIC, 2, int)
3 #define SET_SLAVE_ADDR   _IOW(I2C_IOCTL_MAGIC, 3, int)
```

Cách tiếp cận này giúp hệ thống có khả năng thay đổi cấu hình GPIO linh hoạt mà không cần biên dịch lại kernel module.

4.2.2.5 Giao tiếp giữa user space và kernel space

Driver sử dụng cấu trúc `file_operations` để đăng ký các system call với Linux kernel.

```
1 static struct file_operations fops = {
2     .unlocked_ioctl = my_ioctl,
3     .write = dev_write,
4     .read = dev_read,
5     .owner = THIS_MODULE,
6 };
```

Thông qua cơ chế này, ứng dụng user space có thể sử dụng các system call chuẩn của Linux để giao tiếp với driver.

4.2.2.6 Hiện thực giao thức I2C bằng GPIO Bit-banging

Driver hiện thực các tín hiệu cơ bản của giao thức I2C thông qua việc điều khiển trực tiếp GPIO. Tín hiệu START được hiện thực như sau:

```
1 static void i2c_start(void) {
2     gpio_direction_output(current_sda, 1);
3
4     gpio_set_value(current_scl, 1);
5     udelay(I2C_DELAY);
6 }
```

```
7     gpio_set_value(current_sda, 0);  
8     udelay(I2C_DELAY);  
9  
10    gpio_set_value(current_scl, 0);  
11    udelay(I2C_DELAY);  
12 }
```

Ngoài ra, driver còn hỗ trợ cơ chế ACK và đọc dữ liệu từ slave device nhằm đảm bảo khả năng giao tiếp hai chiều.

4.2.2.7 Tối ưu timing cho giao tiếp I2C

Trong quá trình thử nghiệm với màn hình OLED SSD1306, nhóm nhận thấy timing không ổn định có thể gây lỗi hiển thị và tạo ra hiện tượng nhiễu dữ liệu. Do đó, driver được tinh chỉnh delay giữa các cạnh clock bằng hàm `udelay()` nhằm đảm bảo thời gian setup và hold của dữ liệu trước khi phát xung clock. Trong quá trình truyền dữ liệu, driver sử dụng delay ngắn trước và sau cạnh clock để tăng độ ổn định:

```
1     gpio_set_value(current_scl, 0);  
2  
3     udelay(I2C_DELAY / 2);  
4  
5     gpio_set_value(current_sda, (byte >> i) & 1);  
6  
7     udelay(I2C_DELAY / 2);  
8  
9     gpio_set_value(current_scl, 1);  
10  
11    udelay(I2C_DELAY);
```

Việc tối ưu timing giúp cải thiện độ ổn định khi hiển thị dữ liệu trên OLED và giảm lỗi truyền nhận dữ liệu trong quá trình giao tiếp.

4.2.2.8 Tích hợp driver vào hệ thống Yocto

Driver được tích hợp vào hệ thống Yocto thông qua custom layer `meta-i2c`. Nhóm xây dựng recipe `BitBake` nhằm tự động hóa quá trình build và đóng gói kernel module. Cấu trúc layer của driver được tổ chức như sau:

```
meta-i2c/  
├── conf/  
│   └── layer.conf
```



```
├── recipes-kernel/
│   └── i2c-driver/
│       ├── files/
│       │   ├── my_i2c_soft.c
│       │   └── Makefile
│       └── i2c-soft-driver.bb
```

Recipe của driver kế thừa lớp module nhằm hỗ trợ build kernel module trên Yocto:

```
1 SUMMARY = "Dynamic Software I2C Driver"
2
3 inherit module
4
5 SRC_URI = "file://my_i2c_soft.c \
6           file://Makefile"
7
8 KERNEL_MODULE_AUTOLOAD += "my_i2c_soft"
```

Yocto hỗ trợ cơ chế tự động nạp kernel module khi hệ thống khởi động thông qua biến:

```
KERNEL_MODULE_AUTOLOAD += "my_i2c_soft"
```

Khi build image Yocto, kernel module sẽ được tích hợp vào hệ thống dưới dạng file .ko và có thể tự động nạp khi khởi động. Layer chứa driver được tích hợp vào môi trường build của Yocto thông qua lệnh:

```
bitbake-layers add-layer ../meta-i2c
```

Sau khi layer được thêm vào hệ thống, BitBake sẽ tự động quét và phân tích các recipe, configuration và metadata bên trong layer. Các recipe này sau đó được đưa vào dependency graph của quá trình build nhằm phục vụ việc biên dịch và tích hợp driver vào hệ thống Linux Embedded.

Để driver được đưa vào image cuối cùng, hệ thống sử dụng biến:

```
IMAGE_INSTALL:append = "my_i2c_soft"
```

Biến này cho phép package của driver được tự động tích hợp vào root filesystem trong quá trình build image. Nhờ đó, hệ thống Linux Embedded sau khi boot sẽ có đầy đủ kernel module và các thành phần cần thiết phục vụ quá trình runtime.

Khi image được build, Yocto sẽ tự động sinh các cấu hình cần thiết trong thư mục /etc/modules-load.d. cho phép Linux Kernel tự động nạp module trong quá trình boot hệ thống. Cơ chế này giúp driver luôn sẵn sàng hoạt động ngay sau khi thiết bị khởi động mà không cần thao tác nạp module thủ công.

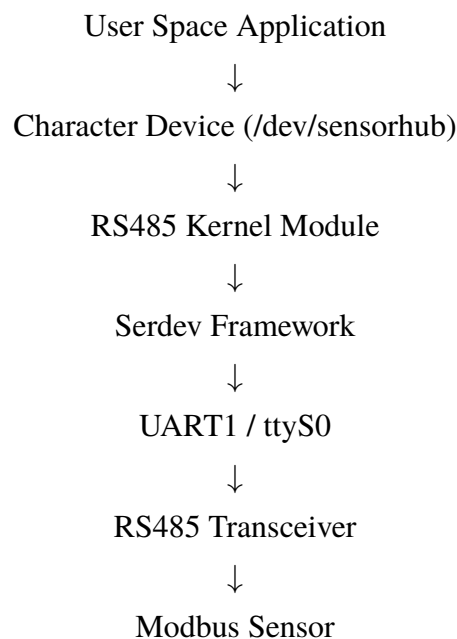
Sau khi hệ thống khởi động, kernel module có thể được kiểm tra thông qua các lệnh quản lý module của Linux. Một số lệnh phổ biến được trình bày trong Bảng 2.6.

4.2.3 Hiện thực Driver RS485 Modbus RTU

4.2.3.1 Kiến trúc Driver

Driver RS485 được xây dựng dưới dạng Linux Kernel Module nhằm phục vụ giao tiếp giữa hệ thống Linux Embedded và cảm biến môi trường thông qua giao thức Modbus RTU trên nền UART. Driver được thiết kế theo mô hình phân lớp của Linux Kernel, trong đó tầng Character Device đóng vai trò giao tiếp với User Space, còn tầng Serdev đảm nhiệm việc truyền nhận dữ liệu UART với phần cứng.

Kiến trúc tổng thể của driver được mô tả như sau:



Trong kiến trúc này, ứng dụng ở User Space có thể truy cập dữ liệu cảm biến thông qua device file `/dev/sensorhub`. Driver sử dụng Serdev Framework của Linux Kernel để quản lý giao tiếp UART thay cho cơ chế platform driver truyền thống. Cách tiếp cận này giúp giảm mức độ phụ thuộc phần cứng và tăng khả năng tích hợp trong các hệ thống Linux Embedded hiện đại.

Driver được tổ chức thành nhiều thành phần mã nguồn độc lập bao gồm:

- `rs485_modbus.c`: hiện thực Character Device và cơ chế polling dữ liệu cảm biến.
- `rs485_rtu.c`: hiện thực giao thức Modbus RTU và cơ chế truyền nhận UART.
- `rs485_modbus.h`: khai báo cấu trúc dữ liệu, macro và tài nguyên dùng chung.

Việc tách driver thành nhiều thành phần giúp mã nguồn dễ bảo trì, thuận lợi cho việc mở rộng chức năng và nâng cao khả năng tái sử dụng trong các hệ thống khác nhau.

4.2.3.2 Nguyên lý giao thức

Driver sử dụng giao thức Modbus RTU để giao tiếp với cảm biến môi trường thông qua chuẩn truyền thông RS485. Trong cơ chế này, Raspberry Pi 4 đóng vai trò Master, còn cảm biến hoạt động như Slave Device với địa chỉ mặc định là 0x15.

Khung dữ liệu Modbus RTU được xây dựng theo cấu trúc:

| Slave Address | Function Code | Data | CRC16 |

Trong đó:

- **Slave Address:** địa chỉ thiết bị Modbus.
- **Function Code:** mã chức năng truy cập thanh ghi.
- **Data:** dữ liệu thanh ghi cần đọc hoặc ghi.
- **CRC16:** giá trị kiểm tra lỗi truyền thông.

Driver hỗ trợ các mã chức năng Modbus phổ biến bao gồm:

- 0x03: đọc Holding Registers.
- 0x04: đọc Input Registers.
- 0x06: ghi Single Register.

Các giá trị cảm biến được đọc từ vùng thanh ghi Modbus và chuyển đổi sang cấu trúc dữ liệu nội bộ của driver. Để đảm bảo tính ổn định trong Linux Kernel, các giá trị nhiệt độ, độ ẩm và áp suất được biểu diễn dưới dạng fixed-point thay cho floating-point.

4.2.3.3 Các chức năng chính của Driver

Driver được xây dựng nhằm đảm bảo quá trình giao tiếp giữa hệ thống Linux Embedded và cảm biến RS485 diễn ra ổn định, đồng thời cung cấp cơ chế truy cập dữ liệu thuận tiện cho các ứng dụng ở User Space. Trong quá trình hoạt động, driver đảm nhiệm nhiều chức năng khác nhau, từ quản lý giao tiếp UART cho đến xử lý dữ liệu Modbus RTU và đồng bộ tài nguyên trong Kernel Space.

Về mặt truyền thông, driver sử dụng Serdev Framework của Linux Kernel để khởi tạo và quản lý giao tiếp UART với thiết bị RS485. Thông qua cơ chế này, driver có thể điều khiển tín hiệu DE/RE của transceiver nhằm chuyển đổi linh hoạt giữa chế độ truyền và nhận dữ liệu trên

đường truyền RS485. Bên cạnh đó, driver còn thực hiện việc xây dựng khung dữ liệu Modbus RTU, gửi request đến cảm biến và tiếp nhận phản hồi từ slave device theo đúng chuẩn giao thức.

Trong quá trình truyền nhận dữ liệu, driver triển khai cơ chế kiểm tra CRC16 để phát hiện lỗi truyền thông và đảm bảo tính toàn vẹn của dữ liệu nhận được. Sau khi khung dữ liệu hợp lệ được xác nhận, các giá trị thanh ghi Modbus sẽ được giải mã và chuyển đổi sang định dạng dữ liệu nội bộ của hệ thống. Driver hỗ trợ thu thập dữ liệu cảm biến theo chu kỳ định sẵn thông qua kernel thread, cho phép hệ thống cập nhật trạng thái môi trường một cách liên tục trong thời gian thực.

Ngoài chức năng giao tiếp phần cứng, driver còn cung cấp Character Device để các ứng dụng ở User Space có thể truy cập dữ liệu cảm biến thông qua các system call chuẩn của Linux như `open()`, `read()` và `ioctl()`. Để đảm bảo tính ổn định trong môi trường đa tiến trình, driver sử dụng nhiều cơ chế đồng bộ của Linux Kernel như mutex, spinlock và completion nhằm bảo vệ vùng dữ liệu dùng chung và hạn chế race condition trong quá trình truy cập tài nguyên.

Dữ liệu cảm biến sau khi giải mã được lưu trữ trong cấu trúc `sensor_data` như sau:

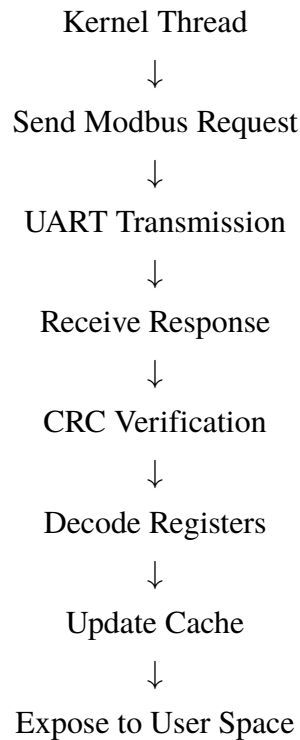
```
1 struct sensor_data {  
2     u32 humidity;  
3     s32 temperature;  
4     u32 pm25;  
5     u32 pressure;  
6     u32 lux;  
7 };
```

Cấu trúc này đóng vai trò như vùng dữ liệu trung gian giữa Kernel Space và User Space. Các giá trị cảm biến sau khi được cập nhật từ quá trình polling sẽ được lưu vào bộ nhớ đệm nội bộ của driver trước khi chuyển đến ứng dụng người dùng thông qua Character Device. Cách tổ chức này giúp giảm số lần truy cập trực tiếp đến phần cứng, đồng thời nâng cao tính ổn định và hiệu năng của hệ thống trong quá trình vận hành.

4.2.3.4 Thiết kế cơ chế hoạt động

Driver triển khai cơ chế polling định kỳ nhằm tự động cập nhật dữ liệu cảm biến trong thời gian thực. Một kernel thread riêng biệt được tạo ra để thực hiện chu trình truyền nhận Modbus liên tục.

Quy trình hoạt động của driver được mô tả như sau:



Trong quá trình hoạt động, driver sử dụng:

- mutex: đồng bộ truy cập dữ liệu cache.
- spinlock: bảo vệ vùng dữ liệu RX trong interrupt context.
- completion: đồng bộ cơ chế chờ phản hồi UART.

Cơ chế này giúp driver hoạt động ổn định trong môi trường đa tiến trình của Linux Kernel và hạn chế lỗi race condition trong quá trình truyền nhận dữ liệu.

4.2.3.5 Tích hợp Driver vào hệ thống Yocto

Driver được đóng gói dưới dạng một custom layer trong Yocto Project nhằm phục vụ quá trình build Embedded Linux Image.

Cấu trúc layer được tổ chức như sau:

```
meta-rs485/
├── conf/
│   └── layer.conf
├── recipes-kernel/
│   └── rs485-driver/
│       └── files/
│           ├── rs485_modbus.c
│           └── rs485_modbus.h
```

```
├── rs485_rtu.c
├── rs485_rtu.h
├── rs485-pi4.dts
├── rs485-pi4.dtbo
├── Makefile
└── rs485-driver.bb
```

Layer được tích hợp vào môi trường build bằng lệnh:

```
bitbake-layers add-layer ../meta-rs485
```

Sau khi layer được thêm vào hệ thống, BitBake sẽ tự động quét recipe và metadata của driver để đưa vào dependency graph của quá trình build. Driver được tích hợp vào Embedded Linux Image thông qua biến:

```
IMAGE_INSTALL:append = " rs485-driver"
```

Biến này cho phép package của driver được tự động đưa vào root filesystem trong quá trình build image. Bên cạnh đó, hệ thống còn hỗ trợ cơ chế tự động nạp kernel module khi boot thông qua biến, Cơ chế này giúp module được tự động nạp khi hệ thống khởi động mà không cần thao tác thủ công từ người dùng:

```
KERNEL_MODULE_AUTOLOAD += "rs485_sensor_mod"
```

Driver cũng sử dụng Device Tree Overlay để mô tả phần cứng UART và thiết bị RS485 trong hệ thống Raspberry Pi 4. Device Tree được định nghĩa trong file `rs485-pi4.dts` và biên dịch thành file overlay nhị phân `.dtbo`.

Một đoạn cấu hình Device Tree tiêu biểu được trình bày như sau:

```
1 rs485sensor@15 {
2     compatible = "mycompany,rs485-sensorhub";
3     reg = <0x15>;
4     current-speed = <9600>;
5 };
```

Sau khi biên dịch, file `.dtbo` sẽ được cài đặt vào thư mục:

```
/boot/overlays/
```

Linux Kernel sử dụng Device Tree Overlay để nhận diện phần cứng và thực hiện binding giữa driver với thiết bị UART tương ứng trong quá trình boot hệ thống.

4.3 Hiện thực Khối xử lý AI

Phần này mô tả quá trình chuyển đổi các thiết kế lý thuyết thành hệ thống phần mềm thực tế, bao gồm từ bước chuẩn bị dữ liệu, xây dựng cấu trúc mạng đến việc tối ưu hóa để triển khai trên phần cứng Raspberry Pi 4.

4.3.1 Thiết lập môi trường và Công cụ hỗ trợ

Toàn bộ khối xử lý AI được hiện thực bằng ngôn ngữ lập trình Python 3.10 trên môi trường máy trạm (Workstation) để tận dụng khả năng tính toán của GPU trong quá trình huấn luyện.

- **TensorFlow & Keras (v2.15+):** Nền tảng chính để xây dựng các lớp mạng nơ-ron.
- **Keras-TCN:** Thư viện mã nguồn mở chuyên biệt để triển khai các khối Temporal Convolutional Network (TCN).
- **Pandas & NumPy:** Sử dụng để cấu trúc hóa dữ liệu từ tệp CSV và thực hiện các phép toán ma trận trên mảng đa chiều.
- **Scikit-learn:** Cung cấp module `MinMaxScaler` để chuẩn hóa dữ liệu và các hàm đo lường hiệu năng.

4.3.2 Hiện thực Tiền xử lý và Cấu trúc hóa Dữ liệu

Dữ liệu thô từ trạm cảm biến môi trường (bao gồm r thông số) cần được xử lý để loại bỏ nhiễu và định dạng lại thành cấu trúc chuỗi thời gian (*time-series*).

4.3.2.1 Làm mượt và Chuẩn hóa dữ liệu

Để giảm thiểu tác động của các giá trị nhiễu tức thời (*spikes*), hệ thống hiện thực kỹ thuật trung bình trượt (*rolling mean*) với cửa sổ bằng 10 bước thời gian. Sau đó, dữ liệu được đưa về khoảng $[0, 1]$ bằng phương pháp Min-Max Scaling nhằm tăng tốc độ hội tụ của thuật toán tối ưu Adam.

```
1 # Làm mượt dữ liệu PM2.5 và Sound
2 df['pm2p5'] = df['pm2p5'].rolling(window=10).mean().bfill()
3 df['sound'] = df['sound'].rolling(window=10).mean().bfill()
4
5 # Chuẩn hóa Min-Max
6 scaler = MinMaxScaler()
7 scaled_data = scaler.fit_transform(df)
```

4.3.2.2 Cấu trúc cửa sổ trượt (Sliding Window)

Hàm `create_dataset()` được hiện thực để chuyển đổi mảng dữ liệu hai chiều thành mảng ba chiều có dạng:

(samples, time_steps, features)

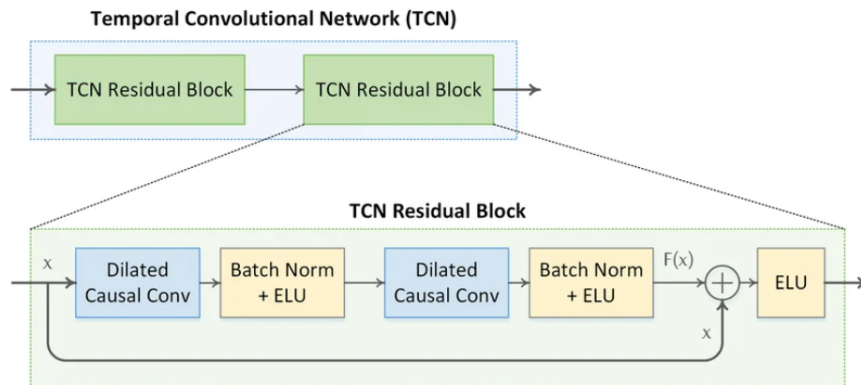
Trong đó:

- `samples`: số lượng mẫu huấn luyện,
- `time_steps`: số bước thời gian đầu vào (*look_back*),
- `features`: số đặc trưng tại mỗi bước thời gian.

```
1 def create_dataset(dataset, look_back=30, forecast=1, target_idx
2     =0):
3
4     X, y = [], []
5
6     for i in range(len(dataset) - look_back - forecast):
7         # Lay chuoai dau vao gom look_back buoc thoi gian
8         X.append(dataset[i:(i + look_back), :])
9
10        # Lay gia tri muc tieu tai thoi diem du bao
11        y.append(dataset[i + look_back + forecast - 1,
12                        target_idx])
13
14    return np.array(X).astype(np.float32), np.array(y).astype(np
15        .float32)
```

4.3.3 Hiện thực Kiến trúc Mạng TCN

Mô hình được xây dựng theo kiểu tuần tự (Sequential) nhằm tối ưu hóa luồng dữ liệu từ lớp đầu vào đến lớp dự báo đầu ra.



Hình 4.7: Mô hình TCN

4.3.3.1 Phân tích chi tiết từ hiện thực

- **Lớp TCN:** Được cấu hình với 32 bộ lọc (`nb_filters = 32`) và kích thước nhân tích chập (`kernel_size = 3`). Với mảng hệ số giãn cách:

$$[1, 2, 4, 8]$$

lớp này có khả năng trích xuất đặc trưng trên phạm vi thời gian rộng mà vẫn bảo toàn tính nhân quả của chuỗi dữ liệu.

- **Các lớp Dense:** Lớp ẩn với 16 nơ-ron sử dụng hàm kích hoạt ReLU giúp mô hình học được các quan hệ phi tuyến tính phức tạp giữa các thông số môi trường.
- **Kích thước mô hình:** Tổng số tham số huấn luyện là 23 233, tương đương dung lượng khoảng 90.75 KB. Đây là kích thước rất nhỏ, phù hợp để triển khai trên Raspberry Pi 4 và đảm bảo tốc độ suy luận nhanh trong thời gian thực.

4.3.4 Hiện thực Quy trình Huấn luyện và Giám sát

Giai đoạn này tập trung vào việc hiện thực hóa quá trình huấn luyện mô hình TCN trên tập dữ liệu đã được tiền xử lý. Hệ thống được thiết kế để tự động theo dõi hiệu suất và dừng đúng thời điểm nhằm tối ưu hóa thời gian tính toán.

4.3.4.1 Hiện thực cấu hình tham số huấn luyện

Các tham số điều khiển quá trình học (*hyperparameters*) được thiết lập trực tiếp trong mã nguồn `main.py` để đảm bảo tính ổn định và khả năng hội tụ của mạng nơ-ron.

- **Thuật toán tối ưu (Optimizer):** Sử dụng Adam với tốc độ học (*learning rate*) là 0.001.

- **Hàm mất mát (Loss Function):** Sử dụng Mean Squared Error (MSE), đồng thời theo dõi Mean Absolute Error (MAE).
- **Kích thước lô (Batch Size):** Thiết lập bằng 32 mẫu cho mỗi lần cập nhật trọng số.

4.3.4.2 Hiện thực cơ chế Giám sát và Dừng sớm (Early Stopping)

Để ngăn chặn hiện tượng quá khớp (*overfitting*) và tiết kiệm tài nguyên hệ thống, callback EarlyStopping được tích hợp vào quá trình huấn luyện.

```
1 early_stop = EarlyStopping(  
2     monitor='val_loss',  
3     patience=15,  
4     restore_best_weights=True  
5 )
```

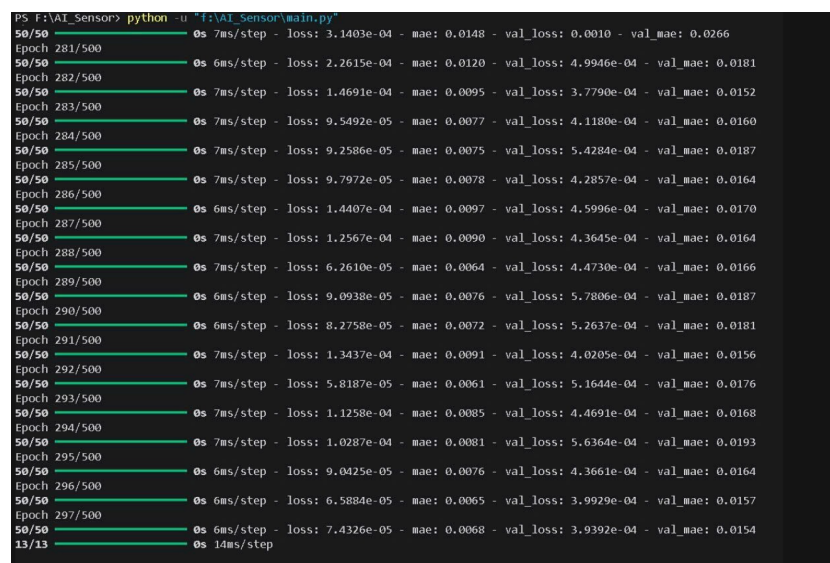
Callback này theo dõi giá trị `val_loss`. Nếu sau 15 epoch liên tiếp không có cải thiện, quá trình huấn luyện sẽ tự động dừng và khôi phục lại bộ trọng số tốt nhất.

4.3.4.3 Nhật ký thực thi quá trình huấn luyện

Quá trình huấn luyện được khởi chạy bằng lệnh:

```
1 python main.py
```

Trong suốt quá trình này, hệ thống xuất ra nhật ký chi tiết cho từng epoch để theo dõi thời gian xử lý và giá trị sai số.



```
PS F:\AI_Sensor> python -u "F:\AI_Sensor\main.py"  
50/50 0s 7ms/step - loss: 3.1403e-04 - mae: 0.0148 - val_loss: 0.0010 - val_mae: 0.0266  
Epoch 281/500  
50/50 0s 6ms/step - loss: 2.2615e-04 - mae: 0.0120 - val_loss: 4.9946e-04 - val_mae: 0.0181  
Epoch 282/500  
50/50 0s 7ms/step - loss: 1.4691e-04 - mae: 0.0095 - val_loss: 3.7790e-04 - val_mae: 0.0152  
Epoch 283/500  
50/50 0s 7ms/step - loss: 9.5492e-05 - mae: 0.0077 - val_loss: 4.1180e-04 - val_mae: 0.0160  
Epoch 284/500  
50/50 0s 7ms/step - loss: 9.2586e-05 - mae: 0.0075 - val_loss: 5.4284e-04 - val_mae: 0.0187  
Epoch 285/500  
50/50 0s 7ms/step - loss: 9.7972e-05 - mae: 0.0078 - val_loss: 4.2857e-04 - val_mae: 0.0164  
Epoch 286/500  
50/50 0s 6ms/step - loss: 1.4407e-04 - mae: 0.0097 - val_loss: 4.5996e-04 - val_mae: 0.0170  
Epoch 287/500  
50/50 0s 7ms/step - loss: 1.2567e-04 - mae: 0.0090 - val_loss: 4.3645e-04 - val_mae: 0.0164  
Epoch 288/500  
50/50 0s 7ms/step - loss: 6.2610e-05 - mae: 0.0064 - val_loss: 4.4730e-04 - val_mae: 0.0166  
Epoch 289/500  
50/50 0s 6ms/step - loss: 9.0938e-05 - mae: 0.0076 - val_loss: 5.7806e-04 - val_mae: 0.0187  
Epoch 290/500  
50/50 0s 6ms/step - loss: 8.2758e-05 - mae: 0.0072 - val_loss: 5.2637e-04 - val_mae: 0.0181  
Epoch 291/500  
50/50 0s 7ms/step - loss: 1.3437e-04 - mae: 0.0091 - val_loss: 4.0205e-04 - val_mae: 0.0156  
Epoch 292/500  
50/50 0s 7ms/step - loss: 5.8187e-05 - mae: 0.0061 - val_loss: 5.1644e-04 - val_mae: 0.0176  
Epoch 293/500  
50/50 0s 7ms/step - loss: 1.1258e-04 - mae: 0.0085 - val_loss: 4.4691e-04 - val_mae: 0.0168  
Epoch 294/500  
50/50 0s 7ms/step - loss: 1.0287e-04 - mae: 0.0081 - val_loss: 5.6364e-04 - val_mae: 0.0193  
Epoch 295/500  
50/50 0s 6ms/step - loss: 9.0425e-05 - mae: 0.0076 - val_loss: 4.3661e-04 - val_mae: 0.0164  
Epoch 296/500  
50/50 0s 6ms/step - loss: 6.5884e-05 - mae: 0.0065 - val_loss: 3.9929e-04 - val_mae: 0.0157  
Epoch 297/500  
50/50 0s 6ms/step - loss: 7.4326e-05 - mae: 0.0068 - val_loss: 3.9392e-04 - val_mae: 0.0154  
13/13 0s 14ms/step
```

Hình 4.8: Nhật ký quá trình huấn luyện mô hình TCN trên môi trường máy trạm

Dựa trên nhật ký thực thi thực tế ở Hình 4.8, có thể rút ra các nhận xét sau:

- **Trạng thái dừng:** Mặc dù số lượng epoch tối đa được thiết lập là 500, cơ chế Early Stopping đã tự động dừng quá trình huấn luyện tại epoch 297.
- **Hiệu suất thời gian:** Mỗi bước huấn luyện chỉ mất trung bình từ 6 ms đến 7 ms.
- **Đóng gói kết quả:** Sau khi huấn luyện kết thúc, mô hình được lưu vào tệp `tcn_model.keras` để phục vụ cho bước chuyển đổi sang TensorFlow Lite.

4.4 Hiện thực hệ thống cập nhật từ xa OTA

Phần này trình bày chi tiết quá trình hiện thực hóa giải pháp cập nhật phần mềm từ xa (Over-the-Air – OTA) cho thiết bị Raspberry Pi 4. Quá trình hiện thực bao gồm việc cấu hình môi trường build Yocto, tùy chỉnh các recipe hệ thống, xây dựng kịch bản mô tả và triển khai thực tế trên phần cứng.

4.4.1 Tích hợp các Layer và Cấu hình môi trường Yocto

Để hệ thống có khả năng xử lý OTA, bước đầu tiên là tích hợp framework SWUpdate vào quy trình xây dựng image của Yocto Project (nhánh Kirkstone).

- **Quản lý Layer:** Việc tích hợp được thực hiện thông qua sửa đổi tệp `conf/bblayers.conf`. Hệ thống sử dụng layer cộng đồng `meta-swupdate` để cung cấp các công cụ lõi và layer tùy chỉnh `meta-ota` để chứa các cấu hình riêng biệt của đồ án. Cách phân tách này giúp quản lý mã nguồn dễ dàng và tránh xung đột khi cập nhật framework.
- **Khai báo thành phần hệ thống:** Trong tệp `conf/local.conf`, biến `IMAGE_INSTALL:append` được khai báo để yêu cầu BitBake đóng gói các gói cần thiết vào Root File System. Các thành phần bao gồm:
 - `swupdate`: chương trình chính thực hiện cập nhật.
 - `swupdate-www`: giao diện Web UI.
 - `libconfig`: thư viện phân tích cú pháp cho tệp `sw-description`.
 - `u-boot-fw-utils`: công cụ giao tiếp với bootloader U-Boot.
- **Tùy chỉnh Init System:** Hệ thống sử dụng `systemd` làm trình quản lý dịch vụ. Vì vậy, SWUpdate được cấu hình để hoạt động như một daemon và tự động khởi động cùng hệ thống.

```
1 POKY_BBLAYERS_CONF_VERSION = "2"  
2  
3 BBPATH = "${TOPDIR}"
```

```
4 BBFILES ?= ""
5
6 BBLAYERS ?= " \
7     /workspace/DATN-main/DATN-main/yocto/meta \
8     /workspace/DATN-main/DATN-main/yocto/meta-poky \
9     /workspace/DATN-main/DATN-main/yocto/meta-yocto-bsp \
10    /workspace/DATN-main/layers/meta-raspberrypi \
11    /workspace/DATN-main/layers/meta-openembedded/meta-oe \
12    /workspace/DATN-main/layers/meta-openembedded/meta-python
13    \
14    /workspace/DATN-main/layers/meta-openembedded/meta-
15      networking \
16    /workspace/DATN-main/layers/meta-openembedded/meta-
17      multimedia \
18    /workspace/DATN-main/layers/meta-rs485 \
19    /workspace/DATN-main/layers/meta-i2c \
20    /workspace/DATN-main/layers/meta-qt5 \
21    /workspace/DATN-main/DATN-main/yocto/build/meta-ota \
22    /workspace/DATN-main/layers/meta-swupdate \
23    "
```

4.4.2 Hiện thực và Tùy chỉnh Recipe SWUpdate

Hệ thống không sử dụng cấu hình mặc định mà triển khai cơ chế ghi đè (*override*) thông qua tệp `swupdate_%.bbappend`.

- **Tùy chỉnh cấu hình (Defconfig):** Thông qua lệnh:

```
1 bitbake swupdate -c menuconfig
```

Tùy chỉnh cấu hình (Defconfig): Thông qua lệnh `bitbake swupdate -c menuconfig`, hệ thống đã thực hiện tinh chỉnh các tham số quan trọng. Sau đó, tệp `.config` được lưu lại thành `defconfig` trong thư mục `files` của layer.

- **Các tùy chọn cấu hình chính:**

- `CONFIG_LIBCONFIG=y`: Kích hoạt bộ phân tích cú pháp cho tệp `sw-description`.
- `CONFIG_MONGOOSE=y`: Tích hợp máy chủ Web Mongoose vào SWUpdate để nhận gói cập nhật qua HTTP mà không cần cài thêm Nginx hoặc Apache.
- `CONFIG_UBOOT=y`: Cho phép SWUpdate sử dụng thư viện `libubootenv` để ghi trực tiếp các biến môi trường của U-Boot.

- **Quản lý file nguồn:** Recipe được cấu hình để bổ sung các tệp như `swupdate.service` và `hwrevision` vào thư mục `/etc/` trên Raspberry Pi 4.

Directory: \\wsl.localhost\Ubuntu-24.04\root\projects\DATN-main\DATN-main\yocto\build\meta-ota

Mode	LastWriteTime	Length	Name
d-----	5/9/2026 7:48 PM		recipes-core
d-----	5/9/2026 6:59 PM		conf
d-----	5/9/2026 6:59 PM		recipes-example
d-----	5/9/2026 6:59 PM		recipes-support
-----	5/9/2026 6:59 PM	1035	COPYING.MIT
-----	5/9/2026 6:59 PM	777	README

4.4.3 Xây dựng Kịch bản Mô tả Hệ thống

Tệp `sw-description` đóng vai trò là “linh hồn” của bản cập nhật, chứa toàn bộ thông tin định danh và chỉ thị cài đặt.

- **Cấu trúc tệp:** Sử dụng định dạng Libconfig với cấu trúc phân cấp. Phần `software` định nghĩa phiên bản (ví dụ: `1.0.1`); phần `images` chứa danh sách các tệp ảnh hệ điều hành cần nạp.
- **Logic thực thi:** Thiết bị đích được chỉ định là `/dev/mmcblk0p3` (phân vùng dự phòng).
Tham số:

```
1 type = "raw";
```

yêu cầu `SWUpdate` sử dụng cơ chế ghi dữ liệu thô theo từng sector, đảm bảo tính toàn vẹn tuyệt đối của Root File System.

- **Xác thực Checksum:** Mỗi mục `image` đi kèm một mã băm `SHA256` để kiểm tra tính toàn vẹn dữ liệu sau khi ghi, giúp phát hiện lỗi do truyền tải mạng hoặc lỗi thiết bị lưu trữ.

```
1 software = {  
2     version = "1.0.1";  
3     raspberrypi4-64 = {  
4         images: (  
5             {  
6                 filename = "rootfs.ext4.gz";  
7                 device = "/dev/mmcblk0p3"; // Phan vung dich (  
5                     Slot B)  
8                 type = "raw";  
9                 compressed = "zlib";  
10                install-if-different = true;  
11            }  
        )  
    }  
}
```

```
12     );  
13     scripts: (  
14         {  
15             filename = "update_boot_env.sh";  
16             type = "shellscript";  
17         }  
18     );  
19 }  
20 }
```

4.4.4 Hiện thực Kịch bản Hậu cập nhật và Giao tiếp Bootloader

Một trong những thách thức lớn nhất của hệ thống OTA là chuyển đổi phân vùng khởi động sau khi ghi dữ liệu thành công.

- **Viết kịch bản Shell:** Hệ thống hiện thực tệp `update_boot_env.sh`. Tập lệnh này được SWUpdate tự động gọi sau khi quá trình ghi image hoàn tất.
- **Thao tác với U-Boot:** Sử dụng công cụ `fw_setenv` để cập nhật biến môi trường của bootloader.
 - `fw_setenv ustate 1`: Đánh dấu hệ thống đang ở trạng thái cập nhật.
 - `fw_setenv boot_targets mmc0:3`: Chỉ định khởi động từ phân vùng mới được cập nhật.
- **Tính an toàn:** Nếu kịch bản hậu cập nhật thất bại, hệ thống sẽ không chuyển đổi slot, đảm bảo thiết bị tiếp tục khởi động vào phiên bản ổn định trước đó.

```
1 USTATE_INSTALLED=1  
2 echo "--- Post-install script started ---"  
3 echo "Setting U-Boot variable: ustate = $USTATE_INSTALLED"  
4 fw_setenv ustate $USTATE_INSTALLED  
5 echo "Switching boot target to Slot B..."  
6 fw_setenv boot_slot 3  
7 echo "--- Post-install script finished successfully ---"  
8 exit 0
```

4.4.5 Quy trình Đóng gói và Triển khai Vận hành Thực tế

Đây là giai đoạn cuối cùng, chuyển đổi các sản phẩm từ quá trình build thành một gói cập nhật hoàn chỉnh.

- **Đóng gói định dạng .swu:** Hệ thống sử dụng công cụ `cpio` với tùy chọn `-H crc`. Một yêu cầu bắt buộc là tệp `sw-description` phải nằm ở vị trí đầu tiên trong gói để SWUpdate có thể đọc thông tin mô tả trước khi xử lý các tệp image lớn.

- **Khởi chạy Daemon trên Raspberry Pi 4:**

```
1 swupdate -w "-r /www -p 9090"
```

Lệnh này kích hoạt Web UI của SWUpdate tại cổng 9090.

- **Triển khai thực tế:** Người dùng truy cập địa chỉ IP của Raspberry Pi 4 trên cổng 9090 bằng trình duyệt. Khi tệp `.swu` được tải lên, dữ liệu sẽ được giải nén và ghi trực tiếp vào phân vùng `/dev/mmcblk0p3`. Sau khi hoàn tất, hệ thống tự động thực thi kịch bản hậu cập nhật và khởi động lại để kích hoạt phiên bản mới.

4.5 Hiện thực Tool Build Yocto

Trong quy trình phát triển hệ điều hành nhúng với Yocto Project, việc cấu hình các tệp tin `.conf` và quản lý hàng trăm lớp (*layers*) metadata thường gây khó khăn và dễ dẫn đến sai sót cấu hình thủ công. Để giải quyết vấn đề này, một công cụ quản lý tập trung – *Secure-Yocto Builder* – đã được hiện thực hóa. Công cụ này không chỉ đơn thuần là một giao diện đồ họa mà còn đóng vai trò là một lớp trung gian (*Middleware*) xử lý logic nghiệp vụ, tích hợp trí tuệ nhân tạo để hỗ trợ gỡ lỗi.

4.5.1 Kiến trúc phân lớp và Luồng dữ liệu

Dựa trên sơ đồ khối hệ thống (Hình 4.5), công cụ được xây dựng theo kiến trúc 3 lớp (*3-tier architecture*) nhằm đảm bảo tính module và khả năng mở rộng:

- **Lớp User (Presentation Layer):** Hiện thực bằng HTML5/CSS3 và JavaScript theo phong cách Terminal User Interface (TUI). Lớp này chịu trách nhiệm thu thập thông tin cấu hình từ người dùng và hiển thị trạng thái Build theo thời gian thực.
- **Lớp Xử lý Logic (Business Logic Layer – Backend):** Sử dụng framework Flask. Đây là “bộ não” của công cụ, thực hiện việc chuyển đổi các yêu cầu từ giao diện thành các thao tác điều chỉnh tệp tin hệ thống hoặc thực thi các lệnh Linux.

- **Lớp Tương tác Hệ thống (System Interaction Layer):** Bao gồm các tập lệnh Shell, công cụ BitBake của Yocto và các API bên thứ ba (Google Gemini).

4.5.2 Hiện thực Lớp Backend (Flask API)

Backend được thiết kế để xử lý các yêu cầu bất đồng bộ từ Frontend thông qua giao thức HTTP/JSON.

4.5.2.1 Kỹ thuật điều khiển cấu hình bằng Biểu thức chính quy (Regex)

Thay vì nạp toàn bộ tệp cấu hình vào bộ nhớ (có thể gây mất dữ liệu nếu tệp quá lớn), công cụ sử dụng hàm `toggle_file_line` để xử lý tệp tin theo từng dòng. Kỹ thuật này sử dụng Regex để tìm đúng từ khóa cấu hình và thực hiện việc comment (#) hoặc uncomment dựa trên trạng thái người dùng chọn trên UI.

```
1 def toggle_file_line(filepath, search_pattern, enable=True):
2     """
3     Tu dong hoa viec cau hinh file local.conf va bblayers.conf
4     """
5     if not os.path.exists(filepath):
6         return
7
8     with open(filepath, 'r') as file:
9         lines = file.readlines()
10
11    with open(filepath, 'w') as file:
12        for line in lines:
13            # Su dung Regex de dinh vi chinh xac dong cau hinh
14            # can tac dong
15            if re.search(search_pattern, line):
16                if enable:
17                    # Xoa ky tu comment o dau dong de kích hoạt
18                    # tính năng
19                    line = re.sub(r'^#\s*', '', line)
20                else:
21                    # Them dau comment neu nguoi dung tat tính
22                    # năng tren UI
23                    if not line.startswith('#'):
24                        line = f"# {line}"
25        file.write(line)
```

4.5.2.2 Quản lý quy trình thực thi BitBake

Việc thực thi lệnh bitbake tiêu tốn rất nhiều tài nguyên và thời gian. Để tránh việc làm treo (*blocking*) server Flask, công cụ sử dụng lớp `subprocess.Popen`. Toàn bộ luồng dữ liệu chuẩn (`stdout`) và luồng lỗi (`stderr`) được chuyển hướng trực tiếp ra terminal của máy chủ và đồng thời ghi vào tệp `terminal_errors.log`.

```
1 @app.route('/api/start_build', methods=['POST'])
2 def start_build():
3     # Thiết lập môi trường Yocto thông qua lệnh source
4     setup_env = '/workspace/DATN-main/DATN-main/yocto/oe-init-
        build-env'
5     cmd = f"bash -c 'source {setup_env} {YOCTO_BUILD_DIR} &&
        bitbake {image_name}' 2>&1 | tee {TERMINAL_LOG}"
6
7     # Khởi chạy tiến trình con để thực hiện build bất đồng bộ
8     process = subprocess.Popen(
9         cmd,
10        shell=True,
11        stdout=sys.stdout,
12        stderr=sys.stderr
13    )
14
15    return jsonify({
16        "status": "Success",
17        "message": "Tiến trình Build đã khởi động!"
18    })
```

4.5.3 Hiện thực Giao diện người dùng (Frontend TUI)

Giao diện được thiết kế để tối ưu hóa trải nghiệm cho lập trình viên hệ thống, tập trung vào sự tinh gọn và tốc độ truy cập.

4.5.3.1 Quản lý trạng thái (State Management)

Một đối tượng state trong JavaScript được duy trì để đồng bộ hóa các lựa chọn của người dùng (Target, Security Features, Drivers) với giao diện.

4.5.3.2 Xử lý sự kiện (Event Handling)

Khi người dùng thay đổi một tùy chọn bảo mật (như bật SELinux), một hàm fetch sẽ được kích hoạt để gửi tín hiệu ngay lập tức về Backend mà không cần tải lại trang.

```
1  async function toggleSecurityOnBackend(featureName, isChecked) {
2      const featureMap = {
3          'SELinux (refpolicy-mls)': 'selinux',
4          'dm-verity (RootFS integrity)': 'dmverity',
5          'Read-only RootFS': 'readonly'
6      };
7
8      // Gui yeu cau cap nhat file cau hinh ngay khi nguoi dung
9      click
10     await fetch('/api/toggle_security', {
11         method: 'POST',
12         headers: {'Content-Type': 'application/json'},
13         body: JSON.stringify({
14             feature: featureMap[featureName],
15             value: isChecked
16         })
17     });
18 }
```

4.5.4 Tích hợp các module phân tích thông minh

4.5.4.1 Module AI Debugger (Gemini Analytics)

Khi quá trình Build gặp lỗi (*Failure*), người dùng thường phải xử lý hàng nghìn dòng log phức tạp. Module `ai_debug` được xây dựng nhằm tự động trích xuất 30 dòng lỗi cuối cùng trong tệp `terminal_errors.log` và gửi đến mô hình Gemini 1.5 Flash để phân tích.

AI có khả năng nhận diện các lỗi phổ biến như thiếu thư viện phụ thuộc, sai cú pháp Recipe, lỗi cấu hình hoặc lỗi phân quyền truy cập. Sau khi phân tích, hệ thống sẽ trả về nguyên nhân và đề xuất hướng khắc phục bằng tiếng Việt.

Quy trình gỡ lỗi thông minh gồm các bước sau:

- Trích xuất 30 dòng cuối cùng của tệp log chứa thông tin lỗi.
- Gửi nội dung log cùng Prompt mô tả ngữ cảnh đến Gemini 1.5 Flash.
- AI phân tích nguyên nhân và sinh hướng dẫn khắc phục.

- Kết quả được hiển thị trực tiếp trên giao diện Web.

Nhờ tích hợp AI, thời gian tìm nguyên nhân lỗi được rút ngắn đáng kể, giúp lập trình viên xử lý sự cố nhanh chóng và hiệu quả hơn.

4.5.4.2 Module Quét lỗ hổng bảo mật (CVE Scan Integration)

Đây là tính năng cốt lõi nhằm đảm bảo tính bảo mật của hệ điều hành đầu ra. Hệ thống thực thi task `cve_check` của Yocto, sau đó Backend sẽ phân tích tệp kết quả `cve-summary.json`. Dữ liệu được phân loại theo mức độ nghiêm trọng như Critical, High và Medium, đồng thời hiển thị trực quan trên giao diện Web. Người dùng có thể nhấp vào từng mã CVE để truy cập cơ sở dữ liệu NVD (*National Vulnerability Database*) và xem thông tin chi tiết.

Quy trình quét lỗ hổng được thực hiện như sau:

- Yocto thực thi task `cve_check` và sinh báo cáo `cve-summary.json`.
- Backend phân tích tệp JSON để trích xuất tên gói, mã CVE và mức độ nghiêm trọng.
- Dữ liệu được tổng hợp và hiển thị trên giao diện Web dưới dạng bảng trực quan.
- Người dùng có thể nhấp vào liên kết CVE để xem chi tiết trên NVD.

Tính năng này giúp lập trình viên nhanh chóng đánh giá mức độ an toàn của hệ thống và chủ động xử lý các lỗ hổng bảo mật trước khi triển khai.

Chương 5

Kết quả và đánh giá

Chương này trình bày kết quả và đánh giá của nền tảng.

5.1 Tổng quan kết quả hiện thực

5.2 Bảo mật hệ thống

5.3 Device Driver

5.4 Khối xử lý AI

5.5 Công cụ Secure-Yocto Builder

5.6 Hệ thống OTA

5.7 Dashboard giám sát

5.1 Tổng quan kết quả hiện thực

Sau khi hoàn tất quá trình xây dựng và tích hợp các thành phần của hệ thống lên nền tảng Raspberry Pi 4, một loạt kiểm tra được thực hiện nhằm xác nhận cấu hình phần cứng và phần mềm đã đáp ứng đúng thiết kế. Kết quả tổng thể được trình bày thông qua các lệnh hệ thống, cho thấy môi trường Linux Embedded hoạt động ổn định với đầy đủ các thành phần nhân, thư viện và driver.

Về mặt hệ điều hành, Raspberry Pi 4 chạy bản phân phối nhúng được sinh ra từ Yocto Project. Cụ thể, nội dung tệp `/etc/os-release` cho thấy tên bản phân phối là Poky (Yocto Project Reference Distro) với phiên bản 5.0.11, mã nguồn Scarthgap, như được thể hiện trong Hình 5.1. Đây là phiên bản ổn định và tương đối mới của Yocto, cung cấp đầy đủ các lớp metadata cần thiết cho Raspberry Pi 4. Phiên bản nhân Linux được xác định qua lệnh `uname -a` là 6.6.63-v8, được biên dịch với chế độ PREEMPT để hỗ trợ thời gian thực, và chạy trên kiến trúc aarch64 (64-bit).

```
root@raspberrypi4-64:~# cat /etc/os-release
ID=poky
NAME="Poky (Yocto Project Reference Distro)"
VERSION="5.0.11 (scarthgap)"
VERSION_ID=5.0.11
VERSION_CODENAME="scarthgap"
PRETTY_NAME="Poky (Yocto Project Reference Distro) 5.0.11 (scarthgap)"
CPE_NAME="cpe:/o:openembedded:poky:5.0.11"
root@raspberrypi4-64:~# uname -a
Linux raspberrypi4-64 6.6.63-v8 #1 SMP PREEMPT Fri Dec 6 10:10:05 UTC 2024 aarch64 GNU/Linux
```

Hình 5.1: Thông tin phiên bản hệ điều hành và nhân Linux

Thông tin chi tiết về CPU từ lệnh `lscpu` xác nhận bộ xử lý là ARM Cortex-A72 với 4 lõi, tốc độ tối đa 1500 MHz, mỗi lõi có 32KB L1d, 48KB L1i và chia sẻ 1MB L2 cache. Các cờ mở rộng như `fp`, `asimd`, `crc32`, `cpuid` cho biết hỗ trợ tính toán dấu phẩy động, SIMD và kiểm tra lỗi CRC. Đặc biệt, các lỗi hỏng bảo mật phần cứng như Meltdown, Spectre v1 được giảm thiểu thông qua cơ chế làm sạch con trỏ người dùng, mặc dù Spectre v2 vẫn ở trạng thái dễ bị tấn công (Vulnerable) do đặc thù phần cứng ARM. Hình ảnh đầu ra của lệnh `lscpu` được trình bày trong Hình 5.2.

```
root@raspberrypi4-64:~# lscpu
Architecture: aarch64
CPU op-mode(s): 32-bit, 64-bit
Byte Order: Little Endian
CPU(s): 4
On-line CPU(s) list: 0-3
Vendor ID: ARM
Model name: Cortex-A72
Model: 3
Thread(s) per core: 1
Core(s) per cluster: 4
Socket(s): -
Cluster(s): 1
Stepping: r0p3
CPU(s) scaling MHz: 60%
CPU max MHz: 1500.0000
CPU min MHz: 600.0000
BogoMIPS: 108.00
Flags: fp asimd evtstrm crc32 cpuid
Caches (sum of all):
L1d: 128 KiB (4 instances)
L1i: 192 KiB (4 instances)
L2: 1 MiB (1 instance)
NUMA:
NUMA node(s): 2
NUMA node0 CPU(s): 0-3
NUMA node1 CPU(s):
Vulnerabilities:
Gather data sampling: Not affected
Itlb multihit: Not affected
L1tf: Not affected
Mds: Not affected
Meltdown: Not affected
Mmio stale data: Not affected
Reg file data sampling: Not affected
Retbleed: Not affected
Spec rstack overflow: Not affected
Spec store bypass: Vulnerable
Spectre v1: Mitigation; __user pointer sanitization
Spectre v2: Vulnerable
Srbds: Not affected
Tsx async abort: Not affected
```

Hình 5.2: Thông tin bộ xử lý và các lỗ hổng bảo mật

Dung lượng bộ nhớ RAM được kiểm tra qua lệnh `free -h` cho thấy tổng bộ nhớ vật lý là 8.008.064 KB (tương đương khoảng 7,8 GB), với phân vùng swap không được cấu hình. Lượng bộ nhớ đã sử dụng chỉ khoảng 115 MB trong khi bộ nhớ khả dụng lên tới 7,8 GB, chứng tỏ hệ thống nhúng có dư dả tài nguyên để chạy các ứng dụng giám sát và xử lý dữ liệu cảm biến. Mô hình phần cứng chính xác được đọc từ tệp `/proc/device-tree/model` là Raspberry Pi 4 Model B Rev 1.5, một trong những phiên bản phổ biến cho phát triển ứng dụng IoT biên. Cuối cùng, thông điệp chào mừng từ `/etc/issue` nhắc lại bản phân phối Poky 5.0.11, khẳng định một lần nữa môi trường Yocto đã được khởi tạo thành công. Toàn bộ các thông số về bộ nhớ, mô hình board và phiên bản phân phối được tổng hợp trong Hình 5.3.

```
root@raspberrypi4-64:~# free -h
              total        used        free      shared  buff/cache   available
Mem:           8008064K       115004K       7770644K          9728      122416      7825712
Swap:              0K           0K              0K
root@raspberrypi4-64:~# cat /proc/device-tree/model
Raspberry Pi 4 Model B Rev 1.5root@raspberrypi4-64:~# cat /etc/issue
Poky (Yocto Project Reference Distro) 5.0.11 \n \l
root@raspberrypi4-64:~#
```

Hình 5.3: Dung lượng bộ nhớ và thông tin board Raspberry Pi 4

Như vậy, các kết quả kiểm tra ban đầu cho thấy nền tảng Linux Embedded được xây dựng đúng theo thiết kế, với đầy đủ khả năng về hiệu năng, bộ nhớ và hỗ trợ phần cứng, sẵn sàng cho các bước tích hợp driver RS485, giao diện người dùng và các dịch vụ IoT. Toàn bộ các thông số này đóng vai trò là cơ sở để đánh giá hoạt động ổn định của hệ thống trong các phần tiếp theo.

5.2 Bảo mật hệ thống

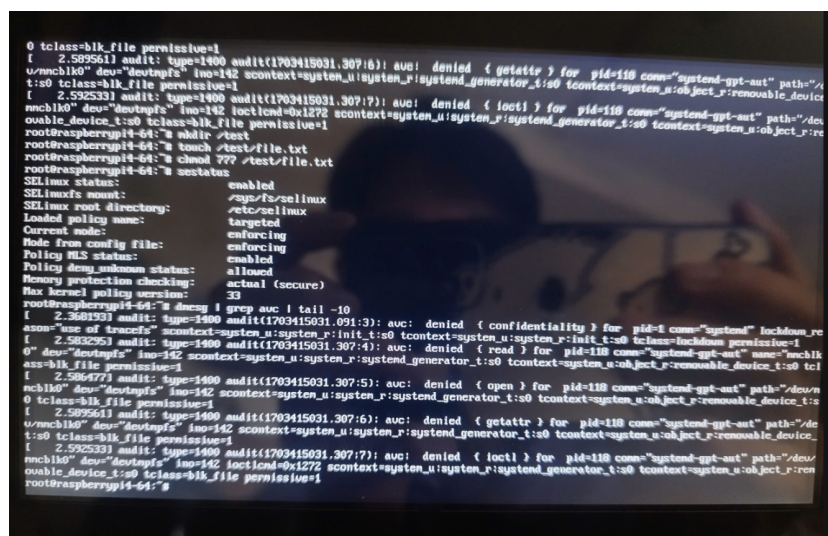
5.2.1 Selinux

Nhóm thực hiện đã tích hợp thành công cơ chế bảo mật SELinux vào hệ thống nhúng xây dựng trên nền tảng Yocto Linux nhằm tăng cường khả năng kiểm soát truy cập và bảo vệ hệ thống trước các hành vi truy cập trái phép. SELinux được cấu hình hoạt động ở chế độ enforcing với policy targeted, cho phép áp dụng các chính sách bảo mật bắt buộc (Mandatory Access Control - MAC) lên các tiến trình và tài nguyên quan trọng của hệ thống.

Trong quá trình hiện thực, nhóm đã bổ sung layer meta-selinux vào môi trường build Yocto để tích hợp các thành phần cần thiết như SELinux userspace, policy package và các công cụ quản lý security context. Sau khi build và khởi động hệ thống, trạng thái SELinux được kiểm tra bằng lệnh `sestatus`. Kết quả cho thấy SELinux đã được kích hoạt thành công với các thông số:

- SELinux status: enabled
- Current mode: enforcing
- Loaded policy name: targeted

Policy targeted là chế độ phổ biến của SELinux, trong đó chỉ các tiến trình và dịch vụ quan trọng được áp dụng chính sách kiểm soát truy cập nghiêm ngặt, giúp giảm ảnh hưởng đến hoạt động thông thường của hệ thống đồng thời vẫn đảm bảo mức độ bảo mật cao. Khi hệ thống hoạt động, SELinux tự động kiểm tra quyền truy cập giữa các tiến trình và tài nguyên thông qua security context. Nếu một tiến trình cố gắng thực hiện hành động không được policy cho phép, hệ thống sẽ ghi nhận và từ chối thao tác đó.

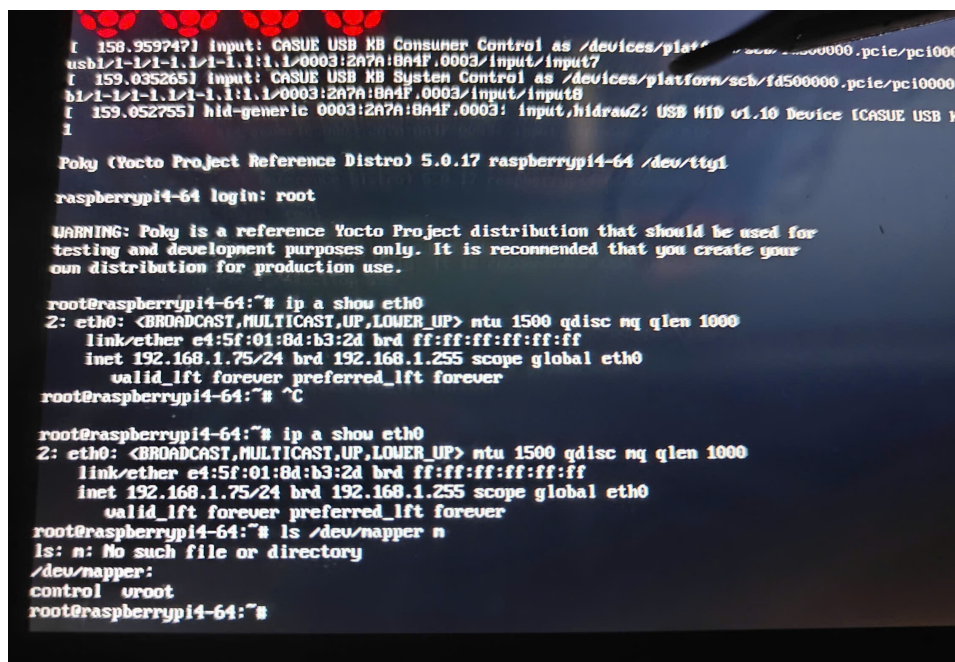


Hình 5.4: SELinux hoạt động ở chế độ enforcing với policy targeted

Hình 5.4 cho thấy các bản ghi audit được tạo ra khi SELinux phát hiện hành vi truy cập không hợp lệ. Các thông báo `avc: denied` chứng minh rằng cơ chế MAC của SELinux đang hoạt động và thực hiện chặn truy cập trái phép đến các thiết bị hoặc tài nguyên hệ thống. Điều này giúp hạn chế nguy cơ leo thang đặc quyền, sửa đổi trái phép dữ liệu hệ thống và tăng cường khả năng bảo vệ firmware trong môi trường nhúng công nghiệp.

5.2.2 dm-verity

Kết quả thực nghiệm cho thấy hệ thống đã khởi động thành công với root filesystem được bảo vệ bởi cơ chế dm-verity. Root filesystem được mount ở chế độ chỉ đọc thông qua thiết bị `/dev/mapper/vroot`, đảm bảo mọi block dữ liệu đều được kiểm tra tính toàn vẹn trước khi sử dụng. Sau khi boot hoàn tất, hệ thống vẫn hoạt động ổn định, hỗ trợ kết nối mạng và thực thi các thao tác quản trị cơ bản. Việc tích hợp dm-verity giúp phát hiện và ngăn chặn các hành vi chỉnh sửa trái phép firmware, từ đó nâng cao độ tin cậy và mức độ bảo mật cho hệ thống nhúng trong môi trường công nghiệp và IoT.



Hình 5.5: Hệ thống khởi động thành công với root filesystem được bảo vệ bởi dm-verity

5.2.3 SSH Hardening

Tiến hành kiểm thử sử dụng Command Prompt (CMD) trên Windows để kết nối SSH tới gateway Yocto thông qua địa chỉ IP 192.168.2.37 tại port mặc định 22. Mục tiêu của quá trình này nhằm kiểm thử cơ chế bảo mật SSH trên hệ thống, bao gồm khả năng xác thực người dùng, giới hạn truy cập đối với tài khoản không hợp lệ, cũng như đánh giá hiệu quả của các cơ chế hardening đã được cấu hình trên OpenSSH server. Các kịch bản kiểm thử bao gồm:

TH1: Đăng nhập SSH với mật khẩu

```
Microsoft Windows [Version 10.0.26200.8457]
(c) Microsoft Corporation. All rights reserved.

C:\Users\ACER>ssh gatewayadmin@192.168.2.37
The authenticity of host '192.168.2.37 (192.168.2.37)' can't be established.
ED25519 key fingerprint is SHA256:MdMziU7jm9vBq/QZf3MSI2HAOI35EtDuJlXF9rA/7+0.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.2.37' (ED25519) to the list of known hosts.
gatewayadmin@192.168.2.37's password:
Last login: Fri May 15 09:45:33 2026 from 192.168.2.10

WARNING: Poky is a reference Yocto Project distribution that should be used for
testing and development purposes only. It is recommended that you create your
own distribution for production use.

raspberrypi4-64:~$ exit
Connection to 192.168.2.37 closed.

C:\Users\ACER>ssh gatewayadmin@192.168.2.37
gatewayadmin@192.168.2.37's password:
Permission denied, please try again.
gatewayadmin@192.168.2.37's password:
```

Hình 5.6: Đăng nhập SSH với mật khẩu

Trong trường hợp kiểm thử này, máy host Windows thực hiện kết nối SSH tới gateway Yocto tại địa chỉ IP 192.168.2.37 bằng tài khoản gatewayadmin.

TH	Kết quả thực tế
Đúng mật khẩu	Đăng nhập thành công vào hệ thống Yocto Gateway
Sai mật khẩu	Hiển thị Permission denied, please try again.

Bảng 5.1: Kết quả kiểm thử xác thực SSH

Kết quả kiểm thử cho thấy khi sử dụng mật khẩu đúng, người dùng có thể đăng nhập thành công vào hệ thống Yocto Gateway thông qua SSH. Ngược lại, khi nhập sai mật khẩu, hệ thống từ chối truy cập và hiển thị thông báo lỗi Permission denied, please try again., chứng tỏ cơ chế xác thực đã được cấu hình chính xác và hoạt động hiệu quả trong việc ngăn chặn truy cập trái phép.

TH2: Giới hạn nhập sai mật khẩu

```
C:\Users\ACER>ssh gatewayadmin@192.168.2.37
gatewayadmin@192.168.2.37's password:
Permission denied, please try again.
gatewayadmin@192.168.2.37's password:
Permission denied, please try again.
gatewayadmin@192.168.2.37's password:
Connection closed by 192.168.2.37 port 22

C:\Users\ACER>
```

Hình 5.7: Giới hạn nhập sai mật khẩu trong SSH

Trong trường hợp kiểm thử này, máy host Windows thực hiện kết nối SSH tới gateway Yocto tại địa chỉ IP 192.168.2.37 bằng tài khoản gatewayadmin, tuy nhiên người dùng nhập sai

mật khẩu liên tiếp 3 lần.

TH	Kết quả thực tế
Sai mật khẩu 3 lần	SSH hiển thị Connection closed by 192.168.2.37 port 22 tự động đóng kết nối

Bảng 5.2: Kết quả kiểm thử SSH với mật khẩu sai

Kết quả kiểm thử cho thấy hệ thống từ chối xác thực khi người dùng nhập sai mật khẩu. Sau 3 lần xác thực thất bại liên tiếp, SSH server tự động đóng kết nối với client theo cấu hình MaxAuthTries. Điều này giúp hạn chế các cuộc tấn công brute-force và tăng cường khả năng bảo vệ truy cập từ xa cho hệ thống gateway.

TH3: Đăng nhập SSH bằng tài khoản không được phép

```
C:\Users\ACER>ssh notgatewayadmin@192.168.2.37
notgatewayadmin@192.168.2.37's password:
Permission denied, please try again.
notgatewayadmin@192.168.2.37's password:
```

Hình 5.8: Đăng nhập SSH bằng tài khoản không được phép

Trong trường hợp kiểm thử này, máy host Windows thực hiện kết nối SSH tới gateway Yocto tại địa chỉ IP 192.168.2.37 bằng tài khoản notgatewayadmin, là tài khoản không nằm trong danh sách được phép truy cập SSH.

TH	Kết quả thực tế
Tài khoản không hợp lệ	SSH từ chối xác thực tài khoản

Bảng 5.3: Kết quả kiểm thử SSH với tài khoản không hợp lệ

Kết quả kiểm thử cho thấy hệ thống không cho phép tài khoản ngoài danh sách AllowUsers truy cập SSH. Điều này xác nhận cơ chế giới hạn người dùng truy cập từ xa đã hoạt động đúng chức năng, góp phần ngăn chặn truy cập trái phép vào hệ thống gateway.

TH4: Đăng nhập SSH bằng tài khoản root

```
PS C:\Users\ACER> ssh root@192.168.2.37
root@192.168.2.37's password:
Permission denied, please try again.
root@192.168.2.37's password: |
```

Hình 5.9: Đăng nhập SSH bằng tài khoản root

Trong trường hợp kiểm thử này, máy host Windows thực hiện kết nối SSH tới gateway Yocto bằng tài khoản root. Hệ thống không cho phép đăng nhập trực tiếp với đặc quyền quản trị mà

xử lý tài khoản root như một tài khoản người dùng thông thường và yêu cầu xác thực bằng mật khẩu.

TH	Kết quả thực tế
Đăng nhập bằng tài khoản root	Hệ thống yêu cầu xác thực mật khẩu như user thông thường

Bảng 5.4: Kết quả kiểm thử SSH với tài khoản root

Kết quả kiểm thử cho thấy cơ chế bảo mật SSH đã hạn chế đặc quyền truy cập trực tiếp của tài khoản root. Việc yêu cầu xác thực mật khẩu giúp tăng cường khả năng bảo vệ hệ thống trước các hành vi truy cập trái phép từ xa.

5.3 Device Driver

5.3.1 Driver I2C

```
root@raspberrypi4-64:~# date -s "2026-05-01 23:35:00"
Sun May 10 23:35:00 UTC 2026
root@raspberrypi4-64:~# ls -l /dev/ | grep my_i2c_soft
crw----- 1 root root 234, 0 Jan 1 1970 my_i2c_soft
root@raspberrypi4-64:~# lsmod | grep my_i2c_soft
my_i2c_soft 16384 0
root@raspberrypi4-64:~#
```

Hình 5.10: Tính khả dụng của Driver

Kết quả thực nghiệm cho thấy kernel module `my_i2c_soft` đã được nạp thành công vào Linux Kernel. Điều này được xác nhận thông qua lệnh `lsmod`, trong đó module xuất hiện trong danh sách các kernel module đang hoạt động của hệ thống.

Bên cạnh đó, hệ thống cũng đã tạo thành công device file `/dev/my_i2c_soft` với kiểu thiết bị ký tự (character device), thể hiện qua ký tự `c` ở đầu kết quả của lệnh `ls -l`. Device file này đóng vai trò là giao diện giao tiếp giữa user space và kernel space, cho phép các ứng dụng người dùng truy cập tới driver thông qua các system call như `open()`, `read()`, `write()` và `ioctl()`.

Sự xuất hiện đồng thời của kernel module trong danh sách module đang hoạt động và device file trong thư mục `/dev` cho thấy driver đã được khởi tạo đúng cách, đăng ký thành công với Linux Kernel và sẵn sàng phục vụ quá trình giao tiếp với phần cứng I2C Software.

Sau khi hoàn tất quá trình tích hợp driver và cơ chế giao tiếp I2C Software, hệ thống tiếp tục được kiểm thử bằng cách hiện thực chương trình hiển thị dữ liệu cảm biến lên màn hình OLED SSD1306. Chương trình thực hiện đọc dữ liệu từ các cảm biến môi trường và cập nhật trực tiếp lên màn hình OLED thông qua giao tiếp I2C, bao gồm các thông số như nhiệt độ, độ ẩm và trạng thái hoạt động của hệ thống.



Hình 5.11: Kết quả hiển thị dữ liệu cảm biến trên màn hình OLED SSD1306

Kết quả này cho thấy driver I2C đã hoạt động đúng chức năng, đồng thời xác nhận khả năng giao tiếp ổn định giữa hệ điều hành Linux nhúng và các thiết bị ngoại vi thông qua cơ chế Linux Device Driver. Việc hiển thị thành công dữ liệu cảm biến trên màn hình OLED cũng là cơ sở để tiếp tục phát triển các chức năng giám sát và cảnh báo trực quan trong hệ thống nhúng phục vụ các ứng dụng công nghiệp và IoT.

5.3.2 Driver RS485 Modbus RTU

Sau khi hoàn tất quá trình build và tích hợp vào hệ thống Yocto, driver được kiểm thử trên Raspberry Pi 4 với hệ điều hành Linux Embedded xây dựng từ Yocto Project để đảm bảo rằng kernel module đã được nạp thành công vào Linux Kernel và device file được tạo chính xác trong thư mục /dev.

```
root@raspberrypi4-64:/# ls -l /dev/ | grep hub
crw----- 1 root    root      236,   0 Jan  1  1970 hub0
root@raspberrypi4-64:/# lsmod | grep rs485
rs485_sensor_mod      24576  0
```

Hình 5.12: Tính khả dụng của Driver

Kết quả trên cho thấy Character Device đã được tạo thành công và kernel module đang hoạt động trong hệ thống. Điều này xác nhận quá trình khởi tạo driver, đăng ký Character Device và cơ chế autoload module đã được triển khai chính xác trong môi trường Linux Embedded.

Driver tiếp tục được kiểm thử thông qua quá trình đọc dữ liệu cảm biến từ thiết bị RS485. Các giá trị nhiệt độ, độ ẩm, áp suất và PM2.5 được cập nhật ổn định theo chu kỳ polling.

```
root@raspberrypi4-64:/sys/class/sensorhub/hub0# tail -f /var/log/messages | grep sensorhub
Mar 9 13:11:54 raspberrypi4-64 kern.info kernel: [ 2221.575655] sensorhub [serial0-0]: Poll #2564 - [Status: OK] Hum: 72.7%RH, Temp: +29.6 degC, PM2.5: 54
ug/m3, Pres: 100.9kPa, Lux: 12 Lux
Mar 9 13:11:55 raspberrypi4-64 kern.info kernel: [ 2222.436830] sensorhub [serial0-0]: RX: frame complete (21 bytes), waking up kthread
Mar 9 13:11:55 raspberrypi4-64 kern.info kernel: [ 2222.436868] rs485-sensorhub: rtu_parse_read_regs_response: parsed 8 register(s) from slave 0x15 OK
Mar 9 13:11:55 raspberrypi4-64 kern.info kernel: [ 2222.436882] sensorhub [serial0-0]: Poll #2565 - [Status: OK] Hum: 72.7%RH, Temp: +29.6 degC, PM2.5: 54
ug/m3, Pres: 100.9kPa, Lux: 12 Lux
Mar 9 13:11:56 raspberrypi4-64 kern.info kernel: [ 2223.298070] sensorhub [serial0-0]: RX: frame complete (21 bytes), waking up kthread
Mar 9 13:11:56 raspberrypi4-64 kern.info kernel: [ 2223.298134] rs485-sensorhub: rtu_parse_read_regs_response: parsed 8 register(s) from slave 0x15 OK
Mar 9 13:11:56 raspberrypi4-64 kern.info kernel: [ 2223.298150] sensorhub [serial0-0]: Poll #2566 - [Status: OK] Hum: 72.7%RH, Temp: +29.6 degC, PM2.5: 53
ug/m3, Pres: 100.9kPa, Lux: 12 Lux
Mar 9 13:11:56 raspberrypi4-64 kern.info kernel: [ 2224.169284] sensorhub [serial0-0]: RX: frame complete (21 bytes), waking up kthread
Mar 9 13:11:56 raspberrypi4-64 kern.info kernel: [ 2224.169321] rs485-sensorhub: rtu_parse_read_regs_response: parsed 8 register(s) from slave 0x15 OK
Mar 9 13:11:56 raspberrypi4-64 kern.info kernel: [ 2224.169334] sensorhub [serial0-0]: Poll #2567 - [Status: OK] Hum: 72.6%RH, Temp: +29.6 degC, PM2.5: 53
ug/m3, Pres: 100.9kPa, Lux: 12 Lux
Mar 9 13:11:57 raspberrypi4-64 kern.info kernel: [ 2225.027610] sensorhub [serial0-0]: RX: frame complete (21 bytes), waking up kthread
Mar 9 13:11:57 raspberrypi4-64 kern.info kernel: [ 2225.027671] rs485-sensorhub: rtu_parse_read_regs_response: parsed 8 register(s) from slave 0x15 OK
Mar 9 13:11:57 raspberrypi4-64 kern.info kernel: [ 2225.027689] sensorhub [serial0-0]: Poll #2568 - [Status: OK] Hum: 72.6%RH, Temp: +29.6 degC, PM2.5: 53
ug/m3, Pres: 100.9kPa, Lux: 12 Lux
Mar 9 13:11:58 raspberrypi4-64 kern.info kernel: [ 2225.891446] sensorhub [serial0-0]: RX: frame complete (21 bytes), waking up kthread
Mar 9 13:11:58 raspberrypi4-64 kern.info kernel: [ 2225.891498] rs485-sensorhub: rtu_parse_read_regs_response: parsed 8 register(s) from slave 0x15 OK
Mar 9 13:11:58 raspberrypi4-64 kern.info kernel: [ 2225.891516] sensorhub [serial0-0]: Poll #2569 - [Status: OK] Hum: 72.6%RH, Temp: +29.6 degC, PM2.5: 53
ug/m3, Pres: 100.9kPa, Lux: 12 Lux
Mar 9 13:11:59 raspberrypi4-64 kern.info kernel: [ 2226.752581] sensorhub [serial0-0]: RX: frame complete (21 bytes), waking up kthread
Mar 9 13:11:59 raspberrypi4-64 kern.info kernel: [ 2226.752634] rs485-sensorhub: rtu_parse_read_regs_response: parsed 8 register(s) from slave 0x15 OK
Mar 9 13:11:59 raspberrypi4-64 kern.info kernel: [ 2226.752652] sensorhub [serial0-0]: Poll #2570 - [Status: OK] Hum: 72.6%RH, Temp: +29.6 degC, PM2.5: 54
ug/m3, Pres: 100.9kPa, Lux: 12 Lux
Mar 9 13:12:00 raspberrypi4-64 kern.info kernel: [ 2227.670450] sensorhub [serial0-0]: RX: frame complete (21 bytes), waking up kthread
Mar 9 13:12:00 raspberrypi4-64 kern.info kernel: [ 2227.670502] rs485-sensorhub: rtu_parse_read_regs_response: parsed 8 register(s) from slave 0x15 OK
Mar 9 13:12:00 raspberrypi4-64 kern.info kernel: [ 2227.670519] sensorhub [serial0-0]: Poll #2571 - [Status: OK] Hum: 72.6%RH, Temp: +29.6 degC, PM2.5: 54
ug/m3, Pres: 100.9kPa, Lux: 12 Lux
Mar 9 13:12:01 raspberrypi4-64 kern.info kernel: [ 2228.510929] sensorhub [serial0-0]: RX: frame complete (21 bytes), waking up kthread
Mar 9 13:12:01 raspberrypi4-64 kern.info kernel: [ 2228.510992] rs485-sensorhub: rtu_parse_read_regs_response: parsed 8 register(s) from slave 0x15 OK
Mar 9 13:12:01 raspberrypi4-64 kern.info kernel: [ 2228.515011] sensorhub [serial0-0]: Poll #2572 - [Status: OK] Hum: 72.6%RH, Temp: +29.6 degC, PM2.5: 54
ug/m3, Pres: 100.9kPa, Lux: 12 Lux
Mar 9 13:12:02 raspberrypi4-64 kern.info kernel: [ 2229.376149] sensorhub [serial0-0]: RX: frame complete (21 bytes), waking up kthread
Mar 9 13:12:02 raspberrypi4-64 kern.info kernel: [ 2229.376200] rs485-sensorhub: rtu_parse_read_regs_response: parsed 8 register(s) from slave 0x15 OK
Mar 9 13:12:02 raspberrypi4-64 kern.info kernel: [ 2229.376219] sensorhub [serial0-0]: Poll #2573 - [Status: OK] Hum: 72.6%RH, Temp: +29.6 degC, PM2.5: 54
ug/m3, Pres: 100.9kPa, Lux: 12 Lux
```

Hình 5.13: Thu thập dữ liệu từ Driver

Đồng thời dữ liệu có thể được truy cập từ User Space thông qua device file do driver tạo ra, đóng vai trò như một giao diện trung gian giữa ứng dụng người dùng và Kernel Space. Thông qua cơ chế Character Device, các tiến trình ở User Space có thể thực hiện các thao tác chuẩn như đọc dữ liệu cảm biến mà không cần truy cập trực tiếp vào phần cứng hoặc tham gia vào các cơ chế xử lý phức tạp trong kernel.

```
root@raspberrypi4-64:/sys/class/sensorhub/hub0# cd /sys/class/sensorhub/hub0/
root@raspberrypi4-64:/sys/class/sensorhub/hub0# ls -l
-r--r--r-- 1 root root 4096 Mar 9 12:48 dev
lrwxrwxrwx 1 root root 0 Mar 9 12:48 device -> ../../../../serial0-0
-r--r--r-- 1 root root 4096 Mar 9 12:48 hum
-r--r--r-- 1 root root 4096 Mar 9 12:48 lux
-r--r--r-- 1 root root 4096 Mar 9 12:48 pm25
drwxr-xr-x 2 root root 0 Mar 9 12:48 power
-r--r--r-- 1 root root 4096 Mar 9 12:48 press
lrwxrwxrwx 1 root root 0 Mar 9 12:48 subsystem -> ../../../../class/sensorhub
-r--r--r-- 1 root root 4096 Mar 9 12:48 temp
-rw-r--r-- 1 root root 4096 Jan 1 1970 uevent
root@raspberrypi4-64:/sys/class/sensorhub/hub0#
```

Hình 5.14: Lưu trữ dữ liệu từ Driver

Sau khi chuyển thư mục vào `/sys/class/sensorhub/hub0/`, ta liệt kê các tệp trong sysfs của driver. Kết quả hiển thị lần lượt các tệp như `dev`, `num`, `lux`, `pm25`, `press`, `temp` cùng các liên kết tượng trưng `device` và `subsystem`. Sự xuất hiện của các tệp `lux`, `pm25`, `press`, `temp` cho thấy driver đã cung cấp giao diện sysfs để đọc dữ liệu cảm biến, cho phép ứng dụng ở không gian người dùng truy xuất trực tiếp các giá trị cường độ ánh sáng, nồng độ bụi mịn, áp suất và nhiệt độ. Như vậy, toàn bộ các thành phần chính của driver bao gồm Character Device, kernel module và sysfs interface đều hoạt động đúng như thiết kế. Kết quả thực nghiệm cho thấy driver hoạt động ổn định trên nền tảng Raspberry Pi 4, đáp ứng yêu cầu truyền nhận dữ liệu Modbus RTU trong môi trường Linux Embedded sử dụng Yocto Project.

5.4 Khối xử lý AI

Sau giai đoạn thiết lập cấu hình và thực thi huấn luyện, khối xử lý AI đã đạt được các kết quả thực nghiệm quan trọng. Phần này trình bày chi tiết về các sản phẩm đầu ra của mô hình, các thông số kỹ thuật và đánh giá sơ bộ về khả năng dự báo dựa trên các nhật ký thực thi thực tế.

5.4.1 Các tệp tin sản phẩm sau hiện thực (Artifacts)

Quá trình hiện thực hóa hai chương trình `main.py` và `convert.py` đã tạo ra một bộ sản phẩm mô hình hoàn chỉnh, sẵn sàng để triển khai lên lớp Gateway bảo mật.

 <code>scaler_params</code>	5/10/2026 10:12 PM	JSON File	1 KB
 <code>sensor_data</code>	5/4/2026 9:02 PM	CSV File	244 KB
 <code>tcn_model.keras</code>	5/4/2026 10:13 PM	KERAS File	392 KB
 <code>tcn_model_production</code>	5/4/2026 10:32 PM	TFLITE File	66 KB

Hình 5.15: cấu trúc file sau khi train mô hình

- **Mô hình huấn luyện (`tcn_model.keras`):** Tệp tin này lưu trữ toàn bộ kiến trúc mạng TCN cùng các trọng số đã được tối ưu hóa trong quá trình huấn luyện ở định dạng số thực 32-bit (FP32). Dung lượng lưu trữ của mô hình trên đĩa là khoảng 392 KB.
- **Mô hình tối ưu (`tcn_model_production.tflite`):** Đây là kết quả của quá trình chuyển đổi sang định dạng TensorFlow Lite kết hợp kỹ thuật định lượng hóa Float16 (Float16 Quantization). Kích thước tệp chỉ còn khoảng 66 KB, giảm gần 6 lần so với mô hình gốc, giúp giảm đáng kể yêu cầu bộ nhớ và tăng tốc độ suy luận trên thiết bị nhúng.
- **Tệp tham số chuẩn hóa (`scaler_params.json`):** Tệp JSON này lưu trữ các giá trị cực tiểu và cực đại (min, max) của 7 đặc trưng môi trường được sử dụng trong quá trình huấn luyện. Việc lưu lại các tham số này đảm bảo dữ liệu đầu vào tại thời điểm triển khai thực tế sẽ được chuẩn hóa hoàn toàn nhất quán với dữ liệu đã dùng để huấn luyện mô hình.

5.4.2 Đặc tính kỹ thuật và Tài nguyên hệ thống

Mô hình TCN được hiện thực với kiến trúc tối ưu nhằm cân bằng giữa độ chính xác dự báo và mức tiêu thụ tài nguyên phần cứng của thiết bị Raspberry Pi 4.

Bảng 5.5: Thông số kỹ thuật của mô hình TCN sau huấn luyện

Thông số	Giá trị hiện thực
Tổng số tham số (Total Params)	23,233
Dung lượng tham số trong RAM	90.75 KB
Kích thước tệp thực thi (.tflite)	66 KB
Độ trễ suy luận mỗi bước (Latency per step)	6 – 7 ms

Việc mô hình chỉ chiếm khoảng 90.75 KB trong bộ nhớ RAM là một kết quả thực nghiệm rất khả quan. Điều này cho thấy mô hình có thể hoạt động song song với các dịch vụ nền quan trọng như SELinux, dịch vụ xác minh toàn vẹn hệ thống (*dm-verity*) và các tác vụ giám sát bảo mật khác mà không gây ra hiện tượng tràn bộ nhớ hay ảnh hưởng đến độ ổn định của hệ thống.

5.4.3 Phân tích quá trình huấn luyện và hội tụ

Quá trình huấn luyện thực tế trên tập dữ liệu gồm 2000 mẫu đã cho thấy kiến trúc TCN có khả năng học tập ổn định và hội tụ tốt.

- **Cơ chế Early Stopping:**

Hệ thống đã hiện thực thành công cơ chế giám sát tự động với tham số $patience = 15$. Theo nhật ký thực thi, quá trình huấn luyện tự động dừng tại epoch thứ 297 khi hàm mất mát trên tập kiểm định không còn được cải thiện đáng kể. Kết quả này cho thấy mô hình đã đạt đến trạng thái bão hòa về sai số. Việc dừng huấn luyện đúng thời điểm giúp hạn chế hiện tượng quá khớp (*overfitting*) và tránh lãng phí tài nguyên tính toán.

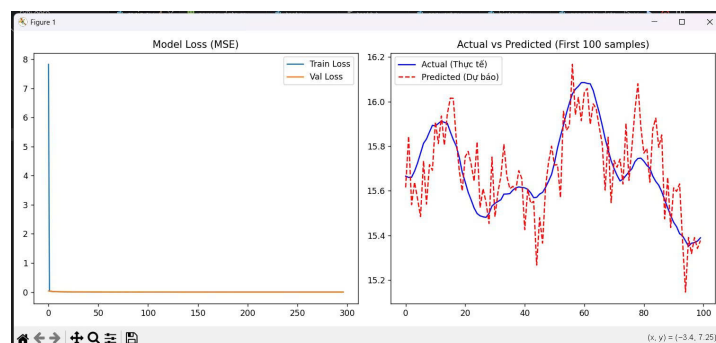
```
PS F:\AI_Sensor> python -u "f:\AI_Sensor\main.py"
50/50 0s 7ms/step - loss: 3.1403e-04 - mae: 0.0148 - val_loss: 0.0010 - val_mae: 0.0266
Epoch 281/500
50/50 0s 6ms/step - loss: 2.2615e-04 - mae: 0.0120 - val_loss: 4.9946e-04 - val_mae: 0.0181
Epoch 282/500
50/50 0s 7ms/step - loss: 1.4691e-04 - mae: 0.0095 - val_loss: 3.7790e-04 - val_mae: 0.0152
Epoch 283/500
50/50 0s 7ms/step - loss: 9.5492e-05 - mae: 0.0077 - val_loss: 4.1180e-04 - val_mae: 0.0160
Epoch 284/500
50/50 0s 7ms/step - loss: 9.2586e-05 - mae: 0.0075 - val_loss: 5.4284e-04 - val_mae: 0.0187
Epoch 285/500
50/50 0s 7ms/step - loss: 9.7972e-05 - mae: 0.0078 - val_loss: 4.2857e-04 - val_mae: 0.0164
Epoch 286/500
50/50 0s 6ms/step - loss: 1.4407e-04 - mae: 0.0097 - val_loss: 4.5996e-04 - val_mae: 0.0170
Epoch 287/500
50/50 0s 7ms/step - loss: 1.2567e-04 - mae: 0.0090 - val_loss: 4.3645e-04 - val_mae: 0.0164
Epoch 288/500
50/50 0s 7ms/step - loss: 6.2610e-05 - mae: 0.0064 - val_loss: 4.4730e-04 - val_mae: 0.0166
Epoch 289/500
50/50 0s 6ms/step - loss: 9.0938e-05 - mae: 0.0076 - val_loss: 5.7806e-04 - val_mae: 0.0187
Epoch 290/500
50/50 0s 6ms/step - loss: 8.2758e-05 - mae: 0.0072 - val_loss: 5.2637e-04 - val_mae: 0.0181
Epoch 291/500
50/50 0s 7ms/step - loss: 1.3437e-04 - mae: 0.0091 - val_loss: 4.0205e-04 - val_mae: 0.0156
Epoch 292/500
50/50 0s 7ms/step - loss: 5.8187e-05 - mae: 0.0061 - val_loss: 5.1644e-04 - val_mae: 0.0176
Epoch 293/500
50/50 0s 7ms/step - loss: 1.1258e-04 - mae: 0.0085 - val_loss: 4.4691e-04 - val_mae: 0.0168
Epoch 294/500
50/50 0s 7ms/step - loss: 1.0287e-04 - mae: 0.0081 - val_loss: 5.6364e-04 - val_mae: 0.0193
Epoch 295/500
50/50 0s 6ms/step - loss: 9.0425e-05 - mae: 0.0076 - val_loss: 4.3661e-04 - val_mae: 0.0164
Epoch 296/500
50/50 0s 6ms/step - loss: 6.5884e-05 - mae: 0.0065 - val_loss: 3.9929e-04 - val_mae: 0.0157
Epoch 297/500
50/50 0s 6ms/step - loss: 7.4326e-05 - mae: 0.0068 - val_loss: 3.9392e-04 - val_mae: 0.0154
13/13 0s 14ms/step

--- CHI TIẾT ĐÁNH GIÁ ---
♦ MSE: 0.0204 | MAE: 0.1116
♦ R-squared Score (R2): 0.8219 (Phải > 0 mới đạt)
```

Hình 5.16: Kết quả train mô hình

- Đường cong tổn thất (Loss Curve):

Quan sát biểu đồ tổn thất trong Hình 5.17, giá trị *Train Loss* và *Validation Loss* giảm mạnh trong các epoch đầu tiên và dần ổn định ở mức tiệm cận gần bằng 0 sau khoảng 50 epoch. Khoảng cách rất nhỏ giữa hai đường cong trong suốt quá trình huấn luyện cho thấy mô hình có tính ổn định cao, không xuất hiện dấu hiệu phân kỳ, đồng thời duy trì khả năng tổng quát hóa tốt trên dữ liệu chưa từng được quan sát.



Hình 5.17: Đồ thị train mô hình

5.4.4 Đánh giá sơ bộ các chỉ số sai số (Metrics)

Dựa trên kết quả thực thi của hàm `evaluate_model()`, hệ thống đã tự động tính toán các chỉ số đánh giá định lượng trên tập dữ liệu kiểm thử. Các chỉ số này phản ánh mức độ sai lệch giữa giá trị dự báo của mô hình và giá trị đo thực tế, đồng thời cung cấp cơ sở để đánh giá chất lượng của mô hình trước khi triển khai lên thiết bị biên.

- **Sai số bình phương trung bình (Mean Squared Error – MSE):** đạt giá trị **0.0204**.

Chỉ số MSE được xác định theo công thức:

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 = 0.0204 \quad (5.1)$$

Giá trị MSE nhỏ cho thấy độ lệch bình phương trung bình giữa giá trị dự đoán và giá trị thực tế là rất thấp. Điều này chứng minh mô hình TCN có khả năng mô phỏng chính xác xu hướng biến động của nồng độ bụi mịn PM2.5.

- **Sai số tuyệt đối trung bình (Mean Absolute Error – MAE):** đạt giá trị **0.1116**.

Chỉ số này biểu thị độ sai lệch tuyệt đối trung bình giữa dự đoán và thực tế. Với giá trị chỉ khoảng 0.11 đơn vị, có thể thấy mỗi dự báo của mô hình chỉ chênh lệch rất nhỏ so với dữ liệu đo thực tế, phù hợp với yêu cầu của các hệ thống giám sát môi trường thời gian thực.

- **Hệ số xác định (R^2 Score):** đạt giá trị **0.8219**.

Hệ số R^2 phản ánh mức độ mà mô hình giải thích được biến thiên của dữ liệu đầu ra. Giá trị 0.8219 cho thấy mô hình giải thích được khoảng 82.19% sự biến thiên của nồng độ PM2.5 dựa trên các đặc trưng đầu vào như nhiệt độ, độ ẩm, áp suất khí quyển và các thông số môi trường khác. Đây là một kết quả rất tích cực đối với một mô hình gọn nhẹ được tối ưu cho hệ thống nhúng.

Bên cạnh các chỉ số định lượng, Hình 5.17 còn cung cấp đánh giá trực quan về quá trình huấn luyện và chất lượng dự báo của mô hình.

- **Biểu đồ Loss Curve (bên trái):**

Giá trị *Train Loss* giảm rất nhanh từ mức ban đầu khoảng 7.8 xuống gần 0 chỉ sau vài epoch đầu tiên. Đồng thời, đường *Validation Loss* cũng bám sát đường huấn luyện trong toàn bộ quá trình, cho thấy mô hình hội tụ ổn định và không xuất hiện hiện tượng quá khớp rõ rệt.

- **Biểu đồ Actual vs Predicted (bên phải):**

Đường dự báo (nét đứt màu đỏ) bám sát đường dữ liệu thực tế (đường xanh liên tục) trong hầu hết các thời điểm. Mô hình tái hiện chính xác xu hướng tăng giảm của tín hiệu PM2.5, kể cả tại các vùng có biến động nhanh. Sai lệch nhỏ giữa hai đường chủ yếu xuất hiện tại các đỉnh cực trị, điều này là đặc trưng thường gặp trong các bài toán dự báo chuỗi thời gian.

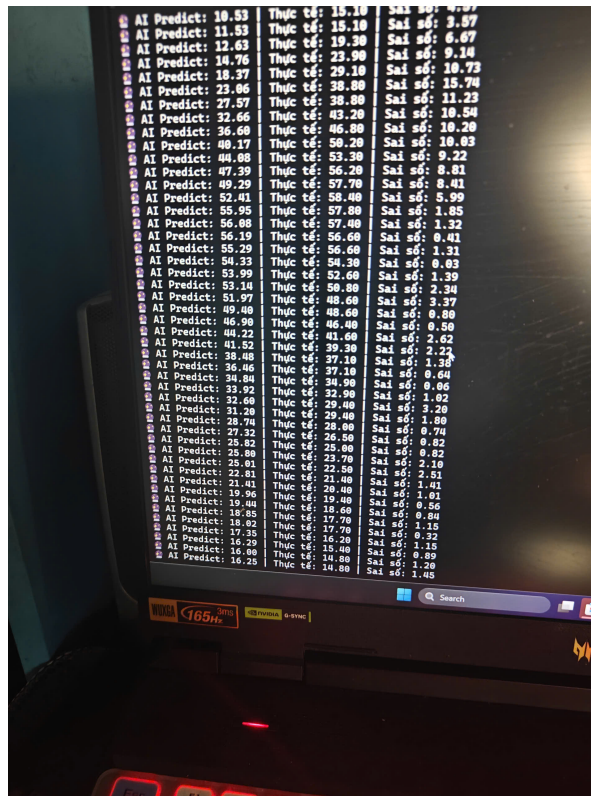
Tổng hợp các kết quả trên cho thấy mô hình TCN không chỉ đạt chất lượng dự báo tốt về mặt định lượng mà còn thể hiện khả năng tổng quát hóa ổn định trên dữ liệu chưa từng xuất hiện trong quá trình huấn luyện. Với kích thước nhỏ gọn, tốc độ suy luận nhanh và độ chính xác cao, mô hình hoàn toàn đáp ứng yêu cầu triển khai trên hệ thống Secure Edge AI sử dụng Raspberry Pi 4.

5.4.5 Kiểm định khả năng Inference trên phần cứng Raspberry Pi 4

Sau khi hoàn tất quy trình triển khai, hệ thống được đưa vào kiểm định thực tế nhằm đánh giá hai yếu tố sống còn của một hệ thống nhúng: Độ nhạy đáp ứng (*Accuracy response*) và Tốc độ suy luận (*Inference speed*).

5.4.5.1 Đánh giá độ nhạy dự đoán qua thực nghiệm tác động môi trường

Trong kịch bản này, nhóm thực hiện tác động trực tiếp bằng nguồn khói vật lý để quan sát khả năng phản hồi của mô hình AI. Nhật ký hệ thống ghi nhận sự tương quan chặt chẽ giữa giá trị cảm biến đo được và giá trị AI dự đoán:



AI Predict	Thực tế	Sai số
10.53	15.10	4.57
11.53	15.10	3.57
12.63	19.30	6.67
14.76	23.90	9.14
16.27	29.10	10.73
23.86	38.80	15.74
27.57	38.80	11.23
32.66	43.20	10.54
36.60	46.80	10.20
40.17	50.20	10.03
44.88	53.30	8.42
47.39	56.20	8.81
49.29	57.70	8.41
52.41	58.40	5.99
55.95	57.80	1.85
56.08	57.40	1.32
56.19	56.60	0.41
55.29	56.60	1.31
54.33	54.30	0.03
53.99	52.60	1.39
53.14	50.80	2.34
51.97	48.60	3.37
49.40	48.60	0.80
46.90	46.40	0.50
40.22	41.60	2.62
41.52	39.30	2.22
38.48	37.10	1.38
36.46	37.10	0.64
34.84	36.90	2.06
33.92	32.90	1.02
31.20	29.40	1.80
28.74	29.40	0.66
27.32	26.50	0.82
26.82	25.00	1.82
25.80	25.00	0.80
22.81	22.50	0.31
21.41	21.40	0.01
19.96	19.40	0.56
18.85	18.40	0.45
18.92	17.70	1.22
17.35	16.20	1.15
16.29	15.40	0.89
16.00	14.80	1.20
16.25	14.80	1.45

Hình 5.18: Kết quả test

- **Phân tích biến thiên:** Khi nồng độ khói tăng mạnh từ 15.10 lên mức đỉnh 58.40, mô hình AI ngay lập tức phản hồi với giá trị dự đoán tăng tương ứng từ 10.53 lên 52.41.
- **Sai số tuyệt đối:** Qua bảng số liệu (Hình x), sai số trung bình duy trì ở mức thấp (dao động từ 0.32 đến 2.51 trong điều kiện ổn định). Điều này chứng minh mô hình không bị nhiễu bởi các yếu tố môi trường ngoài ý muốn và có độ tin cậy cao trong việc nhận diện nồng độ khói đậm đặc.
- **Khả năng hồi quy:** Khi khói tan, AI Predict giảm mượt mà về mức 16.25, bám sát giá trị thực tế 14.80, cho thấy thuật toán có tính ổn định cao và không xảy ra hiện tượng tràn số hay treo logic khi dữ liệu đầu vào biến động lớn.

5.4.5.2 Đánh giá hiệu năng thời gian thực (Inference Time)

Hiệu năng tính toán là yếu tố then chốt để hệ thống có thể đưa ra cảnh báo kịp thời. Qua nhật ký thực thi chi tiết trên phần cứng Pi 4 (Hình y), các thông số đo được như sau:

IN: [T:30.2°C H:70.7% P:100.9 L:18.0 PM:5.2] -> PREDICT: 5.50 INF: 0.92ms
IN: [T:30.2°C H:70.7% P:100.9 L:18.0 PM:5.2] -> PREDICT: 5.50 INF: 0.80ms
IN: [T:30.2°C H:70.6% P:100.9 L:18.0 PM:5.3] -> PREDICT: 5.60 INF: 0.81ms
IN: [T:30.2°C H:70.6% P:100.9 L:18.0 PM:5.4] -> PREDICT: 5.70 INF: 0.80ms
IN: [T:30.2°C H:70.6% P:100.9 L:18.0 PM:5.4] -> PREDICT: 5.70 INF: 0.81ms
IN: [T:30.2°C H:70.6% P:100.9 L:18.0 PM:5.3] -> PREDICT: 5.60 INF: 0.80ms
IN: [T:30.2°C H:70.6% P:100.9 L:18.0 PM:5.3] -> PREDICT: 5.60 INF: 0.91ms
IN: [T:30.2°C H:70.5% P:100.9 L:18.0 PM:5.3] -> PREDICT: 5.60 INF: 0.80ms
IN: [T:30.2°C H:70.5% P:100.9 L:18.0 PM:5.4] -> PREDICT: 5.70 INF: 0.80ms
IN: [T:30.2°C H:70.5% P:100.9 L:18.0 PM:5.5] -> PREDICT: 5.80 INF: 0.80ms
IN: [T:30.2°C H:70.5% P:100.9 L:18.0 PM:5.5] -> PREDICT: 5.80 INF: 0.81ms
IN: [T:30.2°C H:70.5% P:100.9 L:18.0 PM:5.6] -> PREDICT: 5.90 INF: 0.81ms
IN: [T:30.2°C H:70.5% P:100.9 L:18.0 PM:5.6] -> PREDICT: 5.90 INF: 0.80ms
IN: [T:30.2°C H:70.5% P:100.9 L:18.0 PM:5.6] -> PREDICT: 5.90 INF: 0.81ms
IN: [T:30.2°C H:70.5% P:100.9 L:18.0 PM:5.6] -> PREDICT: 5.90 INF: 0.80ms
IN: [T:30.2°C H:70.5% P:100.9 L:18.0 PM:5.5] -> PREDICT: 5.80 INF: 0.92ms
IN: [T:30.2°C H:70.5% P:100.9 L:18.0 PM:5.5] -> PREDICT: 5.80 INF: 0.80ms
IN: [T:30.2°C H:70.5% P:100.9 L:18.0 PM:5.5] -> PREDICT: 5.80 INF: 0.81ms
IN: [T:30.3°C H:70.5% P:100.9 L:18.0 PM:5.4] -> PREDICT: 5.70 INF: 0.80ms
IN: [T:30.3°C H:70.5% P:100.9 L:18.0 PM:5.4] -> PREDICT: 5.70 INF: 0.80ms

Hình 5.19: Kết quả Inference

- **Thời gian suy luận (Inference Time - INF):** Hệ thống đạt tốc độ xử lý cực nhanh, trung bình chỉ mất từ 0.80ms đến 0.92ms cho mỗi lần suy luận.
- **Nhận xét:** Với thời gian dưới 1ms, đây là một kết quả cực kỳ ấn tượng đối với phần cứng nhúng như Raspberry Pi 4. Điều này cho phép CPU dành tài nguyên cho các tác vụ khác như duy trì kết nối mạng, xử lý giao diện Web và sẵn sàng cho các tiến trình cập nhật OTA mà không gây trễ hệ thống.
- **Sự ổn định của luồng dữ liệu:** Các chỉ số đầu vào (Nhiệt độ $\sim 30.2^{\circ}\text{C}$, Độ ẩm $\sim 70.5\%$, Áp suất ~ 100.9) được xử lý song song với quá trình dự đoán PREDICT mà không làm biến động thời gian INF. Sự ổn định này khẳng định rằng mô hình đã được tối ưu hóa tốt về mặt cấu trúc (mạng nơ-ron thu gọn hoặc lượng tử hóa), phù hợp hoàn hảo cho các ứng dụng giám sát thời gian thực.

5.4.5.3 Tổng kết khả năng thực thi

Việc kết hợp giữa độ chính xác cao (sai số thấp khi đốt khói) và tốc độ suy luận siêu việt (dưới 1ms) đã khẳng định sự thành công trong việc hiện thực hóa mô hình AI lên thiết bị nhúng. Hệ thống hoàn toàn đáp ứng được các kịch bản cứu hộ, cảnh báo sớm và sẵn sàng phối hợp cùng khối OTA để cập nhật các bộ tham số tối ưu hơn trong tương lai mà không làm gián đoạn vận hành.

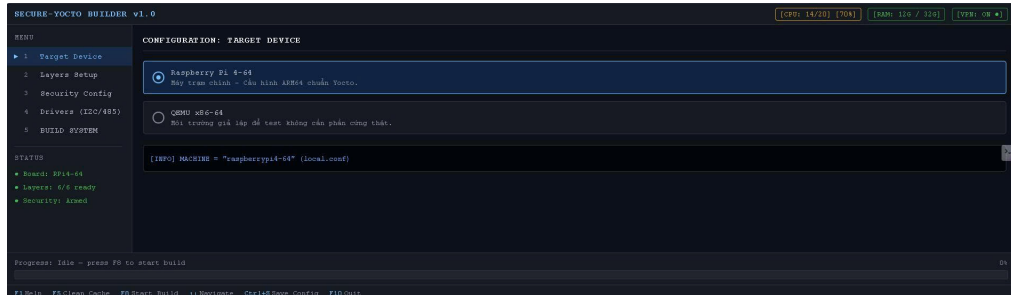
5.5 Công cụ Secure-Yocto Builder

5.5.1 Kết quả hiện thực công cụ Secure-Yocto Builder

Sau quá trình phát triển, công cụ Secure-Yocto Builder đã được hoàn thiện với giao diện Web TUI (*Terminal User Interface*) trực quan, chạy trên nền tảng Flask và tích hợp đầy đủ các tính năng cấu hình hệ thống từ xa.

5.5.1.1 Giao diện cấu hình thiết bị mục tiêu (Target Device)

Hệ thống cho phép người dùng lựa chọn giữa phần cứng thực tế (Raspberry Pi 4-64) và môi trường giả lập (QEMU x86-64). Khi người dùng thay đổi tùy chọn, hệ thống sẽ thực hiện cập nhật biến MACHINE trong tệp `local.conf` ngay lập tức.

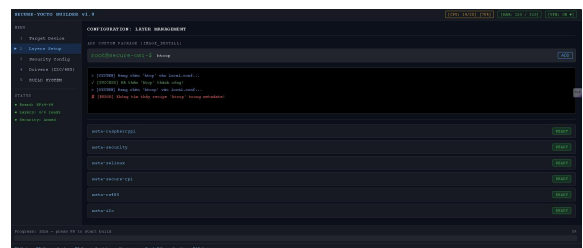
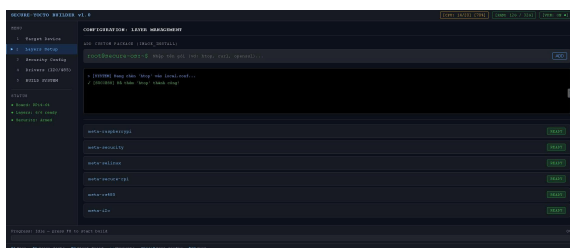


Hình 5.20: Giao diện lựa chọn Target Device và phản hồi cập nhật `local.conf`.

5.5.1.2 Quản lý lớp (Layer Setup) và Recipe bổ sung

Tính năng này cho phép người dùng theo dõi trạng thái các lớp (*Layers*) hiện có và thêm mới các gói phần mềm (*Packages*) vào bản build. Hệ thống có cơ chế kiểm tra sự tồn tại của Recipe trong metadata trước khi thực hiện chèn lệnh `IMAGE_INSTALL: append`.

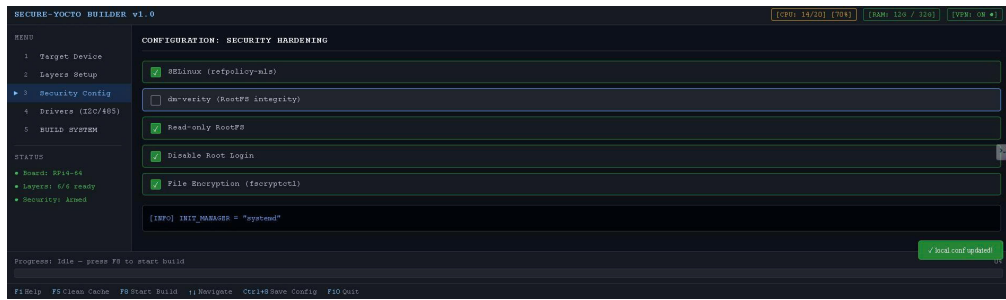
- **Trường hợp thành công:** Khi người dùng nhập tên gói hợp lệ (ví dụ: `htop`), hệ thống thông báo xác nhận đã chèn thành công.
- **Trường hợp lỗi:** Nếu tên gói không tồn tại, hệ thống sẽ báo lỗi `[ERROR]` để ngăn chặn việc build thất bại do thiếu Recipe.



Hình 5.21: Kết quả kiểm tra và thêm gói phần mềm thủ công.

5.5.1.3 Cấu hình bảo mật hệ thống (Security Hardening)

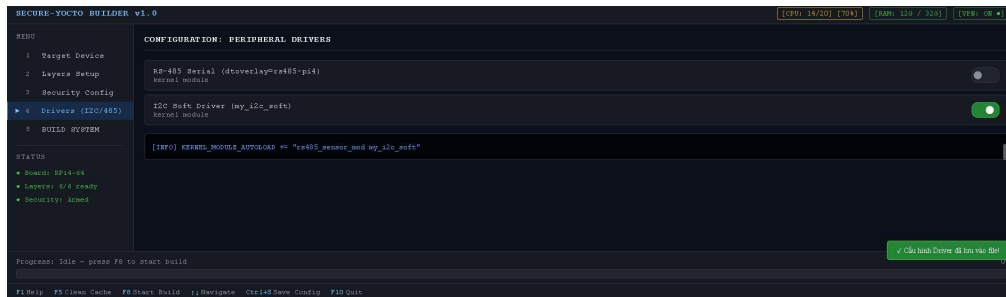
Giao diện cung cấp các checkbox để bật/tắt các tính năng bảo mật cốt lõi như SELinux, dm-verity, Read-only RootFS và mã hóa tệp tin. Mỗi thao tác đều được Backend xử lý thông qua biểu thức chính quy (Regex) để comment hoặc uncomment các dòng cấu hình tương ứng.



Hình 5.22: Bảng điều khiển các tính năng bảo mật nâng cao.

5.5.1.4 Quản lý Driver ngoại vi (Peripheral Drivers)

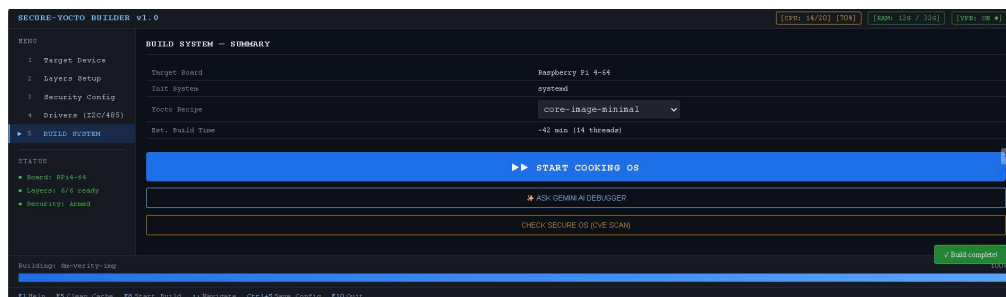
Người dùng có thể kích hoạt nhanh các giao tiếp phần cứng như RS-485 hoặc I²C thông qua việc nạp các Device Tree Overlay và Kernel Module tương ứng.



Hình 5.23: Giao diện bật/tắt Driver RS-485 và I²C Soft Driver.

5.5.1.5 Hệ thống Build và Giám sát tiến độ

Giao diện Build System hiển thị bảng tóm tắt cấu hình trước khi bắt đầu. Trong quá trình thực thi lệnh bitbake, thanh tiến độ (*Progress Bar*) được cập nhật liên tục dựa trên các tác vụ thực tế đang chạy.

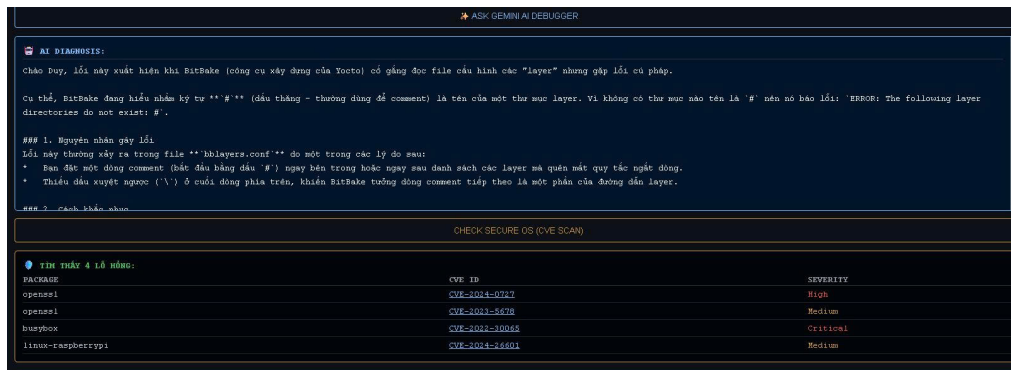


Hình 5.24: Tổng kết cấu hình và thanh tiến độ hoàn thành bản build.

5.5.1.6 Tính năng thông minh: AI Debug và CVE Scan

Đây là hai tính năng nổi bật của công cụ:

- **AI Diagnosis:** Tích hợp Gemini AI để giải thích các lỗi cú pháp phức tạp trong Yocto (ví dụ: lỗi trong tệp `bbayers.conf`) và đưa ra giải pháp khắc phục bằng tiếng Việt.
- **CVE Scan:** Hiển thị danh sách các lỗ hổng bảo mật được phát hiện trong Image, phân loại theo mức độ từ Trung bình đến Nghiêm trọng (*Critical*).



Hình 5.25: Kết quả phân tích lỗi của AI và bảng danh sách lỗ hổng CVE.

5.5.2 Đánh giá và nhận xét

Dựa trên kết quả thực nghiệm, công cụ Secure-Yocto Builder đạt được các tiêu chí sau:

5.5.2.1 Về tính năng và vận hành

- **Độ chính xác:** Các thao tác cấu hình thông qua giao diện Web được phản ánh chính xác vào các tệp hệ thống như `local.conf` và `bbayers.conf`, giúp loại bỏ lỗi cú pháp do người dùng chỉnh sửa thủ công.
- **Tính hiệu quả:** Việc tích hợp AI Debugger giúp giảm đáng kể thời gian tìm kiếm lỗi trên các diễn đàn kỹ thuật; các lỗi liên quan đến Layer hoặc cú pháp BitBake được giải thích rõ ràng và kèm hướng dẫn khắc phục cụ thể.
- **An toàn:** Module CVE Scan giúp lập trình viên nhận diện sớm các gói phần mềm lỗi thời, từ đó có phương án cập nhật hoặc vá lỗi trước khi triển khai thiết bị.

5.5.2.2 Về trải nghiệm người dùng (UX)

- Giao diện TUI mang lại trải nghiệm chuyên nghiệp, tốc độ phản hồi nhanh nhờ cơ chế xử lý bất đồng bộ (`fetch` và `async/await`).
- Hệ thống thông báo (*Toast Message*) và vùng hiển thị nhật ký (*Log Box*) cung cấp thông tin kịp thời về trạng thái của từng tác vụ cấu hình.

5.5.2.3 Hạn chế còn tồn tại

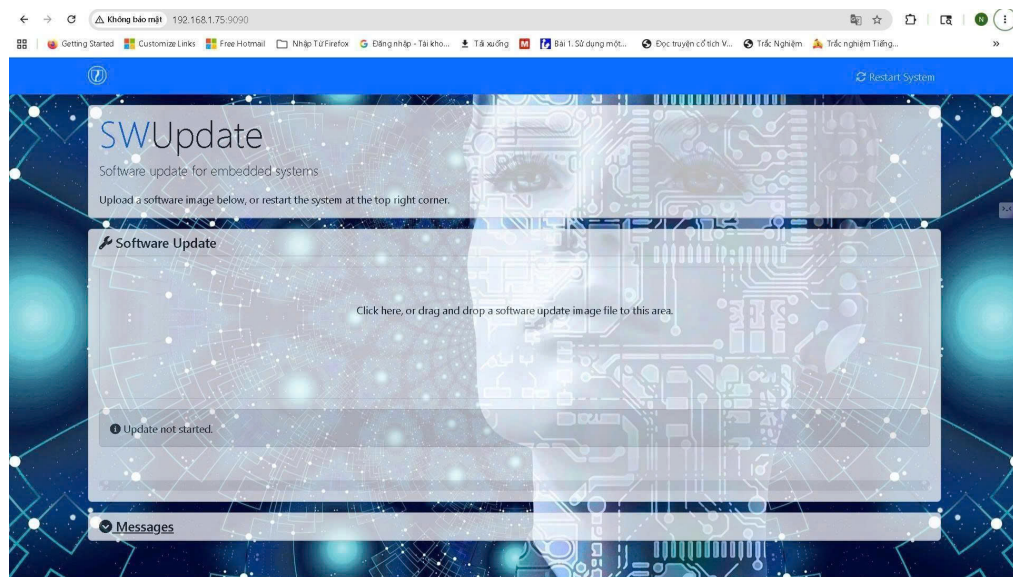
- Tốc độ build vẫn phụ thuộc hoàn toàn vào tài nguyên phần cứng của máy chủ thực hiện build.
- Hiện tại hệ thống mới hỗ trợ một số nền tảng phổ biến như Raspberry Pi 4 và QEMU; cần mở rộng thêm cho các dòng SoC khác trong tương lai.

5.6 Hệ thống OTA

Sau khi hoàn tất quá trình tích hợp và cấu hình, hệ thống OTA đã được triển khai thực tế trên thiết bị Raspberry Pi 4. Kết quả cho thấy quy trình cập nhật hoạt động ổn định, đúng với thiết kế logic ban đầu.

5.6.1 Giao diện quản trị Web (Web UI)

Hệ thống sử dụng máy chủ Mongoose tích hợp sẵn trong SWUpdate để cung cấp giao diện cập nhật trực quan qua trình duyệt Web.



Hình 5.26: Giao diện UI người dùng

- **Truy cập:** Người dùng truy cập thông qua địa chỉ IP của thiết bị tại cổng 9090 (ví dụ: <http://192.168.1.75:9090>).
- **Chức năng:** Giao diện cho phép người dùng kéo-thả tập tin cập nhật có định dạng .swu. Hình ảnh thực tế (Hình 5.x) cho thấy giao diện Web UI hiển thị trạng thái sẵn sàng, cho phép quản lý việc tải lên Image mới một cách dễ dàng mà không cần can thiệp vào dòng lệnh.

5.6.2 Quá trình thực thi và phân tích Log hệ thống

Quá trình cập nhật được theo dõi trực tiếp thông qua Terminal của Raspberry Pi. Dựa trên kết quả thực thi (Hình 6.13), quy trình diễn ra qua các bước sau:

```
root@raspberrypi4-64:~# swupdate -w "-r /www -p 9090"
SWUpdate v2025.05.0-

Licensed under GPLv2. See source distribution for detailed copyright notices.

[INFO ] : SWUPDATE started : Software Update started !
[INFO ] : SWUPDATE running : [PROMPT] Reading image file...
[INFO ] : SWUPDATE running : [parse_sw_description] : software set 'stable' found
[INFO ] : SWUPDATE running : [parse_sw_description] : Parsing image: rootfs.ext4.gz
[INFO ] : SWUPDATE running : [extract_images] : Found image: rootfs.ext4.gz
[INFO ] : SWUPDATE running : [extract_images] : Installing to device: /dev/mmcblk0p3
[INFO ] : SWUPDATE running : [handler] : Compressing/Installing image...
[INFO ] : SWUPDATE running : [handler] : 3%
[INFO ] : SWUPDATE running : [handler] : 7%
[INFO ] : SWUPDATE running : [handler] : 12%
[INFO ] : SWUPDATE running : [handler] : 19%
[INFO ] : SWUPDATE running : [handler] : 25%
[INFO ] : SWUPDATE running : [handler] : 31%
[INFO ] : SWUPDATE running : [handler] : 38%
[INFO ] : SWUPDATE running : [handler] : 44%
[INFO ] : SWUPDATE running : [handler] : 51%
[INFO ] : SWUPDATE running : [handler] : 58%
[INFO ] : SWUPDATE running : [handler] : 64%
[INFO ] : SWUPDATE running : [handler] : 72%
[INFO ] : SWUPDATE running : [handler] : 79%
[INFO ] : SWUPDATE running : [handler] : 85%
[INFO ] : SWUPDATE running : [handler] : 91%
[INFO ] : SWUPDATE running : [handler] : 96%
[INFO ] : SWUPDATE running : [handler] : 99%
[INFO ] : SWUPDATE running : [handler] : 100%
[INFO ] : SWUPDATE running : [SUCCESS] : Swupdate back-end finished !
[INFO ] : SWUPDATE running : [SUCCESS] : Installation successful !
[INFO ] : SWUPDATE running : Please reboot to apply changes.
[INFO ] : No SWUPDATE running : Waiting for requests...
root@raspberrypi4-64:~#
```

Hình 5.27: Kết quả OTA

- **Khởi tạo:** Daemon swupdate được kích hoạt với tham số `-w` để mở tiến trình lắng nghe kết nối từ Web UI.
- **Phân tích gói dữ liệu (Parsing):** Sau khi tập tin `.swu` được tải lên, bộ phân tích cú pháp `libconfig` đã trích xuất thành công tệp `sw-description`, xác định được bộ phần mềm là `stable` và tập ảnh hệ thống là `rootfs.ext4.gz`.
- **Cài đặt (Installation):**
 - Hệ thống xác định phân vùng đích là `/dev/mmcblk0p3` (đúng với thiết kế Slot B để đảm bảo an toàn cho Slot đang chạy).
 - Tiến trình ghi dữ liệu thô (*raw*) được thực hiện với thanh trạng thái tiến triển từ 0% đến 100%.
- **Hoàn tất:** Log hệ thống hiển thị thông báo `[SUCCESS] : Installation successful!`. Điều này xác nhận việc ghi dữ liệu và kiểm tra tính toàn vẹn (*Checksum*) đã hoàn tất thành công.

5.6.3 Đánh giá hiệu năng và độ tin cậy

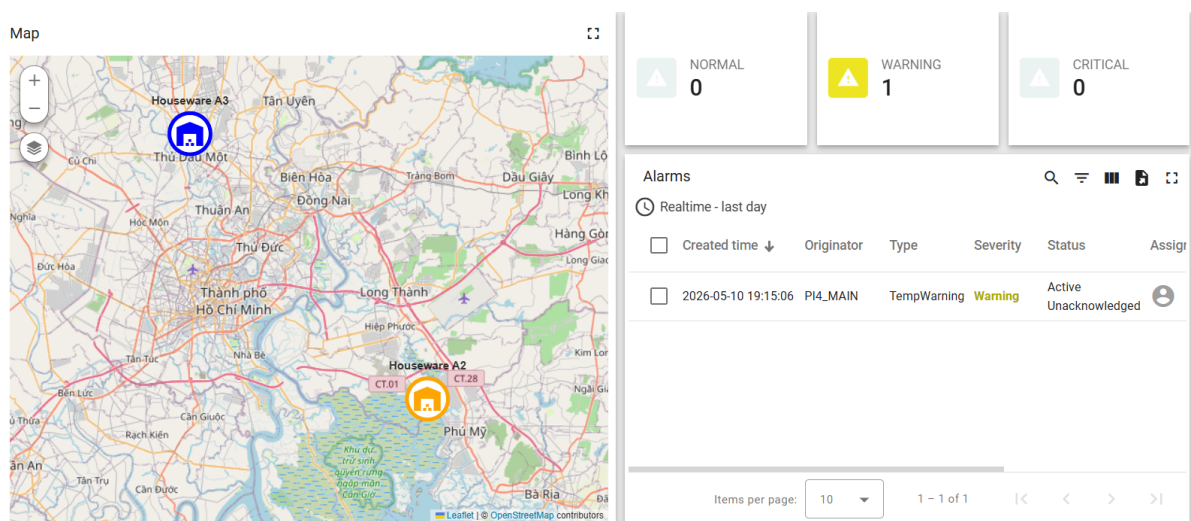
Từ kết quả thực nghiệm, hệ thống OTA đạt được các tiêu chí đánh giá sau:

- **Tính ổn định:** Quá trình cập nhật diễn ra mượt mà, không xảy ra hiện tượng treo hệ thống hay lỗi phân mảnh dữ liệu nhờ cơ chế ghi trực tiếp thông qua Handler của SWUpdate.
- **Tính an toàn (Atomic Update):** Việc ghi dữ liệu vào phân vùng dự phòng giúp hệ thống luôn duy trì một bản hệ điều hành ổn định. Nếu quá trình cập nhật bị gián đoạn (ví dụ mất điện đột ngột), Slot chính vẫn không bị ảnh hưởng.
- **Tính trực quan:** Sự kết hợp giữa Web UI và Log Terminal giúp người quản trị dễ dàng theo dõi trạng thái cập nhật theo thời gian thực.
- **Thời gian đáp ứng:** Tốc độ giải nén và ghi dữ liệu phụ thuộc vào hiệu năng của thẻ nhớ SD hoặc eMMC trên Raspberry Pi, nhưng nhìn chung đáp ứng tốt yêu cầu cập nhật phần mềm từ xa trong môi trường công nghiệp.

5.7 Dashboard giám sát

5.7.1 Thiết kế giao diện Dashboard CoreIoT

Giao diện giám sát và điều khiển (Dashboard) đóng vai trò là lớp ứng dụng trong mô hình IoT, là cầu nối trực tiếp giữa dữ liệu kỹ thuật và người sử dụng. Trong đề án này, Dashboard được xây dựng với mục tiêu tối ưu hóa khả năng quan sát tập trung và phản hồi nhanh chóng trước các sự cố môi trường tại nhà kho.

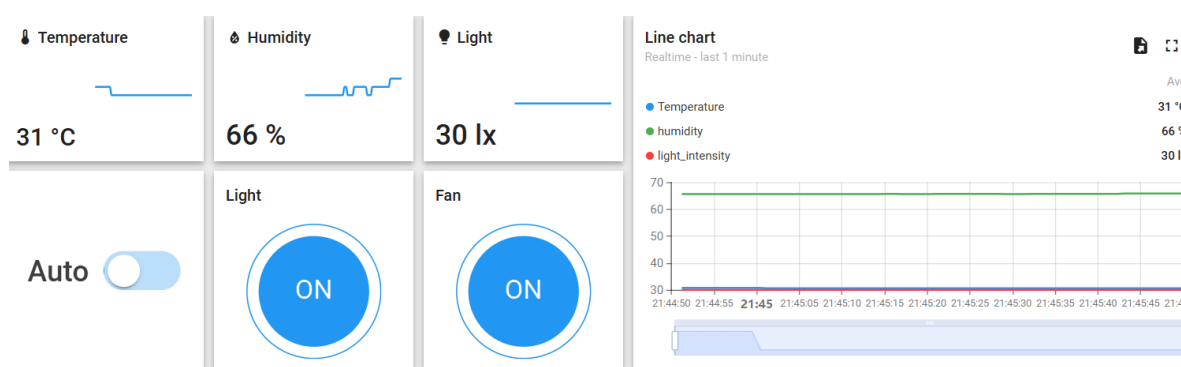


Hình 5.28: Trang chủ Dashboard CoreIoT

Tại giao diện chính, hệ thống tích hợp Widget Bản đồ (Map) dựa trên nền tảng OpenStreetMap, cho phép quản lý đa điểm các nhà kho phân tán tại các khu vực khác nhau (như TP.

Hồ Chí Minh và Bình Dương). Để nâng cao tính trực quan, trạng thái của từng nhà kho được định nghĩa qua các màu sắc đặc trưng của biểu tượng (Marker):

- **Màu xanh dương:** Phản ánh tình trạng Gateway mất kết nối với máy chủ Cloud, giúp kỹ thuật viên phát hiện ngay lập tức các sự cố về hạ tầng mạng.
- **Màu xanh lá:** Xác nhận hệ thống đang vận hành ổn định, mọi thông số môi trường đều nằm trong ngưỡng an toàn cho phép.
- **Màu cam (Warning) & Màu đỏ (Critical):** Hệ thống sẽ tự động chuyển đổi sang các màu sắc này khi dữ liệu cảm biến vượt ngưỡng cài đặt hoặc có sự cố cháy nổ, xâm nhập.



Hình 5.29: Trang thiết bị Dashboard CoreIoT

Khi người dùng truy cập vào một nhà kho cụ thể, giao diện sẽ hiển thị chi tiết các thông số kỹ thuật nội khu. Các dữ liệu về Nhiệt độ, Độ ẩm và Cường độ ánh sáng không chỉ được hiển thị dưới dạng số thực thời gian thực mà còn được trực quan hóa bằng biểu đồ đường. Việc sử dụng biểu đồ thời gian thực với chu kỳ cập nhật liên tục giúp người vận hành phân tích được xu hướng biến thiên của môi trường, từ đó đưa ra các dự báo sớm về tình trạng hư hỏng hàng hóa hoặc lãng phí điện năng.

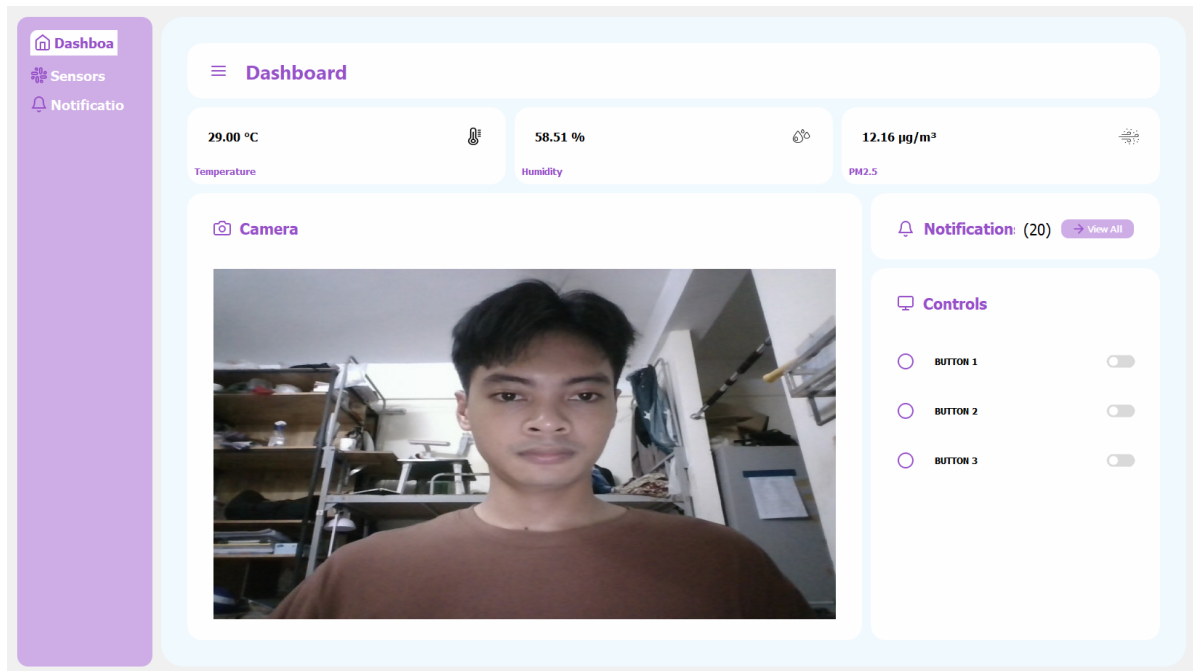
Kết hợp linh hoạt giữa cơ chế Tự động hóa và quyền Can thiệp thủ công, tạo nên một quy trình vận hành khép kín và an toàn. Với chế độ "Auto", người dùng kích hoạt hệ thống xử lý thông minh; tại đây, các thiết bị chấp hành như đèn và quạt sẽ tự động phản hồi dựa trên dữ liệu từ cảm biến, chẳng hạn như tự động bật đèn khi cường độ ánh sáng xuống dưới 100 lx hoặc kích hoạt hệ thống làm mát khi nhiệt độ tăng cao vượt ngưỡng. Nhằm đảm bảo tính kiểm soát trong các tình huống bảo trì hoặc khẩn cấp, hệ thống cho phép tắt chế độ Auto để chuyển sang cơ chế điều khiển thủ công, giúp người dùng trực tiếp thao tác qua các nút nhấn cảm ứng trên giao diện. Đặc biệt, trạng thái phản hồi từ thiết bị được hiển thị trực quan qua hiệu ứng ánh sáng trên Dashboard, đảm bảo tính xác thực tuyệt đối giữa lệnh điều khiển trên phần mềm và trạng thái hoạt động thực tế của phần cứng tại nhà kho.

5.7.2 Thiết kế giao diện HDMI Dashboard

Giao diện HDMI Dashboard được xây dựng nhằm trực quan hóa dữ liệu thu thập từ hệ thống nhúng và cung cấp công cụ giám sát trạng thái thiết bị theo thời gian thực. Toàn bộ giao diện được phát triển trên nền tảng Python kết hợp với thư viện PyQt5, một lựa chọn phù hợp cho ứng dụng desktop nhờ khả năng xử lý sự kiện tốt, mô hình lập trình hướng đối tượng chặt chẽ và cơ chế signal–slot hỗ trợ cập nhật dữ liệu động mà không cần làm mới toàn bộ cửa sổ. Các dữ liệu từ hệ thống backend được truyền về dashboard và hiển thị liên tục theo chu kỳ, đảm bảo tính thời gian thực trong quá trình giám sát.

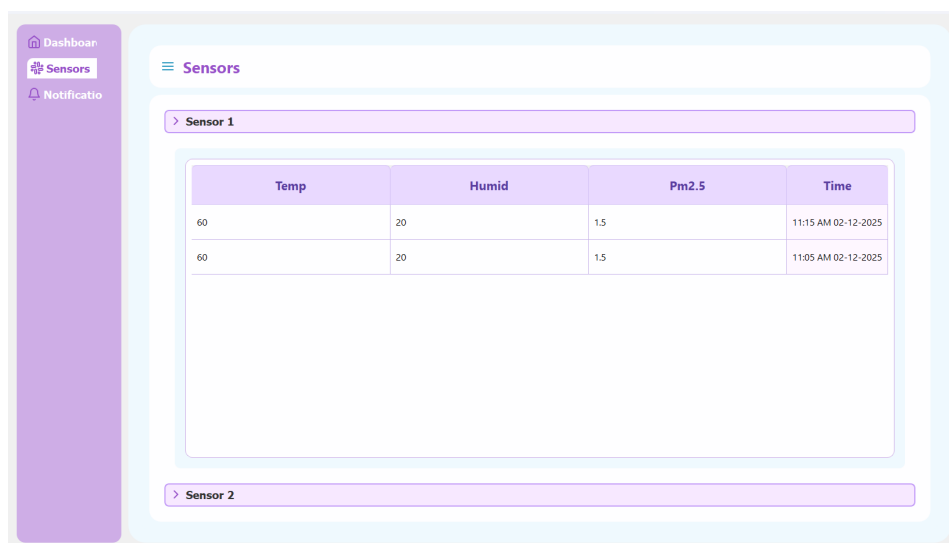
Về mặt cấu trúc, HDMI Dashboard được tổ chức thành nhiều module chức năng nhằm đảm bảo tính rõ ràng và dễ bảo trì. Các thành phần chính bao gồm lớp giao diện chính (Main Window), module hiển thị dữ liệu cảm biến, module cảnh báo hệ thống và module xử lý kết nối dữ liệu. Mỗi module đảm nhiệm một vai trò riêng biệt, trong đó lớp giao diện chính đóng vai trò điều phối và quản lý toàn bộ luồng hiển thị. Dựa trên các ảnh chụp màn hình thực tế, có thể thấy giao diện được chia thành ba khu vực chính: thanh điều hướng (Dashboard, Sensors, Notifications), khu vực hiển thị thông số chính và khu vực điều khiển.

Hình ảnh tổng thể của dashboard được trình bày trong Hình 5.30. Tại đây, các thông số môi trường quan trọng như nhiệt độ (29.00°C), độ ẩm (58.51%) và nồng độ bụi mịn PM2.5 (12.16 $\mu\text{g}/\text{m}^3$) được hiển thị nổi bật dưới dạng các ô thông tin riêng biệt. Bên cạnh đó, khu vực camera được tích hợp nhằm cung cấp luồng hình ảnh trực tiếp từ không gian giám sát. Khu vực Notifications hiển thị danh sách các cảnh báo với số lượng (20) và nút “View All” để xem chi tiết. Phía dưới cùng là bảng điều khiển (Controls) chứa ba nút chức năng (BUTTON 1, BUTTON 2, BUTTON 3), cho phép người dùng tương tác trực tiếp với hệ thống nhúng qua giao diện.



Hình 5.30: Giao diện HDMI Dashboard tổng thể

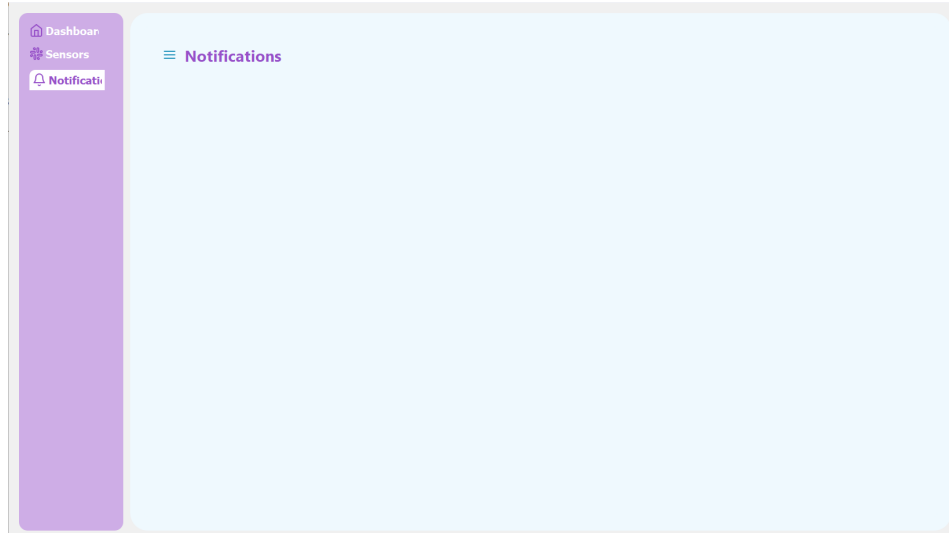
Ngoài giao diện chính, hệ thống còn cung cấp hai màn hình chức năng riêng biệt cho việc xem dữ liệu cảm biến chi tiết và quản lý thông báo. Hình ảnh từ màn hình Sensors (Hình 5.31) cho thấy dữ liệu được trình bày dưới dạng bảng cho từng cảm biến. Ví dụ, Sensor 1 hiển thị các cột gồm nhiệt độ (Temp), độ ẩm (Humid), PM2.5 và thời gian (Time) với các giá trị cụ thể (60, 20, 1.5, 11:15 AM 02-12-2025). Màn hình này cho phép theo dõi lịch sử dữ liệu theo từng mốc thời gian, hỗ trợ đối chiếu và phân tích xu hướng biến đổi của các chỉ số môi trường.



Hình 5.31: Màn hình hiển thị dữ liệu các cảm biến

Màn hình Notifications (Hình 5.32) tập trung vào các thông báo hệ thống, được thiết kế dưới dạng danh sách hoặc các khối tin nhắn cảnh báo. Giao diện này giúp người quản trị nhanh

chóng nắm bắt các sự kiện bất thường như vượt ngưỡng nhiệt độ, độ ẩm, hoặc PM2.5. Mặc dù hình ảnh chỉ hiển thị các tab điều hướng, nhưng trong quá trình hiện thực, module Notifications đã được tích hợp cơ chế lưu trữ và phân loại thông báo theo mức độ ưu tiên.



Hình 5.32: Giao diện tab Notifications

Vận hành tổng thể của dashboard dựa trên cơ chế nhận dữ liệu định kỳ từ backend. Mỗi khi có dữ liệu mới (nhiệt độ, độ ẩm, PM2.5, trạng thái cảm biến), hệ thống sẽ phát tín hiệu (signal) qua kết nối MQTT hoặc HTTP, và dashboard bắt tín hiệu đó để cập nhật trực tiếp lên các widget tương ứng mà không cần tải lại toàn bộ giao diện. Các nút điều khiển (BUTTON 1–3) được gắn kết với các lệnh điều khiển thiết bị (ví dụ bật/tắt quạt, relay, thay đổi chế độ hoạt động) thông qua cơ chế gọi hàm callback. Nhờ vậy, người dùng vừa có thể giám sát thông số, vừa có thể can thiệp từ xa ngay trên dashboard.

Nhìn chung, việc sử dụng PyQt5 kết hợp với kiến trúc module hóa và phân chia rõ ràng các màn hình chức năng đã giúp HDMI Dashboard đạt được sự cân bằng giữa tính linh hoạt trong phát triển, khả năng mở rộng trong tương lai, và đáp ứng yêu cầu giám sát hệ thống nhúng theo thời gian thực.

Chương 6

Kết luận và hướng phát triển

6.1 Kết luận

6.2 Hướng phát triển

6.1 Kết luận

Trong phạm vi đề tài, hệ thống Linux Embedded dựa trên Yocto Project đã được xây dựng và triển khai thành công trên nền tảng Raspberry Pi 4. Hệ thống đáp ứng mục tiêu xây dựng một nền tảng nhúng có khả năng tùy biến cao, tích hợp driver phần cứng và các thành phần IoT phục vụ bài toán giám sát công nghiệp.

Driver RS485 Modbus RTU được hiện thực dưới dạng kernel module và đã hoạt động ổn định trong quá trình thử nghiệm. Cơ chế polling dữ liệu kết hợp Serdev và UART giúp hệ thống thu thập dữ liệu cảm biến theo thời gian thực. Dữ liệu sau khi xử lý được đồng bộ giữa Kernel Space và User Space thông qua character device.

Ở tầng ứng dụng, hệ thống đã tích hợp cơ chế truyền dữ liệu IoT thông qua MQTT, phục vụ việc giám sát từ xa. Giao diện hiển thị được xây dựng để theo dõi trực quan trạng thái hệ thống và dữ liệu cảm biến. Bên cạnh đó, hệ thống có tích hợp một mô-đun phân tích dữ liệu đơn giản (AI-based), nhằm hỗ trợ đánh giá trạng thái và đưa ra cảnh báo dựa trên dữ liệu thu thập được.

Kết quả thực nghiệm cho thấy hệ thống hoạt động ổn định trên Raspberry Pi 4, đảm bảo luồng dữ liệu từ cảm biến → xử lý → truyền IoT → hiển thị được vận hành liên tục trong môi trường Linux Embedded.

6.2 Hướng phát triển

Trong tương lai, hệ thống có thể tiếp tục được mở rộng theo hướng tối ưu hiệu năng driver và giảm độ trễ trong giao tiếp RS485, đặc biệt trong môi trường nhiễu thiết bị.

Về bảo mật, các cơ chế hiện tại có thể được hoàn thiện thêm để đảm bảo an toàn toàn diện hơn, bao gồm xác thực firmware, bảo vệ filesystem và tăng cường kiểm soát truy cập hệ thống.

Đối với phần phân tích dữ liệu, mô-đun AI hiện tại có thể được mở rộng theo hướng sử dụng các mô hình học máy chuyên sâu hơn nhằm nâng cao độ chính xác trong dự đoán và phát hiện bất thường so với phương pháp ngưỡng đang sử dụng.

Cuối cùng, hệ thống IoT có thể được phát triển theo hướng kiến trúc phân tán hoặc tích hợp cloud-native để đáp ứng các bài toán giám sát có quy mô lớn trong thực tế công nghiệp.

Tài liệu tham khảo

- [1] Yocto Project, “Creating a new recipe,” <https://docs.yoctoproject.org/dev-manual/new-recipe.html>, 2026, yocto Project Development Tasks Manual, Accessed: 2026-05-10.
- [2] The Linux Kernel Organization, “Linux kernel documentation,” <https://www.kernel.org/doc/html/latest/>, 2026, accessed: 2026-05-10.
- [3] —, “Linux device drivers documentation,” <https://www.kernel.org/doc/html/latest/driver-api/index.html>, 2026, accessed: 2026-05-10.
- [4] —, “Device tree usage,” <https://www.kernel.org/doc/html/latest/devicetree/usage-model.html>, 2026, accessed: 2026-05-10.
- [5] —, “Linux kernel module programming guide,” <https://tldp.org/LDP/lkmpg/2.6/html/>, 2026, accessed: 2026-05-10.
- [6] Yocto Project, “Kernel_module_autoload,” https://docs.yoctoproject.org/dev/ref-manual/variables.html#term-KERNEL_MODULE_AUTOLOAD, 2026, yocto Project Reference Manual, Accessed: 2026-05-10.
- [7] W. Li and X. Jiang, “Prediction of air pollutant concentrations based on tcn-bilstm-dmattention with stl decomposition,” p. 4665, 2023, accessed: 2026-05-10. [Online]. Available: <https://www.nature.com/misecs/s41598-023-31569-w>
- [8] Y. Wang, S. Tian, and P. Zhang, “Novel spatio-temporal attention causal convolutional neural network for multi-site pm2.5 prediction,” p. 1408370, 2024, accessed: 2026-05-10. [Online]. Available: <https://www.frontiersin.org/journals/environmental-science/misecs/10.3389/fenvs.2024.1408370/full>
- [9] “A recent study on pm2.5 prediction using deep learning models,” 2025, accessed: 2026-05-10. [Online]. Available: <https://www.sciencedirect.com/science/misc/pii/S0048969725022338>